

PetCare — Relatório Técnico do Projeto

Prontuário de saúde para pets — Expo / React Native + TypeScript

1. Visão geral

O PetCare é um aplicativo mobile (Android/iOS/Web via Expo) para gerenciar a saúde de pets: vacinas, consultas, medicamentos, peso, documentos, dados de emergência, busca, backup e modo demonstração. Todos os dados são salvos localmente no dispositivo via AsyncStorage, com uma camada de repositórios já preparada para uma futura migração para SQLite.

2. Stack tecnológica

- **Expo** (SDK ~54) + **React Native** 0.81
- **TypeScript**
- **React Navigation** (native-stack) para navegação entre telas
- **AsyncStorage** para persistência local (chave/valor, dados versionados em JSON)
- **expo-file-system**, **expo-sharing**, **expo-document-picker**, **expo-image-picker**, **expo-print**, **expo-notifications**
- Tema próprio (light/dark) com tokens de cor, espaçamento, raio e tipografia em `src/theme/index.tsx`

3. Funcionalidades por tela

Onboarding (OnboardingScreen)

4 slides explicando os principais recursos do app (cadastro de pets, registros de saúde, lembretes, documentos). Exibido apenas na primeira abertura, com flag persistida em AsyncStorage (onboardingRepository). Botão "Começar" leva ao Dashboard.

Dashboard (DashboardScreen) — Home principal

- Saudação dinâmica ("Bom dia/Boa tarde/Boa noite") + nome do app, logo da marca e toggle de tema claro/escuro.
- Cards de resumo: total de pets, alertas pendentes, remédios ativos.
- Ações rápidas: "Adicionar pet" e "Novo registro" (escolhe o pet automaticamente se houver só um, ou pergunta via Alert se houver vários).
- Seção de alertas prioritários (vacinas atrasadas, retornos, fim de tratamento).
- Seção "Próximos eventos" com botão "Ver mais".
- Seção "Últimos registros": os 3 registros mais recentes de todos os pets, com ícone, título, pet e data.
- Botão "Seus Pets" leva à lista completa (HomeScreen).
- Estado vazio com chamada para cadastrar o primeiro pet.

Lista de Pets (HomeScreen)

Lista todos os pets cadastrados em cards (PetCard), mostrando foto/ícone, nome, espécie/raça/idade, badges de remédios ativos e alertas pendentes. Toque abre o prontuário do pet (PetDetail); botão "+" abre o cadastro de novo pet.

Cadastro/edição de pet (AddPetScreen)

Formulário com foto (galeria/câmera via expo-image-picker), nome, espécie, raça, data de nascimento e ficha médica básica. Usado tanto para criar quanto editar (detecta petId na rota). Salvamento com Button + toast de feedback.

Prontuário do pet (PetDetailScreen)

- Cabeçalho com foto, nome, espécie/raça/idade e ações: compartilhar prontuário (PDF ou texto), editar pet, excluir pet.
- Atalhos rápidos para Vacinas, Peso, Documentos, Emergência, Estatísticas e Remédios.
- Card de ficha médica (castração, alergias, condições crônicas, tipo sanguíneo, veterinário).
- **Linha do tempo** (SectionList agrupada por mês) com todos os eventos: vacinas, consultas, remédios, vermífugos, pesagens, documentos e observações.
- Chips de filtro rápido: Todos / Vacinas / Consultas / Remédios / Vermífugos / Peso / Documentos / Observações.
- Badges de status: "Em dia", "Reforço próximo", "Atrasada", "Reforço aplicado" (vacinas) e "Atrasado" / "Próximo" / "Concluído" / "Retorno realizado" / "Dose aplicada" (demais tipos).
- Toque em um item de registro abre edição; toque longo permite excluir.

Novo registro (AddRecordScreen)

Formulário único e adaptável para os 5 tipos de registro: **vacina** (fabricante, lote, clínica, data do reforço), **consulta** (veterinário, clínica, diagnóstico, data de retorno), **medicamento** (dosagem, frequência contínua/temporária, data de término), **vermífugo** (próxima dose) e **observação** (descrição livre). Suporta lembretes configuráveis (dias antes do evento) e edição de registros existentes.

Peso (WeightScreen / AddWeightScreen)

Histórico de pesagens com gráfico de evolução (WeightChart) e lista de registros (WeightCard). Cadastro/edição de peso (kg) com data.

Vacinas (VaccinesScreen)

"Carteira de vacinação" — lista todas as vacinas do pet com status calculado (em dia / reforço próximo / atrasada / reforço aplicado), data de aplicação e próximo reforço.

Remédios (MedicationsScreen)

- Separa medicamentos em "Em uso" e "Finalizados".
- Badge "Contínuo" ou "Temporário" conforme a frequência.
- Barra de progresso do tratamento (baseada em data de início/término).
- Alerta de "fim do tratamento" quando faltam 3 dias ou menos.
- Botão "Marcar dose tomada" registra a dose (MedicationDose) e mostra "Última dose: há X horas/dias".

Ficha médica (MedicalProfileScreen)

Edição de dados médicos do pet: castrado (sim/não), alergias, condições crônicas, tipo sanguíneo, nome e telefone do veterinário, observações.

Emergência (EmergencyScreen)

Tela de acesso rápido com dados críticos do pet (alergias, condições, tipo sanguíneo, contato do veterinário) e dados do tutor (nome e telefone), editáveis e salvos localmente — pensada para ser mostrada rapidamente em uma emergência.

Documentos (DocumentsScreen / DocumentViewerScreen)

Upload de arquivos (exames, receitas, carteiras de vacinação, outros) via document picker, com título, tipo e data. Visualização integrada do arquivo (DocumentViewerScreen) e exclusão com confirmação.

Estatísticas (StatsScreen)

Resumo estatístico do prontuário do pet: quantidade de registros por tipo, evolução de peso e outros indicadores agregados.

Busca (SearchScreen)

Busca global por título/descrição em todos os registros de todos os pets, com navegação direta ao resultado.

Sobre (AboutScreen)

- Logo, slogan e destaques da marca PetCare.
- **Backup:** exportar dados como JSON (compartilhável), com data do último backup exibida.
- **Restaurar backup:** lê o arquivo antes de aplicar e mostra um resumo (quantos pets/registros tem o backup, data de exportação, e quantos dados atuais serão substituídos) antes de confirmar — ação destrutiva com Alert.
- **Modo demonstração:** botão que substitui os dados atuais por pets e histórico fictícios (src/services/demoData.ts), com confirmação.
- Toggle de tema claro/escuro.

4. Recursos transversais

- **Tema claro/escuro** com paleta de marca PetCare (laranja), aplicado em toda a navegação.
- **Toasts** de feedback ("salvo", "excluído", "dose registrada" etc.) via ToastProvider/useToast.
- **Notificações locais** sincronizadas com os próximos eventos (vacinas, retornos, fim de tratamento) — src/services/notifications.ts.
- **Compartilhamento** do prontuário em PDF ou texto (src/services/pdf.ts / share.ts).
- **Backup/restore** em JSON, com versionamento de cada entidade salva (src/storage/index.ts).
- **Camada de repositórios** (src/repositories/*): hoje são wrappers finos sobre o AsyncStorage, preparados para troca futura por SQLite sem alterar as telas (ver docs/SQLITE_MIGRATION.md).
- **Acessibilidade:** accessibilityRole/accessibilityLabel em botões somente-ícone e hitSlop mínimo de 12px em alvos de toque.
- **Performance:** componentes de lista (PetCard, TimelineItem, ReminderCard, StatCard) memoizados com React.memo; renderers de listas com useCallback.

5. Estrutura de pastas

```
App.tsx
README.md
app.json
docs/SQLITE_MIGRATION.md
index.ts
package.json
src/components/Button.tsx
src/components/EmptyState.tsx
src/components/HealthRecordCard.tsx
src/components/Input.tsx
src/components/MedicalProfileCard.tsx
src/components/PetCard.tsx
src/components/ReminderCard.tsx
src/components/StatCard.tsx
```

```
src/components/ThemeToggle.tsx
src/components/TimelineItem.tsx
src/components/WeightCard.tsx
src/components/WeightChart.tsx
src/hooks/useToast.tsx
src/labels.ts
src/repositories/backupRepository.ts
src/repositories/documentsRepository.ts
src/repositories/medicationsRepository.ts
src/repositories/onboardingRepository.ts
src/repositories/petsRepository.ts
src/repositories/recordsRepository.ts
src/repositories/weightsRepository.ts
src/screens/AboutScreen.tsx
src/screens/AddPetScreen.tsx
src/screens/AddRecordScreen.tsx
src/screens/AddWeightScreen.tsx
src/screens/DashboardScreen.tsx
src/screens/DocumentViewerScreen.tsx
src/screens/DocumentsScreen.tsx
src/screens/EmergencyScreen.tsx
src/screens/HomeScreen.tsx
src/screens/MedicalProfileScreen.tsx
src/screens/MedicationsScreen.tsx
src/screens/OnboardingScreen.tsx
src/screens/PetDetailScreen.tsx
src/screens/SearchScreen.tsx
src/screens/StatsScreen.tsx
src/screens/VaccinesScreen.tsx
src/screens/WeightScreen.tsx
src/services/demoData.ts
src/services/events.ts
src/services/notifications.ts
src/services/pdf.ts
src/services/share.ts
src/storage/files.ts
src/storage/index.ts
src/theme/index.tsx
src/types/index.ts
src/utils/date.ts
tsconfig.json
```

6. Código-fonte completo

A seguir, o código-fonte de todos os arquivos do projeto (configuração, tipos, tema, armazenamento, repositórios, serviços, componentes e telas).

Índice de arquivos

package.json	src/components/HealthRecordCard.tsx
app.json	src/components/Input.tsx
tsconfig.json	src/components/MedicalProfileCard.tsx
index.ts	src/components/PetCard.tsx
App.tsx	src/components/ReminderCard.tsx
README.md	src/components/StatCard.tsx
docs/SQLITE_MIGRATION.md	src/components/ThemeToggle.tsx
src/types/index.ts	src/components/TimelineItem.tsx
src/theme/index.tsx	src/components/WeightCard.tsx
src/labels.ts	src/components/WeightChart.tsx
src/utils/date.ts	src/screens/OnboardingScreen.tsx
src/storage/index.ts	src/screens/DashboardScreen.tsx
src/storage/files.ts	src/screens/HomeScreen.tsx
src/repositories/petsRepository.ts	src/screens/AddPetScreen.tsx
src/repositories/recordsRepository.ts	src/screens/PetDetailScreen.tsx
src/repositories/weightsRepository.ts	src/screens/AddRecordScreen.tsx
src/repositories/documentsRepository.ts	src/screens/AddWeightScreen.tsx
src/repositories/medicationsRepository.ts	src/screens/WeightScreen.tsx
src/repositories/backupRepository.ts	src/screens/VaccinesScreen.tsx
src/repositories/onboardingRepository.ts	src/screens/MedicationsScreen.tsx
src/services/events.ts	src/screens/MedicalProfileScreen.tsx
src/services/notifications.ts	src/screens/EmergencyScreen.tsx
src/services/pdf.ts	src/screens/DocumentsScreen.tsx
src/services/share.ts	src/screens/DocumentViewerScreen.tsx
src/services/demoData.ts	src/screens/StatsScreen.tsx
src/hooks/useToast.tsx	src/screens/SearchScreen.tsx
src/components/Button.tsx	src/screens/AboutScreen.tsx
src/components/EmptyState.tsx	

package.json

```
{
  "name": "petcare",
  "version": "1.0.0",
  "main": "index.ts",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web"
  },
  "dependencies": {
    "@expo/vector-icons": "^15.0.3",
    "@react-native-async-storage/async-storage": "2.2.0",
    "@react-navigation/native": "^6.1.18",
    "@react-navigation/native-stack": "^6.11.0",
    "expo": "~54.0.34",
    "expo-asset": "~12.0.13",
    "expo-constants": "~18.0.13",
    "expo-document-picker": "~14.0.8",
    "expo-file-system": "~19.0.23",
    "expo-image-picker": "~17.0.11",
    "expo-intent-launcher": "~13.0.8",
    "expo-notifications": "~0.32.17",
    "expo-print": "~15.0.8",
    "expo-sharing": "~14.0.8",
    "expo-status-bar": "~3.0.9",
    "react": "19.1.0",
    "react-dom": "19.1.0",
    "react-native": "0.81.5",
    "react-native-safe-area-context": "~5.6.0",
    "react-native-screens": "~4.16.0",
    "react-native-web": "^0.21.0"
  },
  "devDependencies": {
    "@expo/ngrok": "^4.1.3",
    "@types/react": "~19.1.0",
    "typescript": "~5.9.2"
  },
  "private": true
}
```

app.json

```
{
  "expo": {
    "name": "PetCare",
    "slug": "petcare",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "dark",
    "newArchEnabled": true,
    "splash": {
      "image": "./assets/splash-icon.png",
      "resizeMode": "contain",
      "backgroundColor": "#030918"
    },
    "ios": {
      "supportsTablet": true
    },
    "android": {
      "package": "com.proensa.petcare",
      "versionCode": 1,
      "adaptiveIcon": {
        "foregroundImage": "./assets/adaptive-icon.png",
        "backgroundColor": "#030918"
      },
      "edgeToEdgeEnabled": true,
      "predictiveBackGestureEnabled": false,
      "permissions": ["POST_NOTIFICATIONS"]
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
      "expo-asset",
      [
        "expo-notifications",
        {
          "color": "#E66A3A"
        }
      ]
    ]
  }
}
```

tsconfig.json

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true
  }
}
```

index.ts

```
import { registerRootComponent } from 'expo';

import App from './App';

// registerRootComponent calls AppRegistry.registerComponent('main', () => App);
// It also ensures that whether you load the app in Expo Go or in a native build,
// the environment is set up appropriately
registerRootComponent(App);
```

App.tsx

```
import { useEffect, useState } from 'react';
import { View, ActivityIndicator, StyleSheet } from 'react-native';
import { NavigationContainer, DefaultTheme } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { StatusBar } from 'expo-status-bar';
import { ThemeProvider, useTheme } from './src/theme';
import { ToastProvider } from './src/hooks/useToast';
import { onboardingRepository } from './src/repositories/onboardingRepository';
import { RootStackParamList } from './src/types';
import OnboardingScreen from './src/screens/OnboardingScreen';
import DashboardScreen from './src/screens/DashboardScreen';
import HomeScreen from './src/screens/HomeScreen';
import AddPetScreen from './src/screens/AddPetScreen';
import PetDetailScreen from './src/screens/PetDetailScreen';
import AddRecordScreen from './src/screens/AddRecordScreen';
import AddWeightScreen from './src/screens/AddWeightScreen';
import WeightScreen from './src/screens/WeightScreen';
import VaccinesScreen from './src/screens/VaccinesScreen';
import MedicationsScreen from './src/screens/MedicationsScreen';
import MedicalProfileScreen from './src/screens/MedicalProfileScreen';
import EmergencyScreen from './src/screens/EmergencyScreen';
import DocumentsScreen from './src/screens/DocumentsScreen';
import DocumentViewerScreen from './src/screens/DocumentViewerScreen';
import StatsScreen from './src/screens/StatsScreen';
import SearchScreen from './src/screens/SearchScreen';
import AboutScreen from './src/screens/AboutScreen';

const Stack = createNativeStackNavigator<RootStackParamList>();

function Root({ initialRouteName }: { initialRouteName: keyof RootStackParamList }) {
  const { colors, scheme } = useTheme();

  const navTheme = {
    ...DefaultTheme,
    dark: scheme === 'dark',
    colors: {
      ...DefaultTheme.colors,
      primary: colors.primary,
      background: colors.background,
      card: colors.background,
      text: colors.text,
      border: colors.border,
    },
  };

  return (
    <NavigationContainer theme={navTheme}>
      <StatusBar style={scheme === 'dark' ? 'light' : 'dark'} />
      <Stack.Navigator initialRouteName={initialRouteName} screenOptions={{ headerShown: false }}>
        <Stack.Screen name="Onboarding" component={OnboardingScreen} />
        <Stack.Screen name="Dashboard" component={DashboardScreen} />
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="AddPet" component={AddPetScreen} />
        <Stack.Screen name="PetDetail" component={PetDetailScreen} />
        <Stack.Screen name="AddRecord" component={AddRecordScreen} />
        <Stack.Screen name="AddWeight" component={AddWeightScreen} />
        <Stack.Screen name="Weight" component={WeightScreen} />
        <Stack.Screen name="Vaccines" component={VaccinesScreen} />
        <Stack.Screen name="Medications" component={MedicationsScreen} />
        <Stack.Screen name="MedicalProfile" component={MedicalProfileScreen} />
        <Stack.Screen name="Emergency" component={EmergencyScreen} />
        <Stack.Screen name="Documents" component={DocumentsScreen} />
        <Stack.Screen name="DocumentViewer" component={DocumentViewerScreen} />
        <Stack.Screen name="Stats" component={StatsScreen} />
        <Stack.Screen name="Search" component={SearchScreen} />
        <Stack.Screen name="About" component={AboutScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

function LoadingScreen() {
  const { colors } = useTheme();
  return (
    <View style={[styles.loading, { backgroundColor: colors.background }]}>
      <ActivityIndicator color={colors.primary} size="large" />
    </View>
  );
}
```

```

    });
  }

function AppContent() {
  const [onboardingSeen, setOnboardingSeen] = useState<boolean | null>(null);

  useEffect(() => {
    onboardingRepository.getSeen()
      .then(setOnboardingSeen)
      .catch(() => setOnboardingSeen(true));
  }, []);

  if (onboardingSeen === null) return <LoadingScreen />;

  return <Root initialRouteName={onboardingSeen ? 'Dashboard' : 'Onboarding'} />;
}

export default function App() {
  return (
    <ThemeProvider>
      <ToastProvider>
        <AppContent />
      </ToastProvider>
    </ThemeProvider>
  );
}

const styles = StyleSheet.create({
  loading: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
});

```

README.md

PetCare 🐾

App de prontuário de saúde para pets, feito com **Expo / React Native + TypeScript**. Centraliza vacinas, consultas, medicamentos, peso, documentos e contatos de emergência de todos os seus pets em um só lugar – com lembretes, backup e modo offline (tudo guardado no dispositivo).

Funcionalidades

- **Onboarding**: inicial explicando os principais recursos do app.
- **Dashboard**: saudação dinâmica, resumo (pets, alertas, remédios ativos), acesso rápido para cadastrar pet / novo registro, alertas prioritários, próximos eventos e últimos registros de todos os pets.
- **Prontuário por pet**: linha do tempo agrupada por mês, com filtros por tipo (vacinas, consultas, remédios, vermífugos, peso, documentos, observações) e indicadores de status (atrasado, próximo, concluído, em dia).
- **Vacinas**: carteira de vacinação com status (em dia, reforço próximo, atrasada, reforço aplicado).
- **Medicamentos**: controle de doses, marcação de "dose tomada", barra de progresso do tratamento, badge contínuo/temporário e alerta de fim de tratamento.
- **Peso**: histórico e gráfico de evolução.
- **Documentos**: exames, receitas e carteiras de vacinação digitalizados, com visualização integrada.
- **Emergência**: dados do pet, do tutor e contatos importantes sempre à mão.
- **Busca**: global por registros.
- **Backup**: exportar/importar backup em JSON, com resumo de conteúdo antes de restaurar e data do último backup.
- **Modo demonstração**: carrega pets e histórico fictícios para explorar o app rapidamente.
- **Temas**: claro e escuro, com identidade visual própria da marca PetCare.

Stack

- [Expo](https://expo.dev) + React Native
- TypeScript
- React Navigation (native-stack)
- AsyncStorage para persistência local (veja docs/SQLITE_MIGRATION.md para o plano de migração para SQLite)

Como rodar

```
``bash
npm install
npx expo start
``
```

Abra no Expo Go (Android/iOS) escaneando o QR code, ou pressione `w` para abrir no navegador.

Modo demonstração

Na tela **Sobre**, use o botão "Carregar dados de demonstração" para preencher o app com pets fictícios e histórico de exemplo (substitui os dados atuais – peça confirmação antes).

Screenshots

Pendente: capturas de tela do app em uso. Veja docs/screenshots.

Melhorias futuras

- Migração de AsyncStorage para SQLite (plano completo em docs/SQLITE_MIGRATION.md).
- Sincronização em nuvem / backup automático.
- Suporte a múltiplos tutores por pet.
- Notificações push remotas (hoje as notificações são locais).

docs/SQLITE_MIGRATION.md

Plano de migração: AsyncStorage → SQLite

Contexto

Hoje todo o estado do PetCare é persistido no `AsyncStorage` via `src/storage/index.ts`, em chaves separadas por entidade (`@petcare:pets`, `@petcare:records`, etc.), cada uma guardando um JSON `{ version, data: T[] }`. Isso funciona bem para o volume atual de dados, mas tem limitações que aparecem conforme o histórico cresce:

- Toda leitura de uma entidade desserializa o array inteiro (sem índices, sem queries).
- Relacionamentos (pet → registros → doses de remédio) são resolvidos em memória, filtrando arrays a cada acesso.
- Não há transações reais: `importBackup` sobrescreve várias chaves em sequência.

Este documento descreve **como migrar para SQLite** (via `expo-sqlite`) sem reescrever as telas, aproveitando a camada de repositórios criada em `src/repositories/`.

Por que a camada de repositórios já ajuda

As telas tocadas nesta sessão (`DashboardScreen`, `PetDetailScreen`, `MedicationsScreen`, `AboutScreen`, `OnboardingScreen`, `App.tsx`) não chamam mais `src/storage/index.ts` diretamente – chamam objetos em `src/repositories/\*.ts` (`petsRepository`, `recordsRepository`, `weightsRepository`, `documentsRepository`, `medicationsRepository`, `backupRepository`, `onboardingRepository`), que hoje apenas reexportam as funções do `storage`. Quando a implementação trocar para SQLite, **só esses módulos precisam mudar** – as telas continuam chamando `petsRepository.getAll()` etc.

As demais telas ainda importam de `../storage` diretamente. A migração pode ser feita em fases: primeiro trocar a implementação por trás dos repositórios (cobrindo as telas já migradas), depois ir criando/atualizando repositórios para as telas restantes conforme forem sendo tocadas.

Schema relacional proposto

```
```sql
CREATE TABLE pets (
 id TEXT PRIMARY KEY,
 name TEXT NOT NULL,
 species TEXT NOT NULL,
 breed TEXT,
 birth_date TEXT,
 photo TEXT,
 created_at TEXT NOT NULL
);

CREATE TABLE medical_profiles (
 pet_id TEXT PRIMARY KEY REFERENCES pets(id) ON DELETE CASCADE,
 neutered INTEGER,
 allergies TEXT,
 chronic_conditions TEXT,
 blood_type TEXT,
 vet_name TEXT,
 vet_phone TEXT,
 notes TEXT
);

CREATE TABLE records (
 id TEXT PRIMARY KEY,
 pet_id TEXT NOT NULL REFERENCES pets(id) ON DELETE CASCADE,
 type TEXT NOT NULL,
 date TEXT NOT NULL,
 title TEXT NOT NULL,
 description TEXT,
 next_date TEXT,
 end_date TEXT,
 frequency TEXT,
 dosage TEXT,
 manufacturer TEXT,
 batch TEXT,
 clinic TEXT,
 vet TEXT,
 diagnosis TEXT,
 reminder_days TEXT, -- JSON array serializado
 created_at TEXT NOT NULL
);

CREATE INDEX idx_records_pet ON records(pet_id);
CREATE INDEX idx_records_pet_type ON records(pet_id, type);
```

```

CREATE TABLE weights (
 id TEXT PRIMARY KEY,
 pet_id TEXT NOT NULL REFERENCES pets(id) ON DELETE CASCADE,
 date TEXT NOT NULL,
 weight_kg REAL NOT NULL,
 created_at TEXT NOT NULL
);
CREATE INDEX idx_weights_pet ON weights(pet_id);

CREATE TABLE documents (
 id TEXT PRIMARY KEY,
 pet_id TEXT NOT NULL REFERENCES pets(id) ON DELETE CASCADE,
 title TEXT NOT NULL,
 kind TEXT NOT NULL,
 date TEXT NOT NULL,
 uri TEXT NOT NULL,
 mime_type TEXT,
 created_at TEXT NOT NULL
);
CREATE INDEX idx_documents_pet ON documents(pet_id);

CREATE TABLE medication_doses (
 id TEXT PRIMARY KEY,
 record_id TEXT NOT NULL REFERENCES records(id) ON DELETE CASCADE,
 pet_id TEXT NOT NULL REFERENCES pets(id) ON DELETE CASCADE,
 date TEXT NOT NULL,
 created_at TEXT NOT NULL
);
CREATE INDEX idx_doses_record ON medication_doses(record_id);

CREATE TABLE tutor_info (
 id INTEGER PRIMARY KEY CHECK (id = 1), -- linha única
 name TEXT,
 phone TEXT
);

CREATE TABLE app_meta (
 key TEXT PRIMARY KEY,
 value TEXT
); -- onboarding_seen, last_backup, schema_version, migrated_v3 etc.
...

Estratégia de migração de dados existentes

1. **Detectar versão atual**: ao abrir o app, checar `app_meta['migrated_v3']`. Se ausente,
 executar a migração one-shot abaixo (reaproveita o padrão de versionamento já usado em
 `src/storage/index.ts`, onde cada entidade tem um número de versão).
2. **Ler os JSONs atuais do AsyncStorage** – exatamente como `exportBackup()` já faz hoje:
 `getPets()`, `getAllRecords()`, `getWeights` por pet, `getDocuments` por pet,
 `getMedicationDoses` por pet, `getTutorInfo()`.
3. **Inserir nas tabelas relacionais** em uma única transação SQLite (`db.withTransactionAsync`),
 mapeando cada `Pet.medicalProfile` para uma linha em `medical_profiles` e cada
 `MedicalRecord.reminderDays` (array) para uma string JSON na coluna `reminder_days`.
4. **Marcar `app_meta['migrated_v3'] = '1'` ao final. Não apagar as chaves antigas do
 AsyncStorage imediatamente – mantê-las por uma versão como fallback/rollback, removendo
 em uma limpeza posterior.
5. **Backup/restore continuam funcionando**: `exportBackup`/`importBackup` passam a ler/
 escrever via SQL em vez de AsyncStorage, mas o formato do arquivo JSON exportado
 (`Backup` interface) não muda – compatibilidade com backups antigos é preservada.

O que precisa trocar de implementação

Apenas os módulos em `src/repositories`:

- `petsRepository` → `getAll`/`save`/`remove` passam a executar `SELECT`/`INSERT OR
 REPLACE`/`DELETE` em `pets` (+ `medical_profiles`).
- `recordsRepository` → `getByPet`/`getAll`/`save`/`remove` operam em `records`.
- `weightsRepository`, `documentsRepository`, `medicationsRepository` → idem para suas
 tabelas.
- `backupRepository` → `export`/`import` fazem `SELECT *` de todas as tabelas / `INSERT`
 em transação, mantendo o mesmo formato `Backup` JSON.
- `onboardingRepository` → `getSeen`/`setSeen` leem/escrevem `app_meta`.

As telas não mudam. Os repositórios mantêm a mesma assinatura (`Promise<T[]>`,
`Promise<void>`, etc.), então `DashboardScreen`, `PetDetailScreen`, `MedicationsScreen`,
`AboutScreen`, `OnboardingScreen` e `App.tsx` continuam funcionando sem alterações.

Telas ainda não migradas para repositórios

`HomeScreen`, `AddPetScreen`, `AddRecordScreen`, `AddWeightScreen`, `WeightScreen`,

```

```
`VaccinesScreen`, `MedicalProfileScreen`, `EmergencyScreen`, `DocumentsScreen`,
`DocumentViewerScreen`, `StatsScreen`, `SearchScreen` continuam importando de
`../storage`. Para completar a migração, cada uma delas precisa trocar seus imports para
os repositórios equivalentes (todos já existem ou são extensões simples dos atuais) antes
da troca de implementação – caso contrário, continuariam lendo do AsyncStorage enquanto o
resto do app já estaria em SQLite.
```

## Ordem recomendada

1. Adicionar `expo-sqlite` e o módulo de schema/migração one-shot descrito acima.
2. Trocar a implementação dos repositórios já existentes (sem tocar nas telas).
3. Migrar as telas restantes para repositórios, tela por tela.
4. Após validar em produção por uma versão, remover as chaves antigas do AsyncStorage e o código de migração one-shot.

## src/types/index.ts

```
// Chaves estáveis persistidas no storage – a tradução para exibição
// fica em src/labels.ts. Não renomear sem escrever migração.
export type Species = 'dog' | 'cat' | 'bird' | 'other';
export type RecordType = 'vaccine' | 'consultation' | 'medication' | 'deworming' | 'note';
export type Frequency = 'once_daily' | 'twice_daily' | 'every_8h' | 'every_12h' | 'continuous';
export type DocumentKind = 'exam' | 'prescription' | 'vaccination_card' | 'other';

export interface MedicalProfile {
 neutered?: boolean;
 allergies?: string;
 chronicConditions?: string;
 bloodType?: string;
 vetName?: string;
 vetPhone?: string;
 notes?: string;
}

export interface Pet {
 id: string;
 name: string;
 species: Species;
 breed: string;
 birthDate: string;
 photo?: string;
 medicalProfile?: MedicalProfile;
 createdAt: string;
}

export interface MedicalRecord {
 id: string;
 petId: string;
 type: RecordType;
 /** Data principal: aplicação, consulta, início do tratamento ou da observação. */
 date: string;
 /** Nome da vacina, motivo da consulta, nome do remédio ou título da observação. */
 title: string;
 /** Observações livres / descrição da observação. */
 description?: string;
 /** Reforço (vacina), retorno (consulta) ou próxima dose (vermífugo). */
 nextDate?: string;
 /** Fim do tratamento (medicamento). */
 endDate?: string;
 frequency?: Frequency;
 /** Dosagem do medicamento (ex: "250mg, 1 comprimido"). */
 dosage?: string;
 manufacturer?: string;
 batch?: string;
 clinic?: string;
 vet?: string;
 diagnosis?: string;
 /** Dias de antecedência escolhidos para lembrar do próximo evento. */
 reminderDays?: number[];
 createdAt: string;
}

export interface WeightEntry {
 id: string;
 petId: string;
 date: string;
 weightKg: number;
 createdAt: string;
}

export interface PetDocument {
 id: string;
 petId: string;
 title: string;
 kind: DocumentKind;
 date: string;
 uri: string;
 mimeType?: string;
 createdAt: string;
}

export interface TutorInfo {
 name: string;
 phone: string;
}
```

```
}

export interface MedicationDose {
 id: string;
 recordId: string;
 petId: string;
 date: string;
 createdAt: string;
}

export type RootStackParamList = {
 Onboarding: undefined;
 Dashboard: undefined;
 Home: undefined;
 AddPet: { petId?: string };
 PetDetail: { petId: string };
 AddRecord: {
 petId: string;
 recordId?: string;
 initialType?: RecordType;
 prefill?: { title?: string; manufacturer?: string; batch?: string; clinic?: string };
 };
 AddWeight: { petId: string; weightId?: string };
 Weight: { petId: string };
 Vaccines: { petId: string };
 Medications: { petId: string };
 MedicalProfile: { petId: string };
 Emergency: { petId: string };
 Documents: { petId: string; kind?: DocumentKind };
 DocumentViewer: { uri: string; title: string; mimeType?: string };
 Stats: { petId: string };
 Search: undefined;
 About: undefined;
};
```

## src/theme/index.tsx

```
import React, {
 createContext,
 useContext,
 useEffect,
 useMemo,
 useState,
} from 'react';
import { StyleSheet } from 'react-native';
import AsyncStorage from '@react-native-async-storage/async-storage';

export type ThemeScheme = 'dark' | 'light';

export interface Palette {
 background: string;
 surface: string;
 surfaceElevated: string;
 primary: string;
 /** Cor de destaque para ícones e links sobre o fundo. */
 primaryLight: string;
 info: string;
 success: string;
 warning: string;
 danger: string;
 accent: string;
 text: string;
 textMuted: string;
 textSubtle: string;
 border: string;
 /** Cor do texto/ícone sobre fundo `primary` (botões, badges sólidos). */
 onPrimary: string;
}

export const palettes: Record<ThemeScheme, Palette> = {
 // Paleta oficial da marca PetCare:
 // laranja #E66A3A / #F59E0B / #FFB27A · navy #1B2940 / #0B1120 · off-white #F8FAFC
 dark: {
 background: '#0B1120',
 surface: '#1B2940',
 surfaceElevated: '#26364E',
 primary: '#E66A3A',
 primaryLight: '#FFB27A',
 info: '#38BDF8',
 success: '#10B981',
 warning: '#F59E0B',
 danger: '#EF4444',
 accent: '#A78BFA',
 text: '#F8FAFC',
 textMuted: '#64748B',
 textSubtle: '#94A3B8',
 border: '#2A3B55',
 onPrimary: 'FFFFFF',
 },
 light: {
 background: '#F8FAFC',
 surface: 'FFFFFF',
 surfaceElevated: '#EEF2F7',
 primary: '#E66A3A',
 primaryLight: '#C2410C',
 info: '#0369A1',
 success: '#0E9F6E',
 warning: '#B45309',
 danger: '#DC2626',
 accent: '#7C3AED',
 text: '#1B2940',
 textMuted: '#64748B',
 textSubtle: '#475569',
 border: '#E2E8F0',
 onPrimary: 'FFFFFF',
 },
};

export const spacing = {
 xs: 4,
 sm: 8,
 md: 16,
 lg: 24,
 xl: 32,
};
```

```

 xl: 48,
 } as const;

export const radius = {
 sm: 8,
 md: 12,
 lg: 16,
 xl: 24,
 full: 9999,
} as const;

/** Escala tipográfica única – tamanhos e pesos padrão para títulos e textos. */
export const typography = {
 h1: { fontSize: 28, fontWeight: '700' as const },
 h2: { fontSize: 22, fontWeight: '700' as const },
 h3: { fontSize: 18, fontWeight: '600' as const },
 h4: { fontSize: 16, fontWeight: '600' as const },
 body: { fontSize: 15, fontWeight: '400' as const },
 label: { fontSize: 13, fontWeight: '600' as const },
 caption: { fontSize: 12, fontWeight: '400' as const },
 tiny: { fontSize: 11, fontWeight: '400' as const },
} as const;

const THEME_KEY = '@petcare:theme';

interface ThemeContextValue {
 scheme: ThemeScheme;
 colors: Palette;
 toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextValue>({
 scheme: 'dark',
 colors: palettes.dark,
 toggleTheme: () => {},
});

export function ThemeProvider({ children }: { children: React.ReactNode }) {
 const [scheme, setScheme] = useState<ThemeScheme>('dark');

 useEffect(() => {
 AsyncStorage.getItem(THEME_KEY)
 .then(saved => {
 if (saved === 'light' || saved === 'dark') setScheme(saved);
 })
 .catch(() => {});
 }, []);

 const value = useMemo<ThemeContextValue>(
 () => ({
 scheme,
 colors: palettes[scheme],
 toggleTheme: () => {
 setScheme(prev => {
 const next: ThemeScheme = prev === 'dark' ? 'light' : 'dark';
 AsyncStorage.setItem(THEME_KEY, next).catch(() => {});
 return next;
 }),
 },
 [scheme],
)
);

 return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}

export function useTheme(): ThemeContextValue {
 return useContext(ThemeContext);
}

/** Memoiza um StyleSheet construído a partir da paleta atual. */
export function useThemedStyles<T extends StyleSheet.NamedStyles<T>>(
 factory: (colors: Palette) => T,
): T {
 const { colors } = useTheme();
 return useMemo(() => factory(colors), [colors, factory]);
}

```

## src/labels.ts

```
import { Ionicons } from '@expo/vector-icons';
import { Species, RecordType, Frequency, DocumentKind } from './types';
import { Palette } from './theme';

export const SPECIES_LABELS: Record<Species, string> = {
 dog: 'Cão',
 cat: 'Gato',
 bird: 'Pássaro',
 other: 'Outro',
};

export const RECORD_TYPE_LABELS: Record<RecordType, string> = {
 vaccine: 'Vacina',
 consultation: 'Consulta',
 medication: 'Remédio',
 deworming: 'Vermífugo',
 note: 'Observação',
};

export function recordTypeColors(colors: Palette): Record<RecordType, string> {
 return {
 vaccine: colors.success,
 consultation: colors.info,
 medication: colors.warning,
 deworming: colors.accent,
 note: colors.textSubtle,
 };
}

export const RECORD_TYPE_ICONS: Record<RecordType, keyof typeof Ionicons.glyphMap> = {
 vaccine: 'shield-checkmark',
 consultation: 'medical',
 medication: 'medkit',
 deworming: 'bug',
 note: 'document-text',
};

export const FREQUENCY_LABELS: Record<Frequency, string> = {
 once_daily: '1x ao dia',
 twice_daily: '2x ao dia',
 every_8h: 'A cada 8h',
 every_12h: 'A cada 12h',
 continuous: 'Uso contínuo',
};

export const DOCUMENT_KIND_LABELS: Record<DocumentKind, string> = {
 exam: 'Exame',
 prescription: 'Receita',
 vaccination_card: 'Carteirinha',
 other: 'Outro',
};

export const DOCUMENT_KIND_ICONS: Record<DocumentKind, keyof typeof Ionicons.glyphMap> = {
 exam: 'document-text',
 prescription: 'receipt',
 vaccination_card: 'shield-checkmark',
 other: 'document',
};

/** Opções de antecedência para lembretes (em dias antes do evento). */
export const REMINDER_OPTIONS: { days: number; label: string }[] = [
 { days: 0, label: 'No dia' },
 { days: 1, label: '1 dia antes' },
 { days: 3, label: '3 dias antes' },
 { days: 7, label: '7 dias antes' },
 { days: 15, label: '15 dias antes' },
 { days: 30, label: '30 dias antes' },
];
```

## src/utils/date.ts

```
// Utilitários de data compartilhados pelos formulários e telas de detalhe.
// Datas são digitadas como DD/MM/AAAA e persistidas como ISO (AAAA-MM-DD).

export function maskDate(text: string): string {
 const digits = text.replace(/\D/g, '').slice(0, 8);
 if (digits.length <= 2) return digits;
 if (digits.length <= 4) return `${digits.slice(0, 2)}/${digits.slice(2)}`;
 return `${digits.slice(0, 2)}/${digits.slice(2, 4)}/${digits.slice(4)}`;
}

export function isValidDate(d: string): boolean {
 if (d.length < 10) return false;
 const [day, month, year] = d.split('/').map(Number);
 if (!day || !month || !year || year < 1900 || year > 2100) return false;
 const date = new Date(year, month - 1, day);
 return date.getMonth() === month - 1;
}

/** Data BR (DD/MM/AAAA) é posterior ao dia de hoje? */
export function isFuture(d: string): boolean {
 const [day, month, year] = d.split('/').map(Number);
 const now = new Date();
 const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
 return new Date(year, month - 1, day).getTime() > today.getTime();
}

export function toISO(d: string): string {
 const [day, month, year] = d.split('/');
 return `${year}-${month}-${day}`;
}

export function displayDate(iso: string): string {
 if (!iso) return '';
 const [y, m, d] = iso.split('-');
 return `${d}/${m}/${y}`;
}

/** Dias entre hoje e uma data ISO (negativo = já passou). */
export function daysUntilISO(iso: string): number {
 const [y, m, d] = iso.split('-').map(Number);
 const now = new Date();
 const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
 return Math.round((new Date(y, m - 1, d).getTime() - today.getTime()) / 86400000);
}

/** "hoje", "amanhã", "em 5 dias", "há 3 dias". */
export function formatDaysUntil(days: number): string {
 if (days === 0) return 'hoje';
 if (days === 1) return 'amanhã';
 if (days === -1) return 'há 1 dia';
 if (days > 1) return `em ${days} dias`;
 return `há ${-days} dias`;
}

export function calcAge(iso: string): string {
 if (!iso) return '';
 const birth = new Date(iso);
 const now = new Date();
 if (birth > now) return '';
 let years = now.getFullYear() - birth.getFullYear();
 let months = now.getMonth() - birth.getMonth();
 if (months < 0 || (months === 0 && now.getDate() < birth.getDate())) {
 years--;
 months += 12;
 }
 if (years === 0 && months === 0) return 'recém-nascido';
 if (years === 0) return `${months} ${months === 1 ? 'mês' : 'meses'}`;
 if (months === 0) return `${years} ${years === 1 ? 'ano' : 'anos'}`;
 return `${years} ${years === 1 ? 'ano' : 'anos'} e ${months} ${months === 1 ? 'mês' : 'meses'}`;
}
```

## src/storage/index.ts

```
import AsyncStorage from '@react-native-async-storage/async-storage';
import { Platform } from 'react-native';
import * as FileSystem from 'expo-file-system/legacy';
import {
 Pet,
 MedicalRecord,
 WeightEntry,
 PetDocument,
 TutorInfo,
 MedicationDose,
 Species,
 RecordType,
} from '../types';
import { deletePhoto, deleteDocumentFile } from '../files';
import { buildDemoData } from '../services/demoData';

const KEYS = {
 PETS: '@petcare:pets',
 RECORDS: '@petcare:records',
 WEIGHTS: '@petcare:weights',
 DOCUMENTS: '@petcare:documents',
 TUTOR: '@petcare:tutor',
 MEDICATION_DOSES: '@petcare:medication_doses',
 ONBOARDING_SEEN: '@petcare:onboarding_seen',
 LAST_BACKUP: '@petcare:last_backup',
};

const PETS_VERSION = 1;
const RECORDS_VERSION = 2;
const WEIGHTS_VERSION = 1;
const DOCUMENTS_VERSION = 1;
const MEDICATION_DOSES_VERSION = 1;

interface Envelope<T> {
 version: number;
 data: T[];
}

// -----
// Migrações
// v0: array puro com espécie/tipo em português
// v1 (records): campo único `description` + `nextDate` genérico
// v2 (records): `title` obrigatório + campos específicos por tipo
// -----

const LEGACY_SPECIES: Record<string, Species> = {
 'Cão': 'dog',
 'Gato': 'cat',
 'Pássaro': 'bird',
 'Outro': 'other',
};

const LEGACY_RECORD_TYPES: Record<string, RecordType> = {
 'Vacina': 'vaccine',
 'Consulta': 'consultation',
 'Remédio': 'medication',
};

function migratePetV0(pet: Pet & { species: string }): Pet {
 return { ...pet, species: LEGACY_SPECIES[pet.species] ?? (pet.species as Species) };
}

function migrateRecordV0(record: MedicalRecord & { type: string }): MedicalRecord {
 return { ...record, type: LEGACY_RECORD_TYPES[record.type] ?? (record.type as RecordType) };
}

interface RecordV1 {
 id: string;
 petId: string;
 type: RecordType;
 date: string;
 description: string;
 nextDate?: string;
 createdAt: string;
}

function migrateRecordV1(record: RecordV1): MedicalRecord {
```

```

const { description, nextDate, ...base } = record;
const migrated: MedicalRecord = { ...base, title: description || 'Registro' };
if (nextDate) {
 if (record.type === 'medication') {
 migrated.endDate = nextDate;
 } else {
 migrated.nextDate = nextDate;
 }
}
return migrated;
}

// -----
// Leitura/escrita com envelope versionado
// Não engole erros: se os dados estiverem corrompidos, é melhor falhar
// (e a tela avisar) do que retornar [] e deixar um save sobrescrever tudo.
// -----

async function write(key: string, version: number, data: unknown[]): Promise<void> {
 const envelope: Envelope<unknown> = { version, data };
 await AsyncStorage.setItem(key, JSON.stringify(envelope));
}

function parseRaw<T>(raw: string): { version: number; data: T[] } {
 const parsed = JSON.parse(raw) as T[] | Envelope<T>;
 if (Array.isArray(parsed)) return { version: 0, data: parsed };
 return { version: parsed.version, data: parsed.data };
}

async function readPets(): Promise<Pet[]> {
 const raw = await AsyncStorage.getItem(KEYS.PETS);
 if (!raw) return [];
 const { version, data } = parseRaw<Pet>(raw);
 if (version === 0) {
 const migrated = data.map(migratePetV0);
 await write(KEYS.PETS, PETS_VERSION, migrated);
 return migrated;
 }
 return data;
}

async function readRecords(): Promise<MedicalRecord[]> {
 const raw = await AsyncStorage.getItem(KEYS.RECORDS);
 if (!raw) return [];
 let { version, data } = parseRaw<MedicalRecord>(raw);
 const original = version;
 if (version === 0) {
 data = data.map(migrateRecordV0);
 version = 1;
 }
 if (version === 1) {
 data = (data as unknown as RecordV1[]).map(migrateRecordV1);
 version = 2;
 }
 if (version !== original) {
 await write(KEYS.RECORDS, RECORDS_VERSION, data);
 }
 return data;
}

async function readWeights(): Promise<WeightEntry[]> {
 const raw = await AsyncStorage.getItem(KEYS.WEIGHTS);
 if (!raw) return [];
 return parseRaw<WeightEntry>(raw).data;
}

async function readDocuments(): Promise<PetDocument[]> {
 const raw = await AsyncStorage.getItem(KEYS.DOCUMENTS);
 if (!raw) return [];
 return parseRaw<PetDocument>(raw).data;
}

async function readMedicationDoses(): Promise<MedicationDose[]> {
 const raw = await AsyncStorage.getItem(KEYS.MEDICATION_DOSES);
 if (!raw) return [];
 return parseRaw<MedicationDose>(raw).data;
}

function upsert<T extends { id: string }>(list: T[], item: T): T[] {
 const idx = list.findIndex(x => x.id === item.id);
 if (idx >= 0) {

```

```

 const copy = [...list];
 copy[idx] = item;
 return copy;
 }
 return [...list, item];
}

// ----- Pets -----

export async function getPets(): Promise<Pet[]> {
 return readPets();
}

export async function savePet(pet: Pet): Promise<void> {
 const pets = await readPets();
 await write(KEYS.PETS, PETS_VERSION, upsert(pets, pet));
}

export async function deletePet(petId: string): Promise<void> {
 const pets = await readPets();
 await deletePhoto(pets.find(p => p.id === petId)?.photo);
 await write(KEYS.PETS, PETS_VERSION, pets.filter(p => p.id !== petId));

 const records = await readRecords();
 await write(KEYS.RECORDS, RECORDS_VERSION, records.filter(r => r.petId !== petId));

 const weights = await readWeights();
 await write(KEYS.WEIGHTS, WEIGHTS_VERSION, weights.filter(w => w.petId !== petId));

 const documents = await readDocuments();
 const toDelete = documents.filter(d => d.petId === petId);
 for (const doc of toDelete) {
 await deleteDocumentFile(doc.uri);
 }
 await write(KEYS.DOCUMENTS, DOCUMENTS_VERSION, documents.filter(d => d.petId !== petId));
}

// ----- Registros -----

export async function getAllRecords(): Promise<MedicalRecord[]> {
 return readRecords();
}

export async function getRecords(petId: string): Promise<MedicalRecord[]> {
 const all = await readRecords();
 return all.filter(r => r.petId === petId).sort((a, b) => b.date.localeCompare(a.date));
}

export async function saveRecord(record: MedicalRecord): Promise<void> {
 const records = await readRecords();
 await write(KEYS.RECORDS, RECORDS_VERSION, upsert(records, record));
}

export async function deleteRecord(recordId: string): Promise<void> {
 const records = await readRecords();
 await write(KEYS.RECORDS, RECORDS_VERSION, records.filter(r => r.id !== recordId));
}

// ----- Peso -----

export async function getWeights(petId: string): Promise<WeightEntry[]> {
 const all = await readWeights();
 return all.filter(w => w.petId === petId).sort((a, b) => b.date.localeCompare(a.date));
}

export async function saveWeight(entry: WeightEntry): Promise<void> {
 const weights = await readWeights();
 await write(KEYS.WEIGHTS, WEIGHTS_VERSION, upsert(weights, entry));
}

export async function deleteWeight(weightId: string): Promise<void> {
 const weights = await readWeights();
 await write(KEYS.WEIGHTS, WEIGHTS_VERSION, weights.filter(w => w.id !== weightId));
}

// ----- Documentos -----

export async function getDocuments(petId: string): Promise<PetDocument[]> {
 const all = await readDocuments();
 return all.filter(d => d.petId === petId).sort((a, b) => b.date.localeCompare(a.date));
}

```

```

export async function saveDocument(doc: PetDocument): Promise<void> {
 const documents = await readDocuments();
 await write(KEYS.DOCUMENTS, DOCUMENTS_VERSION, upsert(documents, doc));
}

export async function deleteDocument(documentId: string): Promise<void> {
 const documents = await readDocuments();
 await deleteDocumentFile(documents.find(d => d.id === documentId)?.uri);
 await write(KEYS.DOCUMENTS, DOCUMENTS_VERSION, documents.filter(d => d.id !== documentId));
}

// ----- Doses de remédio -----

export async function getMedicationDoses(petId: string): Promise<MedicationDose[]> {
 const all = await readMedicationDoses();
 return all.filter(d => d.petId === petId).sort((a, b) => b.date.localeCompare(a.date));
}

export async function saveMedicationDose(dose: MedicationDose): Promise<void> {
 const doses = await readMedicationDoses();
 await write(KEYS.MEDICATION_DOSES, MEDICATION_DOSES_VERSION, upsert(doses, dose));
}

export async function deleteMedicationDose(doseId: string): Promise<void> {
 const doses = await readMedicationDoses();
 await write(KEYS.MEDICATION_DOSES, MEDICATION_DOSES_VERSION, doses.filter(d => d.id !== doseId));
}

// ----- Onboarding -----

export async function getOnboardingSeen(): Promise<boolean> {
 return (await AsyncStorage.getItem(KEYS.ONBOARDING_SEEN)) === 'true';
}

export async function setOnboardingSeen(): Promise<void> {
 await AsyncStorage.setItem(KEYS.ONBOARDING_SEEN, 'true');
}

// ----- Tutor -----

export async function getTutorInfo(): Promise<TutorInfo> {
 const raw = await AsyncStorage.getItem(KEYS.TUTOR);
 return raw ? (JSON.parse(raw) as TutorInfo) : { name: '', phone: '' };
}

export async function saveTutorInfo(info: TutorInfo): Promise<void> {
 await AsyncStorage.setItem(KEYS.TUTOR, JSON.stringify(info));
}

// ----- Backup -----

const BACKUP_VERSION = 1;

export interface Backup {
 backupVersion: number;
 exportedAt: string;
 pets: Pet[];
 records: MedicalRecord[];
 weights: WeightEntry[];
 documents: PetDocument[];
 medicationDoses?: MedicationDose[];
 tutor: TutorInfo;
}

export async function getLastBackupDate(): Promise<string | null> {
 return AsyncStorage.getItem(KEYS.LAST_BACKUP);
}

export async function setLastBackupDate(isoDate: string): Promise<void> {
 await AsyncStorage.setItem(KEYS.LAST_BACKUP, isoDate);
}

export async function exportBackup(): Promise<string> {
 const exportedAt = new Date().toISOString();
 const backup: Backup = {
 backupVersion: BACKUP_VERSION,
 exportedAt,
 pets: await readPets(),
 records: await readRecords(),
 weights: await readWeights(),
 };

```

```

 documents: await readDocuments(),
 medicationDoses: await readMedicationDoses(),
 tutor: await getTutorInfo(),
 });
 await setLastBackupDate(exportedAt);
 return JSON.stringify(backup, null, 2);
}

/** Lê o resumo de um backup (contagens + data de exportação) sem aplicar nada. */
export function peekBackupSummary(json: string): {
 pets: number;
 records: number;
 exportedAt?: string;
} {
 const parsed = JSON.parse(json) as Backup;
 return {
 pets: Array.isArray(parsed.pets) ? parsed.pets.length : 0,
 records: Array.isArray(parsed.records) ? parsed.records.length : 0,
 exportedAt: parsed.exportedAt,
 };
}

/**
 * Substitui todos os dados pelos do backup. Fotos e arquivos de documentos
 * não fazem parte do JSON – apenas as referências (URI) são restauradas.
 * Em um restore no mesmo aparelho os arquivos persistidos continuam
 * existindo e as fotos/documentos seguem funcionando; em outro aparelho
 * essas referências ficam inválidas e a foto/documento some.
 */
/** Arquivo local persistido (foto/documento) cuja referência pode não existir mais neste aparelho. */
async function localFileExists(uri?: string): Promise<boolean> {
 if (!uri) return false;
 if (Platform.OS === 'web' || !uri.startsWith('file:///')) return true;
 try {
 const info = await FileSystem.getInfoAsync(uri);
 return info.exists;
 } catch {
 return false;
 }
}

export async function importBackup(json: string): Promise<{ pets: number; records: number }> {
 const parsed = JSON.parse(json) as Backup;
 if (
 typeof parsed !== 'object' ||
 parsed === null ||
 typeof parsed.backupVersion !== 'number' ||
 !Array.isArray(parsed.pets) ||
 !Array.isArray(parsed.records) ||
 !Array.isArray(parsed.weights)
) {
 throw new Error('Arquivo de backup inválido.');
```

```

}

// ----- Demo -----

/** Substitui todos os dados pelo conjunto fictício de demonstração. */
export async function loadDemoData(): Promise<void> {
 const demo = buildDemoData();
 await write(KEYS.PETS, PETS_VERSION, demo.pets);
 await write(KEYS.RECORDS, RECORDS_VERSION, demo.records);
 await write(KEYS.WEIGHTS, WEIGHTS_VERSION, demo.weights);
 await write(KEYS.DOCUMENTS, DOCUMENTS_VERSION, demo.documents);
 await write(KEYS.MEDICATION_DOSES, MEDICATION_DOSES_VERSION, []);
 await saveTutorInfo(demo.tutor);
}

```

## src/storage/files.ts

```

import * as FileSystem from 'expo-file-system/legacy';

// Arquivos escolhidos por pickers ficam no cache do app, que o sistema
// pode limpar a qualquer momento. Copiamos para o documentDirectory,
// que persiste enquanto o app estiver instalado.
const PHOTOS_DIR = `${FileSystem.documentDirectory}pet-photos/`;
const DOCS_DIR = `${FileSystem.documentDirectory}pet-docs/`;

function extensionOf(uri: string): string {
 const match = uri.match(/\.(\w+)(?:\?.*)?$/);
 return match ? match[1] : 'jpg';
}

async function persistFile(tempUri: string, dir: string, baseName: string): Promise<string> {
 // Na web não há documentDirectory; os pickers já retornam um data URI estável.
 if (!FileSystem.documentDirectory) return tempUri;
 if (tempUri.startsWith(dir)) return tempUri;
 await FileSystem.makeDirectoryAsync(dir, { intermediates: true });
 const dest = `${dir}${baseName}-${Date.now()}.${extensionOf(tempUri)}`;
 await FileSystem.copyAsync({ from: tempUri, to: dest });
 return dest;
}

async function deleteFile(uri: string | undefined, dir: string): Promise<void> {
 if (!uri || !uri.startsWith(dir)) return;
 try {
 await FileSystem.deleteAsync(uri, { idempotent: true });
 } catch {
 // arquivo órfão não impede a exclusão do item
 }
}

export function persistPhoto(tempUri: string, petId: string): Promise<string> {
 return persistFile(tempUri, PHOTOS_DIR, petId);
}

export function deletePhoto(uri?: string): Promise<void> {
 return deleteFile(uri, PHOTOS_DIR);
}

export function persistDocumentFile(tempUri: string, documentId: string): Promise<string> {
 return persistFile(tempUri, DOCS_DIR, documentId);
}

export function deleteDocumentFile(uri?: string): Promise<void> {
 return deleteFile(uri, DOCS_DIR);
}

```

## src/repositories/petsRepository.ts

```

import { getPets, savePet, deletePet } from '../storage';

export const petsRepository = {
 getAll: getPets,
 save: savePet,
 remove: deletePet,
};

```

### src/repositories/recordsRepository.ts

```
import { getRecords, getAllRecords, saveRecord, deleteRecord } from '../storage';

export const recordsRepository = {
 getByPet: getRecords,
 getAll: getAllRecords,
 save: saveRecord,
 remove: deleteRecord,
};
```

### src/repositories/weightsRepository.ts

```
import { getWeights, saveWeight, deleteWeight } from '../storage';

export const weightsRepository = {
 getByPet: getWeights,
 save: saveWeight,
 remove: deleteWeight,
};
```

### src/repositories/documentsRepository.ts

```
import { getDocuments, saveDocument, deleteDocument } from '../storage';

export const documentsRepository = {
 getByPet: getDocuments,
 save: saveDocument,
 remove: deleteDocument,
};
```

### src/repositories/medicationsRepository.ts

```
import { getMedicationDoses, saveMedicationDose, deleteMedicationDose } from '../storage';

export const medicationsRepository = {
 getDoses: getMedicationDoses,
 saveDose: saveMedicationDose,
 removeDose: deleteMedicationDose,
};
```

### src/repositories/backupRepository.ts

```
import {
 exportBackup,
 importBackup,
 peekBackupSummary,
 getLastBackupDate,
 loadDemoData,
} from '../storage';

export const backupRepository = {
 export: exportBackup,
 import: importBackup,
 peekSummary: peekBackupSummary,
 getLastBackupDate,
 loadDemoData,
};
```

## src/repositories/onboardingRepository.ts

```
import { getOnboardingSeen, setOnboardingSeen } from '../storage';

export const onboardingRepository = {
 getSeen: getOnboardingSeen,
 setSeen: setOnboardingSeen,
};
```

## src/services/events.ts

```
import { MedicalRecord, Pet, RecordType } from '../types';
import { daysUntilISO } from '../utils/date';

export type VaccineStatus = 'ok' | 'due_soon' | 'overdue' | 'completed';

/**
 * Janela padrão de alerta quando o usuário não configurou lembrete. */
const DEFAULT_REMINDER_DAYS = 3;
/**
 * Vacina fica "amarela" quando o reforço está a este número de dias ou menos. */
const VACCINE_DUE_SOON_DAYS = 15;

/**
 * Compara títulos ignorando maiúsculas e marcações de dose ("V10 2ª dose"
 * casa com "V10"), para reconhecer doses da mesma vacina/vermífugo.
 */
function normalizeTitle(title: string): string {
 return title
 .toLowerCase()
 .replace(/\\b\\d+\\s*[a-z]*\\s*dose\\b/g, '')
 .replace(/\\bdose\\s*\\d*\\b/g, '')
 .replace(/\\brefor[cc]o\\b/g, '')
 .replace(/\\s+/g, ' ')
 .trim();
}

/**
 * O próximo evento deste registro já foi cumprido?
 * - Vacina/vermífugo: uma dose posterior com o mesmo nome quita o reforço.
 * - Consulta: qualquer consulta posterior à data do registro quita o retorno.
 */
export function isEventFulfilled(
 record: MedicalRecord,
 all: MedicalRecord[],
 fulfilledIds?: Set<string>,
): boolean {
 if (!record.nextDate) return false;
 if (fulfilledIds) return fulfilledIds.has(record.id);
 if (record.type === 'vaccine' || record.type === 'deworming') {
 return all.some(
 other =>
 other.id !== record.id &&
 other.petId === record.petId &&
 other.type === record.type &&
 normalizeTitle(other.title) === normalizeTitle(record.title) &&
 other.date > record.date,
);
 }
 if (record.type === 'consultation') {
 return all.some(
 other =>
 other.id !== record.id &&
 other.petId === record.petId &&
 other.type === 'consultation' &&
 other.date > record.date,
);
 }
 return false;
}

/**
 * Pré-computa quais registros já têm seu próximo evento (reforço/retorno/dose)
 * cumprido por um registro posterior, evitando uma busca O(n) por registro
 * em listas longas (anos de histórico).
 */
export function getFulfilledRecordIds(records: MedicalRecord[]): Set<string> {
 const fulfilled = new Set<string>();
 const groups = new Map<string, MedicalRecord[]>();
 for (const record of records) {
 let key: string | null = null;
 if (record.type === 'vaccine' || record.type === 'deworming') {
 key = `${record.petId}|${record.type}|${normalizeTitle(record.title)}`;
 } else if (record.type === 'consultation') {
 key = `${record.petId}|consultation`;
 }
 if (!key) continue;
 const group = groups.get(key);
 if (group) group.push(record);
 else groups.set(key, [record]);
 }
}
```

```

 }
 for (const group of groups.values()) {
 for (const record of group) {
 if (!record.nextDate) continue;
 if (group.some(other => other.id !== record.id && other.date > record.date)) {
 fulfilled.add(record.id);
 }
 }
 }
 }
 return fulfilled;
}

export function getVaccineStatus(
 record: MedicalRecord,
 all: MedicalRecord[],
 fulfilledIds?: Set<string>,
): VaccineStatus | null {
 if (record.type !== 'vaccine') return null;
 if (!record.nextDate) return 'ok';
 if (isEventFulfilled(record, all, fulfilledIds)) return 'completed';
 const days = daysUntilISO(record.nextDate);
 if (days < 0) return 'overdue';
 if (days <= VACCINE_DUE_SOON_DAYS) return 'due_soon';
 return 'ok';
}

/** Data do próximo evento gerado por um registro (reforço, retorno, dose, fim). */
export function eventDateOf(record: MedicalRecord): string | undefined {
 if (record.type === 'medication') return record.endDate;
 if (record.type === 'note') return undefined;
 return record.nextDate;
}

const EVENT_LABELS: Partial<Record<RecordType, string>> = {
 vaccine: 'Reforço',
 consultation: 'Retorno',
 medication: 'Fim do tratamento',
 deworming: 'Próxima dose',
};

export interface UpcomingEvent {
 key: string;
 petId: string;
 petName: string;
 petPhoto?: string;
 recordId: string;
 type: RecordType;
 title: string;
 date: string;
 daysUntil: number;
 overdue: boolean;
 /** Dentro da janela de lembrete configurada (ou atrasado). */
 pending: boolean;
}

export function getUpcomingEvents(pets: Pet[], records: MedicalRecord[]): UpcomingEvent[] {
 const petById = new Map(pets.map(p => [p.id, p]));
 const fulfilledIds = getFulfilledRecordIds(records);
 const events: UpcomingEvent[] = [];
 for (const record of records) {
 const date = eventDateOf(record);
 if (!date) continue;
 const pet = petById.get(record.petId);
 if (!pet) continue;
 // Reforço/dose/retorno já cumprido em um registro mais novo: não gera alerta.
 if (isEventFulfilled(record, records, fulfilledIds)) continue;
 // Tratamento que chegou ao fim não está "atrasado" – está concluído.
 if (record.type === 'medication' && daysUntilISO(date) < 0) continue;
 const daysUntil = daysUntilISO(date);
 const window = record.reminderDays?.length
 ? Math.max(...record.reminderDays)
 : DEFAULT_REMINDER_DAYS;
 events.push({
 key: `${record.id}-event`,
 petId: pet.id,
 petName: pet.name,
 petPhoto: pet.photo,
 recordId: record.id,
 type: record.type,
 title: `${EVENT_LABELS[record.type]} · ${record.title}`,
 date,

```

```
 daysUntil,
 overdue: daysUntil < 0,
 pending: daysUntil < 0 || daysUntil <= window,
 });
}
return events.sort((a, b) => a.date.localeCompare(b.date));
}

/** Sem data final e sem uso contínuo, o remédio conta como ativo por este prazo. */
const OPEN_MEDICATION_ACTIVE_DAYS = 30;

/** Medicamento em uso: contínuo, com fim no futuro ou iniciado recentemente. */
export function isActiveMedication(record: MedicalRecord): boolean {
 if (record.type !== 'medication') return false;
 if (record.frequency === 'continuous') return true;
 if (!record.endDate) {
 return daysUntilISO(record.date) >= -OPEN_MEDICATION_ACTIVE_DAYS;
 }
 return daysUntilISO(record.endDate) >= 0;
}
```

## src/services/notifications.ts

```
import { Platform } from 'react-native';
import * as Notifications from 'expo-notifications';
import { Pet, MedicalRecord } from '../types';
import { getUpcomingEvents } from './events';

// Lembretes são exibidos mesmo com o app aberto.
Notifications.setNotificationHandler({
 handleNotification: async () => ({
 shouldShowBanner: true,
 shouldShowList: true,
 shouldPlaySound: false,
 shouldSetBadge: false,
 }),
});

const REMINDER_HOUR = 9; // notificações sempre às 9h da manhã

let permissionAsked = false;
let lastSyncSignature: string | null = null;

export async function ensureNotificationPermission(): Promise<boolean> {
 const settings = await Notifications.getPermissionsAsync();
 if (settings.granted) return true;
 if (!settings.canAskAgain) return false;
 if (permissionAsked) return false;
 permissionAsked = true;
 const request = await Notifications.requestPermissionsAsync();
 return request.granted;
}

function triggerAt(dateISO: string, daysBefore: number): Date | null {
 const [y, m, d] = dateISO.split('-').map(Number);
 if (!y || !m || !d) return null;
 const when = new Date(y, m - 1, d - daysBefore, REMINDER_HOUR, 0, 0);
 return when.getTime() > Date.now() ? when : null;
}

/**
 * Reagenda todas as notificações locais a partir dos eventos futuros
 * (reforço de vacina, retorno, fim de tratamento, próxima dose).
 * Usa os `reminderDays` configurados no registro, ou o dia do evento.
 */
export async function syncEventNotifications(
 pets: Pet[],
 records: MedicalRecord[],
): Promise<void> {
 if (Platform.OS === 'web') return;
 const granted = await ensureNotificationPermission();
 if (!granted) return;

 const events = getUpcomingEvents(pets, records);
 const recordById = new Map(records.map(r => [r.id, r]));

 // Evita cancelar e reagendar tudo de novo se nada relevante mudou desde a última sincronização.
 const signature = JSON.stringify(
 events.map(e => [e.recordId, e.date, recordById.get(e.recordId)?.reminderDays ?? null]),
);
 if (signature === lastSyncSignature) return;
 lastSyncSignature = signature;

 if (Platform.OS === 'android') {
 await Notifications.setNotificationChannelAsync('reminders', {
 name: 'Lembretes',
 importance: Notifications.AndroidImportance.DEFAULT,
 });
 }

 await Notifications.cancelAllScheduledNotificationsAsync();

 for (const event of events) {
 const record = recordById.get(event.recordId);
 const reminderDays = record?.reminderDays?.length ? record.reminderDays : [0];
 for (const daysBefore of reminderDays) {
 const when = triggerAt(event.date, daysBefore);
 if (!when) continue;
 await Notifications.scheduleNotificationAsync({
 content: {

```

```
title: `🐾 ${event.petName}: ${event.title}`,
body:
 daysBefore === 0
 ? 'É hoje! Abra o PetCare para ver os detalhes.'
 : `Faltam ${daysBefore} ${daysBefore === 1 ? 'dia' : 'dias'}. Abra o PetCare para ver os detalhes.`,
data: { petId: event.petId, recordId: event.recordId },
},
trigger: {
 type: Notifications.SchedulableTriggerInputTypes.DATE,
 date: when,
 channelId: 'reminders',
},
});
}
}
```

## src/services/pdf.ts

```
import * as Print from 'expo-print';
import * as Sharing from 'expo-sharing';
import * as FileSystem from 'expo-file-system/legacy';
import { Asset } from 'expo-asset';
import { Pet, MedicalRecord, WeightEntry } from '../types';
import { SPECIES_LABELS, RECORD_TYPE_LABELS, FREQUENCY_LABELS } from '../labels';
import { displayDate, calcAge } from '../utils/date';

// Paleta da marca (mesma do tema claro – o PDF tem fundo branco)
const ORANGE = '#E66A3A';
const NAVY = '#1B2940';
const MUTED = '#64748B';

async function logoDataUri(): Promise<string | null> {
 try {
 const asset = Asset.fromModule(require('../../assets/logo-light.png'));
 await asset.downloadAsync();
 if (!asset.localUri) return null;
 const base64 = await FileSystem.readAsStringAsync(asset.localUri, {
 encoding: FileSystem.EncodingType.Base64,
 });
 return `data:image/png;base64,${base64}`;
 } catch {
 return null;
 }
}

function esc(value: string): string {
 return value.replace(/&/g, '&').replace(/</g, '<').replace(/>/g, '>');
}

function recordDetails(r: MedicalRecord): string {
 const parts: string[] = [];
 if (r.type === 'vaccine' && r.nextDate) parts.push(`Reforço: ${displayDate(r.nextDate)}`);
 if (r.type === 'consultation' && r.diagnosis) parts.push(`Diagnóstico: ${r.diagnosis}`);
 if (r.type === 'medication' && r.dosage) parts.push(r.dosage);
 if (r.type === 'medication' && r.frequency) parts.push(FREQUENCY_LABELS[r.frequency]);
 if (r.description) parts.push(r.description);
 return parts.map(esc).join(' · ');
}

export function buildPetHtml(
 pet: Pet,
 records: MedicalRecord[],
 weights: WeightEntry[],
 logo: string | null,
): string {
 const age = calcAge(pet.birthDate);
 const subtitle = [SPECIES_LABELS[pet.species], pet.breed, age].filter(Boolean).join(' · ');
 const profile = pet.medicalProfile;
 const latest = weights[0];

 const profileRows: string[] = [];
 if (profile) {
 profileRows.push(`<tr><td>Castrado</td><td>${profile.neutered ? 'Sim' : 'Não'}</td></tr>`);
 if (profile.allergies)
 profileRows.push(`<tr><td>Alergias</td><td>${esc(profile.allergies)}</td></tr>`);
 if (profile.chronicConditions)
 profileRows.push(`<tr><td>Doenças crônicas</td><td>${esc(profile.chronicConditions)}</td></tr>`);
 if (profile.bloodType)
 profileRows.push(`<tr><td>Tipo sanguíneo</td><td>${esc(profile.bloodType)}</td></tr>`);
 if (profile.vetName)
 profileRows.push(
 `<tr><td>Veterinário</td><td>${esc(profile.vetName)}${
 profile.vetPhone ? ` (${esc(profile.vetPhone)})` : ''
 }</td></tr>`,
);
 if (profile.notes)
 profileRows.push(`<tr><td>Observações</td><td>${esc(profile.notes)}</td></tr>`);
 }
 if (latest) {
 profileRows.push(
 `<tr><td>Peso atual</td><td>${latest.weightKg.toLocaleString('pt-BR')} kg (${displayDate(latest.date)})</td></tr>`,
);
 }

 const historyRows = records
```



## src/services/share.ts

```
import { Share } from 'react-native';
import { Pet, MedicalRecord, WeightEntry } from '../types';
import { SPECIES_LABELS, RECORD_TYPE_LABELS, FREQUENCY_LABELS } from '../labels';
import { displayDate, calcAge } from '../utils/date';

/**
 * Monta um resumo em texto do prontuário, pronto para enviar por
 * WhatsApp/e-mail ao veterinário. A versão em PDF está prevista para
 * uma próxima versão (ver sharePetPdf).
 */
export function buildPetSummary(
 pet: Pet,
 records: MedicalRecord[],
 weights: WeightEntry[],
): string {
 const lines: string[] = [];
 const age = calcAge(pet.birthDate);

 lines.push(` 📄 PRONTUÁRIO — ${pet.name.toUpperCase()} `);
 lines.push(
 [SPECIES_LABELS[pet.species], pet.breed, age].filter(Boolean).join(' · '),
);
 if (pet.birthDate) lines.push(` Nascimento: ${displayDate(pet.birthDate)} `);

 const profile = pet.medicalProfile;
 if (profile) {
 lines.push('');
 lines.push(' PERFIL MÉDICO ');
 lines.push(` Castrado: ${profile.neutered ? 'Sim' : 'Não'} `);
 if (profile.allergies) lines.push(` Alergias: ${profile.allergies} `);
 if (profile.chronicConditions) lines.push(` Doenças crônicas: ${profile.chronicConditions} `);
 if (profile.bloodType) lines.push(` Tipo sanguíneo: ${profile.bloodType} `);
 if (profile.vetName) {
 lines.push(` Veterinário: ${profile.vetName}${profile.vetPhone ? ` (${profile.vetPhone})` : ''} `);
 }
 if (profile.notes) lines.push(` Observações: ${profile.notes} `);
 }

 if (weights.length > 0) {
 const latest = weights[0];
 lines.push('');
 lines.push(` PESO ATUAL: ${latest.weightKg.toLocaleString('pt-BR')} kg (${displayDate(latest.date)}) `);
 }

 if (records.length > 0) {
 lines.push('');
 lines.push(' HISTÓRICO ');
 for (const r of records) {
 const parts = [
 `${displayDate(r.date)} — ${RECORD_TYPE_LABELS[r.type]}: ${r.title}`,
];
 if (r.type === 'vaccine' && r.nextDate) parts.push(` reforço ${displayDate(r.nextDate)} `);
 if (r.type === 'consultation' && r.diagnosis) parts.push(` diagnóstico: ${r.diagnosis} `);
 if (r.type === 'medication' && r.dosage) parts.push(r.dosage);
 if (r.type === 'medication' && r.frequency) parts.push(FREQUENCY_LABELS[r.frequency]);
 if (r.description) parts.push(r.description);
 lines.push(` • ${parts.join(' · ')} `);
 }
 }

 lines.push('');
 lines.push('Gerado pelo app PetCare');
 return lines.join('\n');
}

export async function sharePetSummary(
 pet: Pet,
 records: MedicalRecord[],
 weights: WeightEntry[],
): Promise<void> {
 const message = buildPetSummary(pet, records, weights);
 await Share.share({ message, title: `Prontuário de ${pet.name}` });
}
```

## src/services/demoData.ts

```
import { Pet, MedicalRecord, WeightEntry, PetDocument, TutorInfo } from '../types';

/**
 * Conjunto fixo de dados fictícios para demonstração do app (modo demo).
 * Datas são geradas em relação a "hoje" para que vacinas/remédios sempre
 * apareçam com status realistas (próximos, atrasados, etc.).
 */

function isoDaysFromNow(days: number): string {
 const d = new Date();
 d.setDate(d.getDate() + days);
 return d.toISOString().slice(0, 10);
}

export interface DemoData {
 pets: Pet[];
 records: MedicalRecord[];
 weights: WeightEntry[];
 documents: PetDocument[];
 tutor: TutorInfo;
}

export function buildDemoData(): DemoData {
 const now = new Date().toISOString();
 const pitocoId = 'demo-pitoco';
 const mimiId = 'demo-mimi';

 const pets: Pet[] = [
 {
 id: pitocoId,
 name: 'Pitoco',
 species: 'dog',
 breed: 'Labrador',
 birthDate: isoDaysFromNow(-365 * 3),
 medicalProfile: {
 neutered: true,
 allergies: 'Nenhuma conhecida',
 bloodType: 'DEA 1.1 positivo',
 vetName: 'Dra. Camila Souza',
 vetPhone: '(11) 98888-1234',
 },
 createdAt: now,
 },
 {
 id: mimiId,
 name: 'Mimi',
 species: 'cat',
 breed: 'Siamês',
 birthDate: isoDaysFromNow(-365 * 5),
 medicalProfile: {
 neutered: true,
 chronicConditions: 'Insuficiência renal leve (acompanhamento)',
 vetName: 'Dr. André Lima',
 vetPhone: '(11) 97777-5678',
 },
 createdAt: now,
 },
];

 const records: MedicalRecord[] = [
 {
 id: 'demo-rec-1',
 petId: pitocoId,
 type: 'vaccine',
 date: isoDaysFromNow(-300),
 title: 'V10 (Polivalente)',
 manufacturer: 'Zoetis',
 batch: 'LZ4421',
 clinic: 'Clínica VetCare',
 nextDate: isoDaysFromNow(20),
 createdAt: now,
 },
 {
 id: 'demo-rec-2',
 petId: pitocoId,
 type: 'consultation',
 date: isoDaysFromNow(-60),
 },
];
}
```

```

 title: 'Check-up de rotina',
 vet: 'Dra. Camila Souza',
 diagnosis: 'Saudável, recomendado manter peso',
 clinic: 'Clínica VetCare',
 nextDate: isoDaysFromNow(120),
 createdAt: now,
 },
 {
 id: 'demo-rec-3',
 petId: pitocoId,
 type: 'medication',
 date: isoDaysFromNow(-10),
 title: 'Antialérgico',
 dosage: '1 comprimido de 10mg',
 frequency: 'once_daily',
 endDate: isoDaysFromNow(5),
 createdAt: now,
 },
 {
 id: 'demo-rec-4',
 petId: pitocoId,
 type: 'deworming',
 date: isoDaysFromNow(-90),
 title: 'Vermífugo',
 nextDate: isoDaysFromNow(-2),
 createdAt: now,
 },
 {
 id: 'demo-rec-5',
 petId: mimiId,
 type: 'vaccine',
 date: isoDaysFromNow(-200),
 title: 'Tríplice felina',
 manufacturer: 'MSD',
 clinic: 'Clínica VetCare',
 nextDate: isoDaysFromNow(-5),
 createdAt: now,
 },
 {
 id: 'demo-rec-6',
 petId: mimiId,
 type: 'medication',
 date: isoDaysFromNow(-30),
 title: 'Ração renal especial (suplemento)',
 dosage: '1/2 sachê',
 frequency: 'continuous',
 createdAt: now,
 },
 {
 id: 'demo-rec-7',
 petId: mimiId,
 type: 'note',
 date: isoDaysFromNow(-15),
 title: 'Observação',
 description: 'Apresentou menos apetite por 2 dias, voltou ao normal depois.',
 createdAt: now,
 },
];

const weights: WeightEntry[] = [
 { id: 'demo-w-1', petId: pitocoId, date: isoDaysFromNow(-150), weightKg: 28.5, createdAt: now },
 { id: 'demo-w-2', petId: pitocoId, date: isoDaysFromNow(-75), weightKg: 29.2, createdAt: now },
 { id: 'demo-w-3', petId: pitocoId, date: isoDaysFromNow(-5), weightKg: 29.8, createdAt: now },
 { id: 'demo-w-4', petId: mimiId, date: isoDaysFromNow(-120), weightKg: 4.1, createdAt: now },
 { id: 'demo-w-5', petId: mimiId, date: isoDaysFromNow(-30), weightKg: 4.0, createdAt: now },
 { id: 'demo-w-6', petId: mimiId, date: isoDaysFromNow(-3), weightKg: 3.9, createdAt: now },
];

const documents: PetDocument[] = [
 {
 id: 'demo-doc-1',
 petId: pitocoId,
 title: 'Exame de sangue - hemograma',
 kind: 'exam',
 date: isoDaysFromNow(-60),
 uri: '',
 createdAt: now,
 },
];

const tutor: TutorInfo = {

```

```
 name: 'Tutor Demonstração',
 phone: '(11) 90000-0000',
 };

 return { pets, records, weights, documents, tutor };
}
```

## src/hooks/useToast.tsx

```
import React, { createContext, useCallback, useContext, useRef, useState } from 'react';
import { Animated, Platform, StyleSheet, Text } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { radius, spacing, typography, useTheme, Palette } from '../theme';

export type ToastVariant = 'success' | 'error';

interface ToastState {
 message: string;
 variant: ToastVariant;
}

interface ToastContextValue {
 showToast: (message: string, variant?: ToastVariant) => void;
}

const ToastContext = createContext<ToastContextValue>({ showToast: () => {} });

export function useToast(): ToastContextValue {
 return useContext(ToastContext);
}

export function ToastProvider({ children }: { children: React.ReactNode }) {
 const { colors } = useTheme();
 const [toast, setToast] = useState<ToastState | null>(null);
 const opacity = useRef(new Animated.Value(0)).current;
 const translateY = useRef(new Animated.Value(20)).current;
 const hideTimer = useRef<ReturnType<typeof setTimeout> | null>(null);

 const showToast = useCallback(
 (message: string, variant: ToastVariant = 'success') => {
 if (hideTimer.current) clearTimeout(hideTimer.current);
 setToast({ message, variant });
 opacity.setValue(0);
 translateY.setValue(20);
 Animated.parallel([
 Animated.timing(opacity, { toValue: 1, duration: 200, useNativeDriver: true }),
 Animated.timing(translateY, { toValue: 0, duration: 200, useNativeDriver: true }),
]).start();

 hideTimer.current = setTimeout(() => {
 Animated.timing(opacity, { toValue: 0, duration: 200, useNativeDriver: true }).start(
 () => setToast(null),
);
 }, 2400);
 },
 [opacity, translateY],
);

 const styles = createStyles(colors);
 const isSuccess = toast?.variant !== 'error';

 return (
 <ToastContext.Provider value={{ showToast }}>
 {children}
 {toast ? (
 <Animated.View
 style={[
 styles.toast,
 { opacity, transform: [{ translateY }] },
 { borderColor: isSuccess ? colors.success : colors.danger },
]}
 accessibilityLiveRegion="polite"
 >
 <Ionicons
 name={isSuccess ? 'checkmark-circle' : 'alert-circle'}
 size={18}
 color={isSuccess ? colors.success : colors.danger}
 />
 <Text style={styles.text}>{toast.message}</Text>
 </Animated.View>
) : null}
 </ToastContext.Provider>
);
}

const createStyles = (colors: Palette) =>
```

```
StyleSheet.create({
 toast: {
 position: 'absolute',
 left: spacing.lg,
 right: spacing.lg,
 bottom: Platform.OS === 'ios' ? spacing.xxl + spacing.md : spacing.xxl,
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.sm,
 backgroundColor: colors.surfaceElevated,
 borderWidth: 1,
 borderRadius: radius.md,
 paddingVertical: spacing.sm + 2,
 paddingHorizontal: spacing.md,
 shadowColor: '#000',
 shadowOffset: { width: 0, height: 4 },
 shadowOpacity: 0.2,
 shadowRadius: 10,
 elevation: 8,
 },
 text: {
 flex: 1,
 fontSize: typography.body.fontSize,
 color: colors.text,
 },
});
```

## src/components/Button.tsx

```
import React from 'react';
import {
 ActivityIndicator,
 StyleSheet,
 Text,
 TouchableOpacity,
 TouchableOpacityProps,
 View,
} from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { radius, spacing, typography, useThemedStyles, Palette } from '../theme';

export type ButtonVariant = 'primary' | 'secondary' | 'destructive' | 'ghost';

interface ButtonProps extends Omit<TouchableOpacityProps, 'style'> {
 label: string;
 icon?: keyof typeof Ionicons.glyphMap;
 loading?: boolean;
 disabled?: boolean;
 variant?: ButtonVariant;
}

export function Button({
 label,
 icon,
 loading,
 disabled,
 variant = 'primary',
 onPress,
 accessibilityLabel,
 ...rest
}: ButtonProps) {
 const styles = useThemedStyles(createStyles);
 const isDisabled = disabled || loading;

 return (
 <TouchableOpacity
 style={[styles.base, styles[variant], isDisabled && styles.disabled]}
 onPress={onPress}
 disabled={isDisabled}
 activeOpacity={0.85}
 accessibilityRole="button"
 accessibilityLabel={accessibilityLabel ?? label}
 accessibilityState={{ disabled: isDisabled }}
 hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
 {...rest}
 >
 {loading ? (
 <ActivityIndicator color={textColor(variant, styles)} size="small" />
) : (
 <View style={styles.content}>
 {icon ? (
 <Ionicons name={icon} size={18} color={textColor(variant, styles)} />
) : null}
 <Text style={[styles.label, styles[`${variant}Text` as const]]>{label}</Text>
 </View>
)}
 </TouchableOpacity>
);
}

function textColor(variant: ButtonVariant, styles: ReturnType<typeof createStyles>): string {
 return (styles[`${variant}Text` as const] as { color: string }).color;
}

const createStyles = (colors: Palette) =>
 StyleSheet.create({
 base: {
 width: '100%',
 borderRadius: radius.md,
 paddingVertical: spacing.md - 2,
 alignItems: 'center',
 justifyContent: 'center',
 minHeight: 48,
 },
 content: {
 flexDirection: 'row',
 }
 });
```

```
 alignItems: 'center',
 justifyContent: 'center',
 gap: spacing.sm,
 },
 label: {
 fontSize: typography.h4.fontSize,
 fontWeight: typography.h4.fontWeight,
 },
 disabled: { opacity: 0.6 },

 primary: {
 backgroundColor: colors.primary,
 shadowColor: colors.primary,
 shadowOffset: { width: 0, height: 4 },
 shadowOpacity: 0.35,
 shadowRadius: 12,
 elevation: 6,
 },
 primaryText: { color: colors.onPrimary },

 secondary: {
 backgroundColor: 'transparent',
 borderWidth: 1,
 borderColor: colors.border,
 },
 secondaryText: { color: colors.text },

 destructive: {
 backgroundColor: 'transparent',
 borderWidth: 1,
 borderColor: colors.danger,
 },
 destructiveText: { color: colors.danger },

 ghost: {
 backgroundColor: 'transparent',
 },
 ghostText: { color: colors.primaryLight },
});
```

## src/components/EmptyState.tsx

```
import React from 'react';
import { View, Text, StyleSheet, Image, ImageSourcePropType } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';

interface Props {
 icon?: keyof typeof Ionicons.glyphMap;
 image?: ImageSourcePropType;
 title: string;
 text?: string;
}

export function EmptyState({ icon, image, title, text }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 return (
 <View style={styles.container}>
 <View style={styles.iconWrapper}>
 {image ? (
 <Image source={image} style={styles.image} resizeMode="contain" />
) : icon ? (
 <Ionicons name={icon} size={36} color={colors.primaryLight} />
) : null}
 </View>
 <Text style={styles.title}>{title}</Text>
 {text ? <Text style={styles.text}>{text}</Text> : null}
 </View>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: {
 alignItems: 'center',
 paddingHorizontal: spacing.xl,
 paddingVertical: spacing.lg,
 },
 iconWrapper: {
 width: 80,
 height: 80,
 borderRadius: radius.full,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 alignItems: 'center',
 justifyContent: 'center',
 marginBottom: spacing.md,
 },
 image: {
 width: 46,
 height: 46,
 },
 title: {
 fontSize: 17,
 fontWeight: '600',
 color: colors.text,
 marginBottom: spacing.xs,
 textAlign: 'center',
 },
 text: {
 fontSize: 13,
 color: colors.textMuted,
 textAlign: 'center',
 lineHeight: 20,
 },
});
```

## src/components/HealthRecordCard.tsx

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { RECORD_TYPE_LABELS, recordTypeColors, RECORD_TYPE_ICONS } from '../labels';
import { displayDate } from '../utils/date';
import { MedicalRecord } from '../types';

interface Props {
 record: MedicalRecord;
 petName?: string;
 onPress: () => void;
}

export function HealthRecordCard({ record, petName, onPress }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const color = recordTypeColors(colors)[record.type];
 return (
 <TouchableOpacity style={styles.card} onPress={onPress} activeOpacity={0.8}>
 <View style={styles.top}>
 <View style={[styles.badge, { backgroundColor: color + '22' }]}>
 <Ionicons name={RECORD_TYPE_ICONS[record.type]} size={11} color={color} />
 <Text style={styles.badgeText, { color }}>{RECORD_TYPE_LABELS[record.type]}</Text>
 </View>
 <Text style={styles.date}>{displayDate(record.date)}</Text>
 </View>
 <Text style={styles.title}>{record.title}</Text>
 {petName ? <Text style={styles.petName}>🐾 {petName}</Text> : null}
 {record.description ? (
 <Text style={styles.description} numberOfLines={2}>
 {record.description}
 </Text>
) : null}
 </TouchableOpacity>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 card: {
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 gap: 4,
 },
 top: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 },
 badge: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: 4,
 paddingHorizontal: spacing.sm,
 paddingVertical: 3,
 borderRadius: radius.full,
 },
 badgeText: { fontSize: 11, fontWeight: '600' },
 date: { fontSize: 12, color: colors.textMuted },
 title: { fontSize: 15, fontWeight: '600', color: colors.text },
 petName: { fontSize: 12, color: colors.textSubtle },
 description: { fontSize: 13, color: colors.textSubtle, lineHeight: 19 },
});
```

## src/components/Input.tsx

```
import React from 'react';
import { View, Text, TextInput, TextInputProps, StyleSheet } from 'react-native';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';

interface Props extends TextInputProps {
 label: string;
}

export function Input({ label, style, ...props }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 return (
 <View style={styles.wrapper}>
 <Text style={styles.label}>{label}</Text>
 <TextInput
 style={style}
 multiline={props.multiline}
 placeholderTextColor={colors.textMuted}
 selectionColor={colors.primaryLight}
 {...props}
 />
 </View>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 wrapper: { gap: spacing.xs },
 label: {
 fontSize: 13,
 fontWeight: '500',
 color: colors.textSubtle,
 letterSpacing: 0.3,
 },
 input: {
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 borderRadius: radius.md,
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.sm + 4,
 color: colors.text,
 fontSize: 15,
 },
 multiline: {
 minHeight: 100,
 paddingTop: spacing.sm + 4,
 textAlignVertical: 'top',
 },
});
```

## src/components/MedicalProfileCard.tsx

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { MedicalProfile } from '../types';

interface Props {
 profile?: MedicalProfile;
 onPress: () => void;
}

export function MedicalProfileCard({ profile, onPress }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const filled =
 profile &&
 (profile.allergies ||
 profile.chronicConditions ||
 profile.bloodType ||
 profile.vetName ||
 profile.notes ||
 profile.neutered !== undefined);

 return (
 <TouchableOpacity style={styles.card} onPress={onPress} activeOpacity={0.8}>
 <View style={styles.iconWrapper}>
 <Ionicons name="fitness" size={18} color={colors.danger} />
 </View>
 <View style={{ flex: 1 }}>
 <Text style={styles.title}>Perfil Médico</Text>
 {filled ? (
 <Text style={styles.subtitle} numberOfLines={2}>
 {[
 profile?.neutered ? 'Castrado' : undefined,
 profile?.allergies ? `Alergias: ${profile.allergies}` : undefined,
 profile?.chronicConditions,
 profile?.vetName,
]
 .filter(Boolean)
 .join(' · ')}
 </Text>
) : (
 <Text style={styles.subtitle}>Toque para preencher alergias, doenças e veterinário</Text>
)}
 </View>
 <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
 </TouchableOpacity>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 card: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 },
 iconWrapper: {
 width: 40,
 height: 40,
 borderRadius: radius.full,
 backgroundColor: colors.danger + '22',
 alignItems: 'center',
 justifyContent: 'center',
 },
 title: { fontSize: 14, fontWeight: '600', color: colors.text },
 subtitle: { fontSize: 12, color: colors.textMuted, marginTop: 2 },
});
```

## src/components/PetCard.tsx

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet, Image } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { SPECIES_LABELS } from '../labels';
import { calcAge } from '../utils/date';
import { Pet } from '../types';

interface Props {
 pet: Pet;
 subtitle?: string;
 activeMeds?: number;
 pendingAlerts?: number;
 onPress: () => void;
}

export const PetCard = React.memo(function PetCard({ pet, subtitle, activeMeds, pendingAlerts, onPress }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 return (
 <TouchableOpacity style={styles.card} onPress={onPress} activeOpacity={0.7}>
 <View style={styles.photoWrapper}>
 {pet.photo ? (
 <Image source={{ uri: pet.photo }} style={styles.photo} />
) : (
 <View style={styles.photoPlaceholder}>
 <Image
 source={require('../assets/icons/paw.png')}
 style={styles.placeholderPaw}
 resizeMode="contain"
 />
 </View>
)}
 </View>

 <View style={styles.cardInfo}>
 <Text style={styles.petName}>{pet.name}</Text>
 <Text style={styles.petSubtitle}>
 {subtitle ??
 `${SPECIES_LABELS[pet.species]}${pet.breed ? ` · ${pet.breed}` : ''}${
 pet.birthDate ? ` · ${calcAge(pet.birthDate)}` : ''
 }`}
 </Text>
 </View>

 {!!pendingAlerts && (
 <View style={[styles.badge, { backgroundColor: colors.warning + '22' }]}>
 <Ionicons name="notifications" size={12} color={colors.warning} />
 <Text style={styles.badgeText, { color: colors.warning }}>{pendingAlerts}</Text>
 </View>
)}

 {!!activeMeds && (
 <View style={[styles.badge, { backgroundColor: colors.accent + '22' }]}>
 <Ionicons name="medkit" size={12} color={colors.accent} />
 <Text style={styles.badgeText, { color: colors.accent }}>{activeMeds}</Text>
 </View>
)}

 <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
 </TouchableOpacity>
);
});

const createStyles = (colors: Palette) => StyleSheet.create({
 card: {
 flexDirection: 'row',
 alignItems: 'center',
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 padding: spacing.md,
 gap: spacing.md,
 borderWidth: 1,
 borderColor: colors.border,
 },
 photoWrapper: {
 width: 54,
 height: 54,
 },
});
```

```
 borderRadius: radius.full,
 overflow: 'hidden',
 },
 photo: {
 width: 54,
 height: 54,
 },
 photoPlaceholder: {
 width: 54,
 height: 54,
 borderRadius: radius.full,
 backgroundColor: colors.surfaceElevated,
 alignItems: 'center',
 justifyContent: 'center',
 },
 placeholderPaw: {
 width: 28,
 height: 28,
 },
 cardInfo: {
 flex: 1,
 gap: 3,
 },
 petName: {
 fontSize: 16,
 fontWeight: '600',
 color: colors.text,
 },
 petSubtitle: {
 fontSize: 13,
 color: colors.textMuted,
 },
 badge: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: 4,
 paddingHorizontal: 8,
 paddingVertical: 4,
 borderRadius: radius.full,
 },
 badgeText: {
 fontSize: 12,
 fontWeight: '700',
 },
 },
));
```

## src/components/ReminderCard.tsx

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { RECORD_TYPE_ICONS } from '../labels';
import { displayDate, formatDaysUntil } from '../utils/date';
import { UpcomingEvent } from '../services/events';

interface Props {
 event: UpcomingEvent;
 onPress: () => void;
}

export const ReminderCard = React.memo(function ReminderCard({ event, onPress }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const color = event.overdue
 ? colors.danger
 : event.daysUntil <= 3
 ? colors.warning
 : colors.primaryLight;

 return (
 <TouchableOpacity style={styles.card} onPress={onPress} activeOpacity={0.8}>
 <View style={[styles.iconWrapper, { backgroundColor: color + '18' }]}>
 <Ionicons name={RECORD_TYPE_ICONS[event.type]} size={16} color={color} />
 </View>
 <View style={{ flex: 1 }}>
 <Text style={styles.petName}>{event.petName}</Text>
 <Text style={styles.title} numberOfLines={1}>
 {event.title}
 </Text>
 </View>
 <View style={styles.right}>
 <Text style={[styles.when, { color }]}>
 {event.overdue ? `atrasado ${formatDaysUntil(event.daysUntil)}` : formatDaysUntil(event.daysUntil)}
 </Text>
 <Text style={styles.date}>{displayDate(event.date)}</Text>
 </View>
 </TouchableOpacity>
);
});

const createStyles = (colors: Palette) => StyleSheet.create({
 card: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 },
 iconWrapper: {
 width: 36,
 height: 36,
 borderRadius: radius.full,
 alignItems: 'center',
 justifyContent: 'center',
 },
 petName: { fontSize: 12, color: colors.textMuted },
 title: { fontSize: 14, fontWeight: '600', color: colors.text, marginTop: 1 },
 right: { alignItems: 'flex-end', gap: 2 },
 when: { fontSize: 12, fontWeight: '600' },
 date: { fontSize: 11, color: colors.textMuted },
});
```

## src/components/StatCard.tsx

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';

interface Props {
 icon: keyof typeof Ionicons.glyphMap;
 label: string;
 value: string | number;
 color?: string;
 onPress?: () => void;
}

export const StatCard = React.memo(function StatCard({ icon, label, value, color, onPress }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const tint = color ?? colors.primaryLight;
 const Wrapper = onPress ? TouchableOpacity : View;
 return (
 <Wrapper style={styles.card} onPress={onPress} activeOpacity={onPress ? 0.8 : undefined}>
 <View style={[styles.iconWrapper, { backgroundColor: tint + '18' }]}>
 <Ionicons name={icon} size={16} color={tint} />
 </View>
 <Text style={styles.value}>{value}</Text>
 <Text style={styles.label} numberOfLines={1}>
 {label}
 </Text>
 </Wrapper>
);
});

const createStyles = (colors: Palette) => StyleSheet.create({
 card: {
 flex: 1,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 gap: spacing.xs,
 },
 iconWrapper: {
 width: 30,
 height: 30,
 borderRadius: radius.full,
 alignItems: 'center',
 justifyContent: 'center',
 },
 value: { fontSize: 22, fontWeight: '700', color: colors.text },
 label: { fontSize: 11, color: colors.textMuted },
});
```

## src/components/ThemeToggle.tsx

```
import React from 'react';
import { TouchableOpacity } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useTheme } from '../theme';

interface Props {
 size?: number;
}

export function ThemeToggle({ size = 22 }: Props) {
 const { colors, scheme, toggleTheme } = useTheme();
 return (
 <TouchableOpacity
 onPress={toggleTheme}
 hitSlop={{ top: 12, bottom: 12, left: 8, right: 8 }}
 accessibilityRole="button"
 accessibilityLabel={scheme === 'dark' ? 'Mudar para tema claro' : 'Mudar para tema escuro'}
 >
 <Ionicons
 name={scheme === 'dark' ? 'sunny-outline' : 'moon-outline'}
 size={size}
 color={colors.textSubtle}
 />
 </TouchableOpacity>
);
}
```

## src/components/TimelineItem.tsx

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useThemedStyles, Palette } from '../theme';

export interface TimelineBadge {
 label: string;
 color: string;
}

interface Props {
 icon: keyof typeof Ionicons.glyphMap;
 color: string;
 dateLabel: string;
 title: string;
 lines?: string[];
 badge?: TimelineBadge;
 isLast?: boolean;
 onPress?: () => void;
 onLongPress?: () => void;
}

export const TimelineItem = React.memo(function TimelineItem({
 icon,
 color,
 dateLabel,
 title,
 lines,
 badge,
 isLast,
 onPress,
 onLongPress,
}: Props) {
 const styles = useThemedStyles(createStyles);
 return (
 <View style={styles.row}>
 <View style={styles.rail}>
 <View style={[[styles.dot, { backgroundColor: color + '22', borderColor: color }]]}>
 <Ionicons name={icon} size={13} color={color} />
 </View>
 {!isLast && <View style={styles.line} />}
 </View>

 <TouchableOpacity
 style={styles.card}
 onPress={onPress}
 onLongPress={onLongPress}
 activeOpacity={onPress || onLongPress ? 0.8 : 1}
 disabled={!onPress && !onLongPress}
 >
 <View style={styles.cardTop}>
 <Text style={styles.date}>{dateLabel}</Text>
 {badge ? (
 <View style={[[styles.badge, { backgroundColor: badge.color + '22' }]]}>
 <Text style={[[styles.badgeText, { color: badge.color }]]}>{badge.label}</Text>
 </View>
) : null}
 </View>
 <Text style={styles.title}>{title}</Text>
 {lines?.map((line, i) => (
 <Text key={i} style={styles.line2}>
 {line}
 </Text>
))}
 </TouchableOpacity>
 </View>
);
});

const createStyles = (colors: Palette) => StyleSheet.create({
 row: {
 flexDirection: 'row',
 paddingHorizontal: spacing.lg,
 },
 rail: {
 width: 36,
 alignItems: 'center',
 },
});
```

```

 },
 dot: {
 width: 28,
 height: 28,
 borderRadius: radius.full,
 borderWidth: 1,
 alignItems: 'center',
 justifyContent: 'center',
 },
 line: {
 flex: 1,
 width: 2,
 backgroundColor: colors.border,
 marginVertical: 4,
 },
 card: {
 flex: 1,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 marginLeft: spacing.sm,
 marginBottom: spacing.sm,
 gap: 3,
 },
 cardTop: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 },
 date: {
 fontSize: 12,
 color: colors.textMuted,
 },
 badge: {
 paddingHorizontal: spacing.sm,
 paddingVertical: 2,
 borderRadius: radius.full,
 },
 badgeText: {
 fontSize: 11,
 fontWeight: '600',
 },
 title: {
 fontSize: 15,
 fontWeight: '600',
 color: colors.text,
 },
 line2: {
 fontSize: 13,
 color: colors.textSubtle,
 lineHeight: 19,
 },
 },
});

```

## src/components/WeightCard.tsx

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
import { Ionicons } from '@expo/vector-icons';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { displayDate } from '../utils/date';
import { WeightEntry } from '../types';

interface Props {
 /** Pesagens em ordem decrescente de data (mais recente primeiro). */
 weights: WeightEntry[];
}

export function WeightCard({ weights }: Props) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const latest = weights[0];
 const previous = weights[1];
 const oldest = weights[weights.length - 1];
 const diff = latest && previous ? latest.weightKg - previous.weightKg : null;
 const totalDiff =
 latest && oldest && weights.length >= 3 ? latest.weightKg - oldest.weightKg : null;

 return (
 <View>
 <View style={styles.card}>
 <View style={styles.iconWrapper}>
 <Ionicons name="scale" size={20} color={colors.primaryLight} />
 </View>
 <View style={{ flex: 1 }}>
 <Text style={styles.label}>Último peso</Text>
 <Text style={styles.value}>
 {latest ? `${latest.weightKg.toLocaleString('pt-BR')} kg` : '-'}
 </Text>
 {latest ? <Text style={styles.date}>{displayDate(latest.date)}</Text> : null}
 </View>
 {diff !== null && (
 <View style={styles.diffWrapper}>
 <Ionicons
 name={diff === 0 ? 'remove' : diff > 0 ? 'trending-up' : 'trending-down'}
 size={16}
 color={diff === 0 ? colors.textMuted : diff > 0 ? colors.warning : colors.success}
 />
 <Text
 style={[
 styles.diff,
 { color: diff === 0 ? colors.textMuted : diff > 0 ? colors.warning : colors.success },
]}
 >
 {diff > 0 ? '+' : ''}
 {diff.toLocaleString('pt-BR', { maximumFractionDigits: 2 })} kg
 </Text>
 </View>
)}
 </View>
 {totalDiff !== null && (
 <Text style={styles.totalDiff}>
 {totalDiff > 0 ? '+' : totalDiff < 0 ? '-' : '±'}
 {totalDiff.toLocaleString('pt-BR', { maximumFractionDigits: 2 })} kg desde {displayDate(oldest.date)}
 </Text>
)}
 </View>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 card: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 },
 iconWrapper: {
 width: 44,
 },
});
```

```
height: 44,
borderRadius: radius.full,
backgroundColor: colors.surfaceElevated,
alignItems: 'center',
justifyContent: 'center',
},
label: { fontSize: 12, color: colors.textMuted },
value: { fontSize: 20, fontWeight: '700', color: colors.text },
date: { fontSize: 12, color: colors.textMuted },
diffWrapper: { flexDirection: 'row', alignItems: 'center', gap: 4 },
diff: { fontSize: 14, fontWeight: '600' },
totalDiff: { fontSize: 12, color: colors.textMuted, marginTop: spacing.xs, marginLeft: spacing.xs },
});
```

## src/components/WeightChart.tsx

```
import React from 'react';
import { View, Text, ScrollView, StyleSheet } from 'react-native';
import { spacing, radius, useThemedStyles, Palette } from '../theme';
import { displayDate } from '../utils/date';
import { WeightEntry } from '../types';

const CHART_HEIGHT = 120;

interface Props {
 /** Pesagens em ordem decrescente de data (mais recente primeiro). */
 weights: WeightEntry[];
 title?: string;
}

export function WeightChart({ weights, title = 'Evolução' }: Props) {
 const styles = useThemedStyles(createStyles);
 // Ordem cronológica para o gráfico.
 const data = [...weights].reverse();
 if (data.length < 2) return null;

 const values = data.map(w => w.weightKg);
 const max = Math.max(...values);
 const min = Math.min(...values);
 const range = max - min || 1;

 return (
 <View style={styles.wrapper}>
 <Text style={styles.title}>{title}</Text>
 <ScrollView horizontal showsHorizontalScrollIndicator={false}>
 <View style={styles.bars}>
 {data.map(w => {
 const ratio = 0.25 + (0.75 * (w.weightKg - min)) / range;
 return (
 <View key={w.id} style={styles.barColumn}>
 <Text style={styles.barValue}>
 {w.weightKg.toLocaleString('pt-BR', { maximumFractionDigits: 1 })}
 </Text>
 <View style={[styles.bar, { height: CHART_HEIGHT * ratio }]} />
 <Text style={styles.barLabel}>{displayDate(w.date).slice(0, 5)}</Text>
 </View>
);
 })}
 </ScrollView>
 </View>
);
 }

const createStyles = (colors: Palette) => StyleSheet.create({
 wrapper: {
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 gap: spacing.sm,
 },
 title: { fontSize: 13, fontWeight: '600', color: colors.textSubtle },
 bars: { flexDirection: 'row', alignItems: 'flex-end', gap: spacing.sm },
 barColumn: { alignItems: 'center', gap: 4, width: 44 },
 bar: {
 width: 22,
 borderRadius: 6,
 backgroundColor: colors.primary,
 },
 barValue: { fontSize: 11, color: colors.textSubtle, fontWeight: '600' },
 barLabel: { fontSize: 10, color: colors.textMuted },
});
```

## src/screens/OnboardingScreen.tsx

```
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, typography, useThemedStyles, Palette } from '../theme';
import { onboardingRepository } from '../repositories/onboardingRepository';
import { Button } from '../components/Button';
import { RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Onboarding'>;

const SLIDES: Array<{ image: any; title: string; text: string }> = [
 {
 image: require('../assets/icons/paw.png'),
 title: 'Cadastre seus pets',
 text: 'Adicione foto, espécie, raça e data de nascimento de cada um dos seus companheiros.',
 },
 {
 image: require('../assets/icons/heart.png'),
 title: 'Registre vacinas, consultas, remédios e peso',
 text: 'Mantenha o histórico de saúde completo e organizado, sempre à mão.',
 },
 {
 image: require('../assets/icons/shield.png'),
 title: 'Receba lembretes',
 text: 'Vacinas, retornos e fim de tratamentos avisam você antes de vencer.',
 },
 {
 image: require('../assets/icons/document.png'),
 title: 'Guarde documentos importantes',
 text: 'Exames, receitas e carteiras de vacinação digitalizados em um só lugar.',
 },
];

export default function OnboardingScreen() {
 const navigation = useNavigation<Nav>();
 const styles = useThemedStyles(createStyles);

 async function handleStart() {
 await onboardingRepository.setSeen();
 navigation.replace('Dashboard');
 }

 return (
 <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
 <View style={styles.header}>
 <Text style={styles.appName}>PetCare</Text>
 <Text style={styles.subtitle}>Tudo sobre a saúde do seu pet, em um só lugar</Text>
 </View>

 <View style={styles.slides}>
 {SLIDES.map(slide => (
 <View key={slide.title} style={styles.slide}>
 <View style={styles.iconWrap}>
 <Image source={slide.image} style={styles.icon} resizeMode="contain" />
 </View>
 <View style={styles.slideText}>
 <Text style={styles.slideTitle}>{slide.title}</Text>
 <Text style={styles.slideBody}>{slide.text}</Text>
 </View>
 </View>
))}
 </View>

 <View style={styles.footer}>
 <Button label="Começar" onPress={handleStart} accessibilityLabel="Começar a usar o PetCare" />
 </View>
 </SafeAreaView>
);
}

const createStyles = (colors: Palette) =>
 StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 paddingHorizontal: spacing.lg,
 },
 });
```

```

paddingTop: spacing.lg,
paddingBottom: spacing.xl,
},
appName: {
 fontSize: typography.h1.fontSize,
 fontWeight: typography.h1.fontWeight,
 color: colors.primary,
 letterSpacing: -0.5,
},
subtitle: {
 fontSize: typography.body.fontSize,
 color: colors.textMuted,
 marginTop: spacing.xs,
},
slides: {
 flex: 1,
 paddingHorizontal: spacing.lg,
 justifyContent: 'center',
 gap: spacing.xl,
},
slide: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
},
iconWrap: {
 width: 56,
 height: 56,
 borderRadius: 28,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 alignItems: 'center',
 justifyContent: 'center',
},
icon: { width: 28, height: 28 },
slideText: { flex: 1, gap: 2 },
slideTitle: {
 fontSize: typography.h4.fontSize,
 fontWeight: typography.h4.fontWeight,
 color: colors.text,
},
slideBody: {
 fontSize: typography.label.fontSize,
 color: colors.textSubtle,
 lineHeight: 18,
},
footer: {
 paddingHorizontal: spacing.lg,
 paddingBottom: spacing.lg,
},
});

```

## src/screens/DashboardScreen.tsx

```
import React, { useState, useCallback } from 'react';
import {
 View,
 Text,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 Alert,
 Image,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useFocusEffect, useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, typography, useTheme, useThemedStyles, Palette } from '../theme';
import { petsRepository } from '../repositories/petsRepository';
import { recordsRepository } from '../repositories/recordsRepository';
import { getUpcomingEvents, isActiveMedication, UpcomingEvent } from '../services/events';
import { syncEventNotifications } from '../services/notifications';
import { StatCard } from '../components/StatCard';
import { ReminderCard } from '../components/ReminderCard';
import { EmptyState } from '../components/EmptyState';
import { Button } from '../components/Button';
import { RECORD_TYPE_ICONS, RECORD_TYPE_LABELS, recordTypeColors } from '../labels';
import { displayDate } from '../utils/date';
import { Pet, MedicalRecord, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Dashboard'>;

function getGreeting(): string {
 const hour = new Date().getHours();
 if (hour < 12) return 'Bom dia';
 if (hour < 18) return 'Boa tarde';
 return 'Boa noite';
}

export default function DashboardScreen() {
 const navigation = useNavigation<Nav>();
 const { colors, scheme, toggleTheme } = useTheme();
 const styles = useThemedStyles(createStyles);
 const TYPE_COLORS = recordTypeColors(colors);
 const [pets, setPets] = useState<Pet[]>([]);
 const [records, setRecords] = useState<MedicalRecord[]>([]);
 const [events, setEvents] = useState<UpcomingEvent[]>([]);
 const [activeMedsCount, setActiveMedsCount] = useState(0);
 const [showAllUpcoming, setShowAllUpcoming] = useState(false);

 useFocusEffect(
 useCallback(() => {
 Promise.all([petsRepository.getAll(), recordsRepository.getAll()])
 .then(([loadedPets, loadedRecords]) => {
 setPets(loadedPets);
 setRecords(loadedRecords);
 setEvents(getUpcomingEvents(loadedPets, loadedRecords));
 setActiveMedsCount(loadedRecords.filter(isActiveMedication).length);
 // Mantém as notificações locais alinhadas com os eventos atuais.
 syncEventNotifications(loadedPets, loadedRecords).catch(() => {});
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar a visão geral.'));
 }, []),
);

 const alerts = events.filter(e => e.pending);
 const allUpcoming = events.filter(e => !e.pending && e.daysUntil >= 0);
 const upcoming = showAllUpcoming ? allUpcoming : allUpcoming.slice(0, 5);
 const hasPets = pets.length > 0;
 const recentRecords = [...records]
 .sort((a, b) => b.createdAt.localeCompare(a.createdAt))
 .slice(0, 3);

 const petName = useCallback((petId: string): string => {
 return pets.find(p => p.id === petId)?.name ?? '';
 }, [pets]);

 const handleNewRecord = useCallback(() => {
 if (pets.length === 0) {
 navigation.navigate('AddPet', {});
 }
 }, [navigation, pets]);
```

```

 return;
 }
 if (pets.length === 1) {
 navigation.navigate('AddRecord', { petId: pets[0].id });
 return;
 }
 Alert.alert(
 'Novo registro',
 'Para qual pet?',
 [
 ...pets.map(p => ({
 text: p.name,
 onPress: () => navigation.navigate('AddRecord', { petId: p.id }),
 })),
 { text: 'Cancelar', style: 'cancel' as const },
],
);
}, [pets, navigation]);

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <ScrollView contentContainerStyle={styles.scroll} showsVerticalScrollIndicator={false}>
 <View style={styles.header}>
 <Image
 source={
 scheme === 'dark'
 ? require('../assets/logo-dark.png')
 : require('../assets/logo-light.png')
 }
 style={styles.logo}
 resizeMode="contain"
 />
 <View style={styles.headerActions}>
 <TouchableOpacity
 onPress={toggleTheme}
 hitSlop={{ top: 12, bottom: 12, left: 8, right: 8 }}
 accessibilityRole="button"
 accessibilityLabel={scheme === 'dark' ? 'Mudar para tema claro' : 'Mudar para tema escuro'}
 >
 <Ionicons
 name={scheme === 'dark' ? 'sunny-outline' : 'moon-outline'}
 size={21}
 color={colors.textSubtle}
 />
 </TouchableOpacity>
 <TouchableOpacity
 onPress={() => navigation.navigate('Search')}
 hitSlop={{ top: 12, bottom: 12, left: 8, right: 8 }}
 accessibilityRole="button"
 accessibilityLabel="Buscar registros"
 >
 <Ionicons name="search" size={22} color={colors.textSubtle} />
 </TouchableOpacity>
 <TouchableOpacity
 onPress={() => navigation.navigate('About')}
 hitSlop={{ top: 12, bottom: 12, left: 8, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Sobre o app"
 >
 <Ionicons name="information-circle-outline" size={22} color={colors.textSubtle} />
 </TouchableOpacity>
 </View>
 </View>

 <Text style={styles.title}>{getGreeting()}!</Text>
 <Text style={styles.subtitle}>
 {hasPets ? 'Aqui está o resumo de hoje' : 'Vamos cadastrar seu primeiro pet?'}
 </Text>

 {hasPets ? (
 <>
 <View style={styles.statsRow}>
 <StatCard icon="paw" label="Pets" value={pets.length} />
 <StatCard
 icon="notifications"
 label="Alertas pendentes"
 value={alerts.length}
 color={alerts.length > 0 ? colors.warning : colors.success}
 />
 <StatCard
 icon="medkit"

```

```

 label="Remédios ativos"
 value={activeMedsCount}
 color={colors.accent}
 onPress={() => navigation.navigate('Home')}
 />
 </View>

 <View style={styles.quickActions}>
 <View style={styles.quickActionItem}>
 <Button
 label="Adicionar pet"
 icon="add-circle-outline"
 variant="secondary"
 onPress={() => navigation.navigate('AddPet', {})}
 />
 </View>
 <View style={styles.quickActionItem}>
 <Button
 label="Novo registro"
 icon="create-outline"
 variant="secondary"
 onPress={handleNewRecord}
 />
 </View>
 </View>

 {alerts.length > 0 && (
 <View style={styles.section}>
 <Text style={styles.sectionTitle}>⚠️ Alertas</Text>
 {alerts.map(event => (
 <ReminderCard
 key={event.key}
 event={event}
 onPress={() => navigation.navigate('PetDetail', { petId: event.petId })}
 />
))}
 </View>
)}

 <View style={styles.section}>
 <Text style={styles.sectionTitle}>Próximos eventos</Text>
 {upcoming.length === 0 && alerts.length === 0 ? (
 <View style={styles.calmCard}>
 <Icons name="checkmark-circle" size={18} color={colors.success} />
 <Text style={styles.calmText}>
 Tudo em dia! Nenhum reforço, retorno ou dose programada para os próximos dias.
 </Text>
 </View>
) : upcoming.length === 0 ? (
 <Text style={styles.emptyHint}>Nenhum outro evento programado.</Text>
) : (
 upcoming.map(event => (
 <ReminderCard
 key={event.key}
 event={event}
 onPress={() => navigation.navigate('PetDetail', { petId: event.petId })}
 />
))
)}
 {!showAllUpcoming && allUpcoming.length > 5 && (
 <TouchableOpacity
 onPress={() => setShowAllUpcoming(true)}
 activeOpacity={0.7}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel={`Ver mais ${allUpcoming.length - 5} eventos`}
 >
 <Text style={styles.seeMore}>Ver mais {allUpcoming.length - 5}</Text>
 </TouchableOpacity>
)}
 </View>

 {recentRecords.length > 0 && (
 <View style={styles.section}>
 <Text style={styles.sectionTitle}>Últimos registros</Text>
 {recentRecords.map(record => (
 <TouchableOpacity
 key={record.id}
 style={styles.recordRow}
 onPress={() => navigation.navigate('PetDetail', { petId: record.petId })}
 activeOpacity={0.7}
 />
))}
 </View>
)}
 </View>
)
}

```

```

 accessibilityRole="button"
 accessibilityLabel={` ${RECORD_TYPE_LABELS[record.type]}: ${record.title}, ${petName(record.petId)}`}
 >
 <View style={[styles.recordIcon, { backgroundColor: TYPE_COLORS[record.type] + '22' }]}>
 <Ionicons name={RECORD_TYPE_ICONS[record.type]} size={16} color={TYPE_COLORS[record.type]} />
 </View>
 <View style={styles.recordInfo}>
 <Text style={styles.recordTitle} numberOfLines={1}>{record.title}</Text>
 <Text style={styles.recordMeta} numberOfLines={1}>
 {petName(record.petId)} · {RECORD_TYPE_LABELS[record.type]} · {displayDate(record.date)}
 </Text>
 </View>
 <Ionicons name="chevron-forward" size={16} color={colors.textMuted} />
 </TouchableOpacity>
)}
</View>
)}

<TouchableOpacity
 style={styles.petsBtn}
 onPress={() => navigation.navigate('Home')}
 activeOpacity={0.85}
 accessibilityRole="button"
 accessibilityLabel="Ver seus pets"
>
 <Ionicons name="paw" size={18} color={colors.onPrimary} />
 <Text style={styles.petsBtnText}>
 Seus Pets ({pets.length})
 </Text>
 <Ionicons name="chevron-forward" size={16} color={colors.onPrimary} />
</TouchableOpacity>
</>
) : (
 <View style={styles.emptyWrapper}>
 <EmptyState
 image={require('../../assets/icons/paw.png')}
 title="Bem-vindo ao PetCare"
 text="Cadastro seu primeiro pet para acompanhar vacinas, consultas, peso e muito mais."
 />
 <TouchableOpacity
 style={styles.petsBtn}
 onPress={() => navigation.navigate('AddPet', {})}
 activeOpacity={0.85}
 accessibilityRole="button"
 accessibilityLabel="Adicionar pet"
 >
 <Ionicons name="add" size={18} color={colors.onPrimary} />
 <Text style={styles.petsBtnText}>Adicionar Pet</Text>
 </TouchableOpacity>
 </View>
)
 </ScrollView>
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 scroll: { paddingHorizontal: spacing.lg, paddingBottom: 48 },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingTop: spacing.sm,
 paddingBottom: spacing.xs,
 },
 headerActions: { flexDirection: 'row', alignItems: 'center', gap: spacing.md },
 logo: {
 width: 150,
 height: 50,
 },
 title: {
 fontSize: typography.h1.fontSize,
 fontWeight: typography.h1.fontWeight,
 color: colors.text,
 letterSpacing: -0.5,
 marginTop: spacing.sm,
 },
 subtitle: {
 fontSize: typography.body.fontSize,
 color: colors.textMuted,
 },
});

```

```

 marginTop: 2,
 marginBottom: spacing.lg,
 },
 statsRow: { flexDirection: 'row', gap: spacing.sm, marginBottom: spacing.md },
 quickActions: { flexDirection: 'row', gap: spacing.sm, marginBottom: spacing.lg },
 quickActionItem: { flex: 1 },
 section: { gap: spacing.sm, marginBottom: spacing.lg },
 sectionTitle: { fontSize: typography.h4.fontSize, fontWeight: typography.h4.fontWeight, color: colors.text, marginBottom: 2 },
 calmCard: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.sm,
 backgroundColor: colors.success + '12',
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.success + '33',
 padding: spacing.md,
 },
 calmText: { flex: 1, fontSize: 13, color: colors.textSubtle, lineHeight: 19 },
 emptyHint: { fontSize: 13, color: colors.textMuted },
 seeMore: {
 fontSize: 13,
 fontWeight: '600',
 color: colors.primaryLight,
 textAlign: 'center',
 paddingVertical: spacing.xs,
 },
 recordRow: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.sm,
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.sm + 2,
 },
 recordIcon: {
 width: 32,
 height: 32,
 borderRadius: radius.full,
 alignItems: 'center',
 justifyContent: 'center',
 },
 recordInfo: { flex: 1, gap: 1 },
 recordTitle: { fontSize: 14, fontWeight: '600', color: colors.text },
 recordMeta: { fontSize: 12, color: colors.textSubtle },
 petsBtn: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'center',
 gap: spacing.sm,
 backgroundColor: colors.primary,
 borderRadius: radius.md,
 paddingVertical: spacing.md,
 marginTop: spacing.sm,
 },
 petsBtnText: { fontSize: 15, fontWeight: '600', color: colors.onPrimary },
 emptyWrapper: { marginTop: spacing.xxl, gap: spacing.lg },
});

```

## src/screens/HomeScreen.tsx

```
import React, { useState, useCallback } from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet, Alert, Image } from 'react-native';
import { SafeAreaView, useSafeAreaInsets } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useFocusEffect, useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getPets, getAllRecords } from '../storage';
import { getUpcomingEvents, isActiveMedication } from '../services/events';
import { PetCard } from '../components/PetCard';
import { EmptyState } from '../components/EmptyState';
import { ThemeToggle } from '../components/ThemeToggle';
import { Pet, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Home'>;

export default function HomeScreen() {
 const navigation = useNavigation<Nav>();
 const insets = useSafeAreaInsets();
 const styles = useThemedStyles(createStyles);
 const { scheme } = useTheme();
 const [pets, setPets] = useState<Pet[]>([]);
 const [activeMedsByPet, setActiveMedsByPet] = useState<Record<string, number>>({});
 const [alertsByPet, setAlertsByPet] = useState<Record<string, number>>({});

 useFocusEffect(
 useCallback(() => {
 Promise.all([getPets(), getAllRecords()])
 .then(([loadedPets, records]) => {
 setPets(loadedPets);

 const recordsByPet = new Map<string, typeof records>();
 records.forEach(r => {
 const list = recordsByPet.get(r.petId);
 if (list) list.push(r);
 else recordsByPet.set(r.petId, [r]);
 });

 const meds: Record<string, number> = {};
 loadedPets.forEach(pet => {
 meds[pet.id] = (recordsByPet.get(pet.id) ?? []).filter(isActiveMedication).length;
 });

 const alerts: Record<string, number> = {};
 getUpcomingEvents(loadedPets, records).forEach(e => {
 if (e.pending) alerts[e.petId] = (alerts[e.petId] ?? 0) + 1;
 });

 setActiveMedsByPet(meds);
 setAlertsByPet(alerts);
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar seus pets.'));
 }, []),
);

 const isEmpty = pets.length === 0;
 const fabBottom = Math.max(36, insets.bottom + 16);

 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <Image
 source={
 scheme === 'dark'
 ? require('../assets/logo-dark.png')
 : require('../assets/logo-light.png')
 }
 style={styles.headerLogo}
 resizeMode="contain"
 />
 <View style={styles.headerSpacer} />
 <ThemeToggle />
 </View>

 <View style={styles.titleSection}>
 <Text style={styles.title}>Seus Pets</Text>
 {!isEmpty && (
```

```

 <Text style={styles.subtitle}>
 {pets.length} {pets.length === 1 ? 'pet cadastrado' : 'pets cadastrados'}
 </Text>
)}
 </View>

 <FlatList
 data={pets}
 keyExtractor={item => item.id}
 renderItem={({ item }) => (
 <PetCard
 pet={item}
 activeMeds={activeMedsByPet[item.id]}
 pendingAlerts={alertsByPet[item.id]}
 onPress={() => navigation.navigate('PetDetail', { petId: item.id })}
 />
)}
 contentContainerStyle={isEmpty ? styles.emptyContainer : styles.listContainer}
 showsVerticalScrollIndicator={false}
 ListEmptyComponent={
 <EmptyState
 image={require('../assets/icons/paw.png')}
 title="Nenhum pet cadastrado"
 text="Adicione seu primeiro pet e comece a registrar o histórico de saúde dele."
 />
 }
 />

 <TouchableOpacity
 style={[styles.fab, { bottom: fabBottom }]}
 onPress={() => navigation.navigate('AddPet', {})}
 activeOpacity={0.85}
 accessibilityRole="button"
 accessibilityLabel="Adicionar pet"
 >
 <Ionicons name="add" size={30} color="#fff" />
 </TouchableOpacity>
 </SafeAreaView>
);
}

```

```

const createStyles = (colors: Palette) => StyleSheet.create({
 container: {
 flex: 1,
 backgroundColor: colors.background,
 },
 header: {
 paddingHorizontal: spacing.lg,
 paddingTop: spacing.sm,
 paddingBottom: spacing.xs,
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.xs,
 },
 headerLogo: {
 width: 150,
 height: 50,
 },
 headerSpacer: {
 flex: 1,
 },
 titleSection: {
 paddingHorizontal: spacing.lg,
 paddingTop: spacing.sm,
 paddingBottom: spacing.lg,
 },
 title: {
 fontSize: 34,
 fontWeight: '700',
 color: colors.text,
 letterSpacing: -0.5,
 },
 subtitle: {
 fontSize: 14,
 color: colors.textMuted,
 marginTop: 4,
 },
 listContainer: {
 paddingHorizontal: spacing.lg,
 gap: spacing.sm,
 paddingBottom: 100,
 },
});

```

```
},
emptyContainer: {
 flex: 1,
 paddingHorizontal: spacing.lg,
 justifyContent: 'center',
 paddingBottom: 80,
},
fab: {
 position: 'absolute',
 right: spacing.lg,
 width: 62,
 height: 62,
 borderRadius: radius.full,
 backgroundColor: colors.primary,
 alignItems: 'center',
 justifyContent: 'center',
 shadowColor: colors.primary,
 shadowOffset: { width: 0, height: 6 },
 shadowOpacity: 0.5,
 shadowRadius: 16,
 elevation: 10,
},
});
```

## src/screens/AddPetScreen.tsx

```
import React, { useState, useEffect } from 'react';
import {
 View,
 Text,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 KeyboardAvoidingView,
 Platform,
 Alert,
 Image,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp } from '@react-navigation/native';
import * as ImagePicker from 'expo-image-picker';
import { Input } from '../components/Input';
import { ThemeToggle } from '../components/ThemeToggle';
import { Button } from '../components/Button';
import { useToast } from '../hooks/useToast';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getPets, savePet } from '../storage';
import { persistPhoto, deletePhoto } from '../storage/files';
import { maskDate, isValidDate, isFuture, toISO, displayDate } from '../utils/date';
import { SPECIES_LABELS } from '../labels';
import { Pet, Species, RootStackParamList } from '../types';

const SPECIES: Species[] = ['dog', 'cat', 'bird', 'other'];

export default function AddPetScreen() {
 const navigation = useNavigation();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { showToast } = useToast();
 const { petId } = useRoute<RouteProp<RootStackParamList, 'AddPet'>>().params;
 const [original, setOriginal] = useState<Pet | null>(null);
 const [name, setName] = useState('');
 const [species, setSpecies] = useState<Species>('dog');
 const [breed, setBreed] = useState('');
 const [birthDate, setBirthDate] = useState('');
 const [photo, setPhoto] = useState<string | undefined>();
 const [saving, setSaving] = useState(false);

 useEffect(() => {
 if (!petId) return;
 getPets()
 .then(pets => {
 const pet = pets.find(p => p.id === petId);
 if (!pet) return;
 setOriginal(pet);
 setName(pet.name);
 setSpecies(pet.species);
 setBreed(pet.breed);
 setBirthDate(displayDate(pet.birthDate));
 setPhoto(pet.photo);
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar os dados do pet.'));
 }, [petId]);

 const PICKER_OPTIONS = {
 mediaTypes: ['images'],
 allowsEditing: true,
 aspect: [1, 1],
 quality: 0.8,
 };
 satisfies ImagePicker.ImagePickerOptions;

 async function pickFromGallery() {
 const perm = await ImagePicker.requestMediaLibraryPermissionsAsync();
 if (!perm.granted) {
 Alert.alert('Permissão necessária', 'Precisamos de acesso à galeria para selecionar a foto.');
```

```

const perm = await ImagePicker.requestCameraPermissionsAsync();
if (!perm.granted) {
 Alert.alert('Permissão necessária', 'Precisamos de acesso à câmera para tirar a foto.');
```

return;

}

```

const result = await ImagePicker.launchCameraAsync(PICKER_OPTIONS);
if (!result.canceled) setPhoto(result.assets[0].uri);
}

function pickPhoto() {
 const options: Array<{ text: string; onPress?: () => void; style?: 'cancel' | 'destructive' }> = [
 { text: 'Tirar foto', onPress: pickFromCamera },
 { text: 'Galeria', onPress: pickFromGallery },
];
 if (photo) {
 options.push({ text: 'Remover foto', onPress: () => setPhoto(undefined), style: 'destructive' });
 }
 options.push({ text: 'Cancelar', style: 'cancel' });
 Alert.alert('Foto do pet', 'De onde vem a foto?', options);
}

async function handleSave() {
 if (!name.trim()) {
 Alert.alert('Campo obrigatório', 'Informe o nome do pet.');
```

return;

}

```

 if (birthDate && !isValidDate(birthDate)) {
 Alert.alert('Data inválida', 'Verifique a data de nascimento (DD/MM/AAAA).');
```

return;

}

```

 if (birthDate && isFuture(birthDate)) {
 Alert.alert('Data inválida', 'A data de nascimento não pode ser no futuro.');
```

return;

}

```

 setSaving(true);
 try {
 const id = original?.id ?? `${Date.now()}-${Math.random().toString(36).slice(2)}`;
 const storedPhoto = photo ? await persistPhoto(photo, id) : undefined;
 if (original?.photo && original.photo !== storedPhoto) {
 await deletePhoto(original.photo);
 }
 await savePet({
 id,
 name: name.trim(),
 species,
 breed: breed.trim(),
 birthDate: birthDate ? toISO(birthDate) : '',
 photo: storedPhoto,
 createdAt: original?.createdAt ?? new Date().toISOString(),
 });
 navigation.goBack();
 showToast(petId ? 'Pet atualizado com sucesso' : 'Pet adicionado com sucesso');
```

} catch {

```

 setSaving(false);
 Alert.alert('Erro', 'Não foi possível salvar o pet. Tente novamente.');
```

}

}

```

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>{petId ? 'Editar Pet' : 'Novo Pet'}</Text>
 <ThemeToggle />
 </View>

 <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>
 <ScrollView contentContainerStyle={styles.scroll} keyboardShouldPersistTaps="handled">

 <TouchableOpacity
 style={styles.photoBtn}
 onPress={pickPhoto}
 activeOpacity={0.8}
 accessibilityRole="button"
 >

```

```

 accessibilityLabel="Adicionar foto do pet"
 >
 {photo ? (
 <Image source={{ uri: photo }} style={styles.photo} />
) : (
 <View style={styles.photoPlaceholder}>
 <Ionicons name="camera" size={28} color={colors.primaryLight} />
 </View>
)}
 <View style={styles.photoBadge}>
 <Ionicons name="add" size={14} color="#fff" />
 </View>
 </TouchableOpacity>
 <Text style={styles.photoHint}>Toque para adicionar foto</Text>

 <View style={styles.form}>
 <Input
 label="Nome do pet"
 placeholder="Ex: Thor"
 value={name}
 onChangeText={setName}
 autoCapitalize="words"
 returnKeyType="next"
 />

 <View style={styles.field}>
 <Text style={styles.fieldLabel}>Espécie</Text>
 <View style={styles.chips}>
 {SPECIES.map(s => (
 <TouchableOpacity
 key={s}
 style={[styles.chip, species === s && styles.chipActive]}
 onPress={() => setSpecies(s)}
 activeOpacity={0.7}
 >
 <Text style={[styles.chipText, species === s && styles.chipTextActive]}>
 {SPECIES_LABELS[s]}
 </Text>
 </TouchableOpacity>
))}
 </View>
 </View>

 <Input
 label="Raça (opcional)"
 placeholder="Ex: Golden Retriever"
 value={breed}
 onChangeText={setBreed}
 autoCapitalize="words"
 returnKeyType="next"
 />

 <Input
 label="Data de nascimento (opcional)"
 placeholder="DD/MM/AAAA"
 value={birthDate}
 onChangeText={t => setBirthDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 returnKeyType="done"
 />
 </View>

 <Button
 label={petId ? 'Salvar Alterações' : 'Salvar Pet'}
 onPress={handleSave}
 loading={saving}
 />
 </ScrollView>
</KeyboardAvoidingView>
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 },
});

```

```

paddingVertical: spacing.md,
},
headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
scroll: {
paddingHorizontal: spacing.lg,
paddingBottom: 48,
alignItems: 'center',
},
photoBtn: {
width: 100,
height: 100,
borderRadius: 50,
marginTop: spacing.md,
marginBottom: spacing.sm,
position: 'relative',
},
photo: { width: 100, height: 100, borderRadius: 50 },
photoPlaceholder: {
width: 100,
height: 100,
borderRadius: 50,
backgroundColor: colors.surface,
borderWidth: 1,
borderColor: colors.border,
alignItems: 'center',
justifyContent: 'center',
},
photoBadge: {
position: 'absolute',
bottom: 2,
right: 2,
width: 26,
height: 26,
borderRadius: 13,
backgroundColor: colors.primary,
alignItems: 'center',
justifyContent: 'center',
borderWidth: 2,
borderColor: colors.background,
},
photoHint: { fontSize: 13, color: colors.textMuted, marginBottom: spacing.xl },
form: { width: '100%', gap: spacing.lg, marginBottom: spacing.xl },
field: { gap: spacing.xs },
fieldLabel: { fontSize: 13, fontWeight: '500', color: colors.textSubtle, letterSpacing: 0.3 },
chips: { flexDirection: 'row', gap: spacing.sm, flexWrap: 'wrap' },
chip: {
paddingHorizontal: spacing.md,
paddingVertical: spacing.sm,
borderRadius: radius.full,
backgroundColor: colors.surface,
borderWidth: 1,
borderColor: colors.border,
},
chipActive: { backgroundColor: colors.primary, borderColor: colors.primary },
chipText: { fontSize: 14, color: colors.textMuted, fontWeight: '500' },
chipTextActive: { color: '#fff' },
});

```

## src/screens/PetDetailScreen.tsx

```
import React, { useState, useCallback, useMemo } from 'react';
import {
 View,
 Text,
 SectionList,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 Image,
 Alert,
 ActivityIndicator,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, typography, useTheme, useThemedStyles, Palette } from '../theme';
import { petsRepository } from '../repositories/petsRepository';
import { recordsRepository } from '../repositories/recordsRepository';
import { weightsRepository } from '../repositories/weightsRepository';
import { documentsRepository } from '../repositories/documentsRepository';
import { calcAge, displayDate, formatDaysUntil, daysUntilISO } from '../utils/date';
import {
 SPECIES_LABELS,
 RECORD_TYPE_LABELS,
 recordTypeColors,
 RECORD_TYPE_ICONS,
 FREQUENCY_LABELS,
 DOCUMENT_KIND_LABELS,
 DOCUMENT_KIND_ICONS,
} from '../labels';
import { getVaccineStatus, eventDateOf, isEventFulfilled, getFulfilledRecordIds } from '../services/events';
import { sharePetSummary } from '../services/share';
import { sharePetPdf } from '../services/pdf';
import { TimelineItem, TimelineBadge } from '../components/TimelineItem';
import { EmptyState } from '../components/EmptyState';
import { ThemeToggle } from '../components/ThemeToggle';
import { MedicalProfileCard } from '../components/MedicalProfileCard';
import { Pet, MedicalRecord, WeightEntry, PetDocument, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'PetDetail'>;
type Route = RouteProp<RootStackParamList, 'PetDetail'>;

type FilterKey = 'all' | 'vaccine' | 'consultation' | 'medication' | 'deworming' | 'note' | 'weight' | 'document';

const FILTERS: { key: FilterKey; label: string }[] = [
 { key: 'all', label: 'Todos' },
 { key: 'vaccine', label: 'Vacinas' },
 { key: 'consultation', label: 'Consultas' },
 { key: 'medication', label: 'Remédios' },
 { key: 'deworming', label: 'Vermífugos' },
 { key: 'weight', label: 'Peso' },
 { key: 'document', label: 'Documentos' },
 { key: 'note', label: 'Observações' },
];

const MONTH_NAMES = [
 'Janeiro', 'Fevereiro', 'Março', 'Abril', 'Maio', 'Junho',
 'Julho', 'Agosto', 'Setembro', 'Outubro', 'Novembro', 'Dezembro',
];

function monthLabel(isoDate: string): string {
 const [year, month] = isoDate.split('-').map(Number);
 return `${MONTH_NAMES[month - 1]} ${year}`;
}

interface TimelineEntry {
 id: string;
 date: string;
 createdAt: string;
 kind: FilterKey;
 record?: MedicalRecord;
 weight?: WeightEntry;
 document?: PetDocument;
}

function QuickAction({
```

```

 icon,
 label,
 color,
 onPress,
 }; {
 icon: keyof typeof Icons.glyphMap;
 label: string;
 color?: string;
 onPress: () => void;
 }) {
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const tint = color ?? colors.primaryLight;
 return (
 <TouchableOpacity
 style={styles.quickAction}
 onPress={onPress}
 activeOpacity={0.8}
 accessibilityRole="button"
 accessibilityLabel={label}
 >
 <View style={styles.quickActionIcon, { backgroundColor: tint + '18' }}>
 <Icons name={icon} size={18} color={tint} />
 </View>
 <Text style={styles.quickActionLabel}>{label}</Text>
 </TouchableOpacity>
);
 }

function vaccineStatusBadges(colors: Palette): Record<string, TimelineBadge> {
 return {
 ok: { label: 'Em dia', color: colors.success },
 due_soon: { label: 'Reforço próximo', color: colors.warning },
 overdue: { label: 'Atrasada', color: colors.danger },
 completed: { label: 'Reforço aplicado', color: colors.info },
 };
}

/** Badge de status para consultas, vermífugos e remédios (vacinas usam vaccineStatusBadges). */
function extendedStatusBadge(
 record: MedicalRecord,
 all: MedicalRecord[],
 fulfilledIds: Set<string>,
 colors: Palette,
): TimelineBadge | undefined {
 const eventDate = eventDateOf(record);
 if (!eventDate) return undefined;

 if (record.type === 'medication') {
 if (daysUntilISO(eventDate) < 0) return { label: 'Tratamento concluído', color: colors.info };
 return undefined;
 }

 if (isEventFulfilled(record, all, fulfilledIds)) {
 return { label: record.type === 'consultation' ? 'Retorno realizado' : 'Dose aplicada', color: colors.info };
 }
 const days = daysUntilISO(eventDate);
 if (days < 0) return { label: 'Atrasado', color: colors.danger };
 if (days <= 15) return { label: 'Próximo', color: colors.warning };
 return undefined;
}

function recordLines(record: MedicalRecord): string[] {
 const lines: string[] = [];
 if (record.type === 'vaccine') {
 const details = [record.manufacturer, record.batch, record.clinic].filter(Boolean).join(' · ');
 if (details) lines.push(details);
 if (record.nextDate) lines.push(`Reforço: ${displayDate(record.nextDate)}`);
 }
 if (record.type === 'consultation') {
 const details = [record.vet, record.clinic].filter(Boolean).join(' · ');
 if (details) lines.push(details);
 if (record.diagnosis) lines.push(`Diagnóstico: ${record.diagnosis}`);
 if (record.nextDate) lines.push(`Retorno: ${displayDate(record.nextDate)}`);
 }
 if (record.type === 'medication') {
 const details = [
 record.frequency ? FREQUENCY_LABELS[record.frequency] : undefined,
 record.endDate ? `até ${displayDate(record.endDate)}` : undefined,
].filter(Boolean).join(' · ');
 if (details) lines.push(details);
 }
}

```

```

 }
 if (record.type === 'deworming' && record.nextDate) {
 lines.push(`Próxima dose: ${displayDate(record.nextDate)}`);
 }
 if (record.description) lines.push(record.description);
 return lines;
 }

export default function PetDetailScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const RECORD_TYPE_COLORS = recordTypeColors(colors);
 const VACCINE_STATUS_BADGES = vaccineStatusBadges(colors);
 const { petId } = useRoute<Route>().params;
 const [pet, setPet] = useState<Pet | null>(null);
 const [records, setRecords] = useState<MedicalRecord[]>([]);
 const [weights, setWeights] = useState<WeightEntry[]>([]);
 const [documents, setDocuments] = useState<PetDocument[]>([]);
 const [loading, setLoading] = useState(true);
 const [filter, setFilter] = useState<FilterKey>('all');
 const fulfilledIds = useMemo(() => getFulfilledRecordIds(records), [records]);

 useEffect(() => {
 useCallcallback(() => {
 setLoading(true);
 Promise.all([
 petsRepository.getAll().then(pets => setPet(pets.find(p => p.id === petId) ?? null)),
 recordsRepository.getByPet(petId).then(setRecords),
 weightsRepository.getByPet(petId).then(setWeights),
 documentsRepository.getByPet(petId).then(setDocuments),
])
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar os dados do pet.'))
 .finally(() => setLoading(false));
 }), [petId]),
);

 function confirmDeleteRecord(id: string) {
 Alert.alert('Excluir registro', 'Deseja remover este registro?', [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Excluir',
 style: 'destructive',
 onPress: async () => {
 try {
 await recordsRepository.remove(id);
 setRecords(prev => prev.filter(r => r.id !== id));
 } catch {
 Alert.alert('Erro', 'Não foi possível excluir o registro.');
 }
 },
 },
],
);
}

function confirmDeletePet() {
 Alert.alert(
 `Excluir ${pet?.name}?`,
 'Todo o histórico de saúde também será removido. Essa ação não pode ser desfeita.',
 [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Excluir',
 style: 'destructive',
 onPress: async () => {
 try {
 await petsRepository.remove(petId);
 navigation.goBack();
 } catch {
 Alert.alert('Erro', 'Não foi possível excluir o pet.');
 }
 },
 },
],
);
}

if (loading) {
 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.loadingContainer}>

```

```

 <ActivityIndicator color={colors.primary} size="large" />
 </View>
 </SafeAreaView>
);
}

if (!pet) {
 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity>
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 </View>
 <View style={styles.loadingContainer}>
 <Text style={styles.errorText}>Pet não encontrado.</Text>
 </View>
 </SafeAreaView>
);
}

const age = calcAge(pet.birthDate);

const timeline: TimelineEntry[] = [
 ...records.map(r => ({ id: r.id, date: r.date, createdAt: r.createdAt, kind: r.type as FilterKey, record: r })),
 ...weights.map(w => ({ id: w.id, date: w.date, createdAt: w.createdAt, kind: 'weight' as FilterKey, weight: w })),
 ...documents.map(d => ({ id: d.id, date: d.date, createdAt: d.createdAt, kind: 'document' as FilterKey, document: d })),
].sort((a, b) => b.date.localeCompare(a.date) || b.createdAt.localeCompare(a.createdAt));

const filteredTimeline = filter === 'all' ? timeline : timeline.filter(e => e.kind === filter);

const sections = useMemo(() => {
 const groups = new Map<string, TimelineEntry[]>();
 for (const entry of filteredTimeline) {
 const key = entry.date.slice(0, 7);
 const group = groups.get(key);
 if (group) group.push(entry);
 else groups.set(key, [entry]);
 }
 return Array.from(groups.entries()).map(([key, data]) => ({
 title: monthLabel(`${key}-01`),
 data,
 }));
}, [filteredTimeline]);

const renderEntry = useCallback(({ item, index, section }: { item: TimelineEntry; index: number; section: { data: TimelineEntry[] } }) => {
 const isLast = index === section.data.length - 1;
 if (item.weight) {
 return (
 <TimelineItem
 icon="scale"
 color={colors.primaryLight}
 dateLabel={displayDate(item.weight.date)}
 title={`Peso: ${item.weight.weightKg.toLocaleString('pt-BR')} kg`}
 isLast={isLast}
 onPress={() => navigation.navigate('AddWeight', { petId, weightId: item.weight!.id })}
 />
);
 }
 if (item.document) {
 const doc = item.document;
 return (
 <TimelineItem
 icon={DOCUMENT_KIND_ICONS[doc.kind]}
 color={colors.info}
 dateLabel={`${displayDate(doc.date)} · ${DOCUMENT_KIND_LABELS[doc.kind]} `}
 title={doc.title}
 isLast={isLast}
 onPress={() => navigation.navigate('DocumentViewer', { uri: doc.uri, title: doc.title, mimeType: doc.mimeType })}
 />
);
 }
 const record = item.record!;
 const vaccineStatus = getVaccineStatus(record, records, fulfilledIds);
 const eventDate = eventDateOf(record);

```

```

const lines = recordLines(record);
if (eventDate && daysUntilISO(eventDate) >= 0 && !isEventFulfilled(record, records, fulfilledIds)) {
 lines.push('📅 ${formatDaysUntil(daysUntilISO(eventDate))}');
}
const badge = record.type === 'vaccine'
 ? (vaccineStatus ? VACCINE_STATUS_BADGES[vaccineStatus] : undefined)
 : extendedStatusBadge(record, records, fulfilledIds, colors);
return (
 <TimelineItem
 icon={RECORD_TYPE_ICONS[record.type]}
 color={RECORD_TYPE_COLORS[record.type]}
 dateLabel={` ${displayDate(record.date)} · ${RECORD_TYPE_LABELS[record.type]} `}
 title={record.title}
 lines={lines}
 badge={badge}
 isLast={isLast}
 onPress={() => navigation.navigate('AddRecord', { petId, recordId: record.id })}
 onLongPress={() => confirmDeleteRecord(record.id)}
 />
);
}, [records, fulfilledIds, colors, navigation, petId]);

const renderSectionHeader = useCallback(({ section }: { section: { title: string } }) => (
 <Text style={styles.monthHeader}>{section.title}</Text>
), [styles]);

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>{pet.name}</Text>
 <View style={styles.headerActions}>
 <ThemeToggle size={20} />
 <TouchableOpacity
 onPress={() => {
 Alert.alert('Compartilhar prontuário', 'Escolha o formato:', [
 {
 text: 'PDF',
 onPress: async () => {
 try {
 await sharePetPdf(pet!, records, weights);
 } catch {
 Alert.alert('Erro', 'Não foi possível gerar o PDF.');
```

```

 onPress={confirmDeletePet}
 hitSlop={{ top: 12, bottom: 12, left: 8, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Excluir pet"
 >
 <Icons name="trash-outline" size={20} color={colors.textMuted} />
 </TouchableOpacity>
 </View>
 </View>

 <SectionList
 sections={sections}
 keyExtractor={item => item.id}
 showsVerticalScrollIndicator={false}
 contentContainerStyle={styles.list}
 stickySectionHeadersEnabled={false}
 renderSectionHeader={renderSectionHeader}
 ListHeaderComponent={
 <View style={styles.hero}>
 <View style={styles.photoWrapper}>
 {pet.photo ? (
 <Image source={{ uri: pet.photo }} style={styles.photo} />
) : (
 <View style={styles.photoPlaceholder}>
 <Icons name="paw" size={32} color={colors.primaryLight} />
 </View>
)}
 </View>

 <Text style={styles.petName}><pet.name></Text>
 <Text style={styles.petMeta}>
 {[SPECIES_LABELS[pet.species], pet.breed, age].filter(Boolean).join(' · ')}
 </Text>

 <View style={styles.quickActions}>
 <QuickAction
 icon="shield-checkmark"
 label="Vacinas"
 color={colors.success}
 onPress={() => navigation.navigate('Vaccines', { petId })}
 />
 <QuickAction
 icon="scale"
 label="Peso"
 onPress={() => navigation.navigate('Weight', { petId })}
 />
 <QuickAction
 icon="folder-open"
 label="Documentos"
 onPress={() => navigation.navigate('Documents', { petId })}
 />
 <QuickAction
 icon="warning"
 label="Emergência"
 color={colors.danger}
 onPress={() => navigation.navigate('Emergency', { petId })}
 />
 <QuickAction
 icon="stats-chart"
 label="Estatísticas"
 color={colors.accent}
 onPress={() => navigation.navigate('Stats', { petId })}
 />
 <QuickAction
 icon="medkit"
 label="Remédios"
 color={colors.warning}
 onPress={() => navigation.navigate('Medications', { petId })}
 />
 </View>

 <View style={{ width: '100%', marginBottom: spacing.lg }}>
 <MedicalProfileCard
 profile={pet.medicalProfile}
 onPress={() => navigation.navigate('MedicalProfile', { petId })}
 />
 </View>

 <View style={styles.divider} />

 <View style={styles.sectionRow}>

```

```

 <Text style={styles.sectionTitle}>Linha do Tempo</Text>
 <TouchableOpacity
 style={styles.addBtn}
 onPress={() => navigation.navigate('AddRecord', { petId })}
 activeOpacity={0.8}
 accessibilityRole="button"
 accessibilityLabel="Novo registro"
 >
 <Icons name="add" size={16} color={colors.onPrimary} />
 <Text style={styles.addBtnText}>Novo</Text>
 </TouchableOpacity>
 </View>

 <ScrollView
 horizontal
 showsHorizontalScrollIndicator={false}
 style={styles.filterScroll}
 contentContainerStyle={styles.filterRow}
 >
 {FILTERS.map(f => (
 <TouchableOpacity
 key={f.key}
 style={[styles.filterChip, filter === f.key && styles.filterChipActive]}
 onPress={() => setFilter(f.key)}
 activeOpacity={0.7}
 accessibilityRole="button"
 accessibilityLabel={`Filtrar por ${f.label}`}
 accessibilityState={{ selected: filter === f.key }}
 >
 <Text style={[styles.filterChipText, filter === f.key && styles.filterChipTextActive]}>
 {f.label}
 </Text>
 </TouchableOpacity>
))}
 </ScrollView>
 </View>
 }
 renderItem={renderEntry}
 ListEmptyComponent={
 <EmptyState
 image={require('../../assets/icons/document.png')}
 title={filter === 'all' ? 'Nenhum registro ainda' : 'Nada por aqui'}
 text={
 filter === 'all'
 ? 'Toque em "Novo" para registrar vacinas, consultas, remédios, vermífugos ou observações.'
 : 'Nenhum item encontrado para este filtro.'
 }
 >
 </>
 }
 />
}
/>
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 loadingContainer: { flex: 1, alignItems: 'center', justifyContent: 'center' },
 errorText: { fontSize: 15, color: colors.textMuted },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: typography.h4.fontSize, fontWeight: typography.h4.fontWeight, color: colors.text },
 headerActions: { flexDirection: 'row', alignItems: 'center', gap: spacing.md },
 list: { paddingBottom: 48 },
 hero: {
 alignItems: 'center',
 paddingHorizontal: spacing.lg,
 paddingBottom: spacing.md,
 },
 photoWrapper: {
 width: 84,
 height: 84,
 borderRadius: 42,
 overflow: 'hidden',
 marginBottom: spacing.md,
 },
 photo: { width: 84, height: 84 },
});

```

```

photoPlaceholder: {
 width: 84,
 height: 84,
 borderRadius: 42,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 alignItems: 'center',
 justifyContent: 'center',
},
petName: {
 fontSize: 26,
 fontWeight: '700',
 color: colors.text,
 letterSpacing: -0.5,
 marginBottom: 4,
},
petMeta: { fontSize: 14, color: colors.textMuted, marginBottom: spacing.lg },
quickActions: {
 flexDirection: 'row',
 flexWrap: 'wrap',
 gap: spacing.sm,
 width: '100%',
 marginBottom: spacing.md,
},
quickAction: {
 flexBasis: '30%',
 flexGrow: 1,
 alignItems: 'center',
 gap: spacing.xs,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 paddingVertical: spacing.md,
},
quickActionIcon: {
 width: 36,
 height: 36,
 borderRadius: radius.full,
 alignItems: 'center',
 justifyContent: 'center',
},
quickActionLabel: { fontSize: 12, fontWeight: '500', color: colors.textSubtle },
divider: { width: '100%', height: 1, backgroundColor: colors.border, marginBottom: spacing.lg },
sectionRow: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 width: '100%',
 marginBottom: spacing.md,
},
sectionTitle: { fontSize: typography.h3.fontSize, fontWeight: typography.h3.fontWeight, color: colors.text },
addBtn: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: 4,
 backgroundColor: colors.primary,
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.xs + 2,
 borderRadius: radius.full,
},
addBtnText: { fontSize: 13, fontWeight: '600', color: colors.onPrimary },
filterScroll: { width: '100%', marginBottom: spacing.md },
filterRow: { gap: spacing.sm, paddingRight: spacing.lg },
filterChip: {
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.xs + 2,
 borderRadius: radius.full,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
},
filterChipActive: {
 backgroundColor: colors.primary + '22',
 borderColor: colors.primaryLight,
},
filterChipText: { fontSize: 13, fontWeight: '500', color: colors.textMuted },
filterChipTextActive: { color: colors.primaryLight, fontWeight: '600' },
monthHeader: {
 fontSize: typography.label.fontSize,

```

```
fontWeight: typography.label.fontWeight,
color: colors.textSubtle,
textTransform: 'uppercase',
letterSpacing: 0.6,
paddingHorizontal: spacing.lg,
marginTop: spacing.md,
marginBottom: spacing.sm,
},
));
```

## src/screens/AddRecordScreen.tsx

```
import React, { useState, useEffect } from 'react';
import {
 View,
 Text,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 KeyboardAvoidingView,
 Platform,
 Alert,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp } from '@react-navigation/native';
import { Input } from '../components/Input';
import { ThemeToggle } from '../components/ThemeToggle';
import { Button } from '../components/Button';
import { useToast } from '../hooks/useToast';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getRecords, saveRecord, deleteRecord } from '../storage';
import { maskDate, isValidDate, isFuture, toISO, displayDate } from '../utils/date';
import {
 RECORD_TYPE_LABELS,
 recordTypeColors,
 FREQUENCY_LABELS,
 REMINDER_OPTIONS,
} from '../labels';
import { MedicalRecord, RecordType, Frequency, RootStackParamList } from '../types';

type Route = RouteProp<RootStackParamList, 'AddRecord'>;

const RECORD_TYPES: RecordType[] = ['vaccine', 'consultation', 'medication', 'deworming', 'note'];
const FREQUENCIES: Frequency[] = ['once_daily', 'twice_daily', 'every_8h', 'every_12h', 'continuous'];

const TITLE_LABELS: Record<RecordType, string> = {
 vaccine: 'Nome da vacina',
 consultation: 'Motivo da consulta',
 medication: 'Nome do medicamento',
 deworming: 'Nome do vermifugo',
 note: 'Título',
};

const TITLE_PLACEHOLDERS: Record<RecordType, string> = {
 vaccine: 'Ex: V10',
 consultation: 'Ex: Consulta de rotina',
 medication: 'Ex: Antibiótico 250mg',
 deworming: 'Ex: Vermifugo oral',
 note: 'Ex: Mudança de raça',
};

const DATE_LABELS: Record<RecordType, string> = {
 vaccine: 'Data de aplicação',
 consultation: 'Data da consulta',
 medication: 'Data de início',
 deworming: 'Data de aplicação',
 note: 'Data',
};

const NEXT_DATE_LABELS: Partial<Record<RecordType, string>> = {
 vaccine: 'Data de reforço (opcional)',
 consultation: 'Retorno (opcional)',
 deworming: 'Próxima dose (opcional)',
};

function trimOrUndefined(value: string): string | undefined {
 const trimmed = value.trim();
 return trimmed ? trimmed : undefined;
}

export default function AddRecordScreen() {
 const navigation = useNavigation();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const RECORD_TYPE_COLORS = recordTypeColors(colors);
 const { showToast } = useToast();
 const route = useRoute<Route>();
 const { petId, recordId, initialType, prefill } = route.params;
```

```

const [original, setOriginal] = useState<MedicalRecord | null>(null);
const [type, setType] = useState<RecordType>(initialType ?? 'vaccine');
const [title, setTitle] = useState(prefill?.title ?? '');
const [date, setDate] = useState('');
const [nextDate, setNextDate] = useState('');
const [endDate, setEndDate] = useState('');
const [frequency, setFrequency] = useState<Frequency | undefined>();
const [dosage, setDosage] = useState('');
const [manufacturer, setManufacturer] = useState(prefill?.manufacturer ?? '');
const [batch, setBatch] = useState(prefill?.batch ?? '');
const [clinic, setClinic] = useState(prefill?.clinic ?? '');
const [vet, setVet] = useState('');
const [diagnosis, setDiagnosis] = useState('');
const [description, setDescription] = useState('');
const [reminderDays, setReminderDays] = useState<number[]>([]);
const [customReminder, setCustomReminder] = useState('');
const [saving, setSaving] = useState(false);

useEffect(() => {
 if (!recordId) return;
 getRecords(petId)
 .then(records => {
 const record = records.find(r => r.id === recordId);
 if (!record) return;
 setOriginal(record);
 setType(record.type);
 setTitle(record.title);
 setDate(displayDate(record.date));
 setNextDate(record.nextDate ? displayDate(record.nextDate) : '');
 setEndDate(record.endDate ? displayDate(record.endDate) : '');
 setFrequency(record.frequency);
 setDosage(record.dosage ?? '');
 setManufacturer(record.manufacturer ?? '');
 setBatch(record.batch ?? '');
 setClinic(record.clinic ?? '');
 setVet(record.vet ?? '');
 setDiagnosis(record.diagnosis ?? '');
 setDescription(record.description ?? '');
 const days = record.reminderDays ?? [];
 const presets = REMINDER_OPTIONS.map(o => o.days);
 setReminderDays(days.filter(d => presets.includes(d)));
 const custom = days.find(d => !presets.includes(d));
 setCustomReminder(custom !== undefined ? String(custom) : '');
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar o registro.'));
}, [petId, recordId]);

const futureDate = type === 'medication' ? endDate : nextDate;
const showReminders = type !== 'note' && futureDate.length === 10;

function handleTypeChange(t: RecordType) {
 setType(t);
 if (t === 'note') {
 setNextDate('');
 setEndDate('');
 setFrequency(undefined);
 setReminderDays([]);
 setCustomReminder('');
 } else if (t === 'medication') {
 setNextDate('');
 } else {
 setEndDate('');
 setFrequency(undefined);
 setDosage('');
 }
}

function toggleReminder(days: number) {
 setReminderDays(prev =>
 prev.includes(days) ? prev.filter(d => d !== days) : [...prev, days],
);
}

function validate(): string | null {
 if (!title.trim()) return `Informe o campo "${TITLE_LABELS[type]}".`;
 if (!date || !isValidDate(date)) return `Informe uma data válida em "${DATE_LABELS[type]}".`;
 if (isFuture(date)) return `"${DATE_LABELS[type]}" não pode ser no futuro.`;
 if (type !== 'medication' && type !== 'note' && nextDate) {
 if (!isValidDate(nextDate)) return 'Verifique a próxima data (DD/MM/AAAA).';
 if (toISO(nextDate) <= toISO(date)) return 'A próxima data deve ser depois da data do registro.';
 }
}

```

```

 }
 if (type === 'medication' && endDate) {
 if (!isValidDate(endDate)) return 'Verifique a data final (DD/MM/AAAA).';
 if (toISO(endDate) < toISO(date)) return 'A data final deve ser depois do início.';
 }
 if (customReminder && (!/^\\d+$/).test(customReminder) || Number(customReminder) > 365) {
 return 'O lembrete personalizado deve ser um número de dias (até 365).';
 }
 return null;
 }
}

async function handleSave() {
 const error = validate();
 if (error) {
 Alert.alert('Verifique os campos', error);
 return;
 }
 setSaving(true);
 try {
 const allReminders = [...reminderDays];
 if (customReminder) {
 const custom = Number(customReminder);
 if (!allReminders.includes(custom)) allReminders.push(custom);
 }
 const record: MedicalRecord = {
 id: original?.id ?? `${Date.now()}-${Math.random().toString(36).slice(2)}`,
 petId,
 type,
 date: toISO(date),
 title: title.trim(),
 description: trimOrUndefined(description),
 createdAt: original?.createdAt ?? new Date().toISOString(),
 };
 if (type !== 'medication' && type !== 'note' && nextDate) record.nextDate = toISO(nextDate);
 if (type === 'medication') {
 // Uso contínuo não tem data final, mesmo que o campo tenha sido
 // preenchido antes de trocar a frequência.
 if (endDate && frequency !== 'continuous') record.endDate = toISO(endDate);
 record.frequency = frequency;
 record.dosage = trimOrUndefined(dosage);
 }
 if (type === 'vaccine') {
 record.manufacturer = trimOrUndefined(manufacturer);
 record.batch = trimOrUndefined(batch);
 record.clinic = trimOrUndefined(clinic);
 }
 if (type === 'consultation') {
 record.vet = trimOrUndefined(vet);
 record.clinic = trimOrUndefined(clinic);
 record.diagnosis = trimOrUndefined(diagnosis);
 }
 if (showReminders && allReminders.length > 0) {
 record.reminderDays = allReminders.sort((a, b) => a - b);
 }
 await saveRecord(record);
 navigation.goBack();
 showToast(recordId ? 'Registro atualizado' : 'Registro salvo com sucesso');
 } catch {
 setSaving(false);
 Alert.alert('Erro', 'Não foi possível salvar o registro. Tente novamente.');
```

```

 }
}

function confirmDelete() {
 if (!original) return;
 Alert.alert('Excluir registro', 'Deseja remover este registro?', [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Excluir',
 style: 'destructive',
 onPress: async () => {
 try {
 await deleteRecord(original.id);
 navigation.goBack();
 showToast('Registro excluído');
 } catch {
 Alert.alert('Erro', 'Não foi possível excluir o registro.');
```

```

}

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity>
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Icons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>{recordId ? 'Editar Registro' : 'Novo Registro'}</Text>
 <ThemeToggle />
 </View>

 <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>
 <ScrollView contentContainerStyle={styles.scroll} keyboardShouldPersistTaps="handled">

 <View style={styles.field}>
 <Text style={styles.fieldLabel}>Tipo de registro</Text>
 <View style={styles.typeRow}>
 {RECORD_TYPES.map(t => (
 <TouchableOpacity>
 key={t}
 style={[
 styles.typeChip,
 type === t && {
 backgroundColor: RECORD_TYPE_COLORS[t] + '22',
 borderColor: RECORD_TYPE_COLORS[t],
 },
]}
 onPress={() => handleTypeChange(t)}
 activeOpacity={0.7}
 >
 <Text>
 style={[styles.typeChipText, type === t && { color: RECORD_TYPE_COLORS[t] }]}
 >
 {RECORD_TYPE_LABELS[t]}
 </Text>
 </TouchableOpacity>
))}
 </View>
 </View>

 <Input>
 label={TITLE_LABELS[type]}
 placeholder={TITLE_PLACEHOLDERS[type]}
 value={title}
 onChangeText={setTitle}
 autoCapitalize="sentences"
 returnKeyType="next"
 />

 <Input>
 label={DATE_LABELS[type]}
 placeholder="DD/MM/AAAA"
 value={date}
 onChangeText={t => setDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 returnKeyType="next"
 />

 {type === 'vaccine' && (
 <>
 <Input>
 label={NEXT_DATE_LABELS.vaccine!}
 placeholder="DD/MM/AAAA"
 value={nextDate}
 onChangeText={t => setNextDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />
 <Input>
 label="Fabricante (opcional)"
 placeholder="Ex: Zoetis"
 value={manufacturer}
 onChangeText={setManufacturer}
 />
 </>
)}
 </ScrollView>
 </KeyboardAvoidingView>
 </SafeAreaView>
)

```

```

 <Input
 label="Lote (opcional)"
 placeholder="Ex: L2026-04"
 value={batch}
 onChangeText={setBatch}
 />
 <Input
 label="Clínica (opcional)"
 placeholder="Ex: Clínica VetVida"
 value={clinic}
 onChangeText={setClinic}
 />
 </>
)}

 {type === 'consultation' && (
 <>
 <Input
 label="Veterinário (opcional)"
 placeholder="Ex: Dra. Ana Souza"
 value={vet}
 onChangeText={setVet}
 />
 <Input
 label="Clínica (opcional)"
 placeholder="Ex: Clínica VetVida"
 value={clinic}
 onChangeText={setClinic}
 />
 <Input
 label="Diagnóstico (opcional)"
 placeholder="Ex: Otite leve no ouvido direito"
 value={diagnosis}
 onChangeText={setDiagnosis}
 multiline
 />
 <Input
 label={NEXT_DATE_LABELS.consultation!}
 placeholder="DD/MM/AAAA"
 value={nextDate}
 onChangeText={t => setNextDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />
 </>
)}

 {type === 'medication' && (
 <>
 <Input
 label="Dosagem (opcional)"
 placeholder="Ex: 250mg, 1 comprimido"
 value={dosage}
 onChangeText={setDosage}
 />
 <View style={styles.field}>
 <Text style={styles.fieldLabel}>Frequência</Text>
 <View style={styles.typeRow}>
 {FREQUENCIES.map(f => (
 <TouchableOpacity
 key={f}
 style={[styles.freqChip, frequency === f && styles.freqChipActive]}
 onPress={() => {
 setFrequency(prev => {
 const next = prev === f ? undefined : f;
 if (next === 'continuous') setEndDate('');
 return next;
 });
 }}
 activeOpacity={0.7}
 >
 <Text
 style={[styles.typeChipText, frequency === f && { color: colors.primaryLight }}
 >
 {FREQUENCY_LABELS[f]}
 </Text>
 </TouchableOpacity>
)]}
 </View>
 </View>
 {frequency !== 'continuous' && (

```

```

 <Input
 label="Data final (opcional)"
 placeholder="DD/MM/AAAA"
 value={endDate}
 onChangeText={t => setEndDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />
)}
 </>
)}

 {type === 'deworming' && (
 <Input
 label={NEXT_DATE_LABELS.deworming!}
 placeholder="DD/MM/AAAA"
 value={nextDate}
 onChangeText={t => setNextDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />
)}

 <Input
 label={type === 'note' ? 'Descrição' : 'Observações (opcional)'}
 placeholder={
 type === 'note'
 ? 'Ex: Vomitou durante a madrugada, mas comeu normalmente pela manhã'
 : 'Anotações adicionais'
 }
 value={description}
 onChangeText={setDescription}
 multiline
 />

 {showReminders && (
 <View style={styles.field}>
 <Text style={styles.fieldLabel}>Lembrete</Text>
 <Text style={styles.fieldHint}>
 Escolha com quantos dias de antecedência você quer ser avisado.
 </Text>
 <View style={styles.typeRow}>
 {REMINDER_OPTIONS.map(opt => {
 const active = reminderDays.includes(opt.days);
 return (
 <TouchableOpacity
 key={opt.days}
 style={[styles.freqChip, active && styles.freqChipActive]}
 onPress={() => toggleReminder(opt.days)}
 activeOpacity={0.7}
 >
 <Text style={[styles.typeChipText, active && { color: colors.primaryLight }]}>
 {opt.label}
 </Text>
 </TouchableOpacity>
);
 })}
 </View>
 <Input
 label="Personalizado (dias antes, opcional)"
 placeholder="Ex: 10"
 value={customReminder}
 onChangeText={t => setCustomReminder(t.replace(/\D/g, '').slice(0, 3))}
 keyboardType="numeric"
 maxLength={3}
 />
 </View>
)}

 <Button
 label={recordId ? 'Salvar Alterações' : 'Salvar Registro'}
 onPress={handleSave}
 loading={saving}
 />

 {original && (
 <TouchableOpacity style={styles.deleteBtn} onPress={confirmDelete} activeOpacity={0.7}>
 <Ionicons name="trash-outline" size={16} color={colors.danger} />
 <Text style={styles.deleteBtnText}>Excluir registro</Text>
 </TouchableOpacity>
)}

```

```

 </ScrollView>
 </KeyboardAvoidingView>
 </SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 scroll: { padding: spacing.lg, gap: spacing.lg, paddingBottom: 48 },
 field: { gap: spacing.xs },
 fieldLabel: { fontSize: 13, fontWeight: '500', color: colors.textSubtle, letterSpacing: 0.3 },
 fieldHint: { fontSize: 12, color: colors.textMuted, marginBottom: spacing.xs },
 typeRow: { flexDirection: 'row', gap: spacing.sm, flexWrap: 'wrap' },
 typeChip: {
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.sm + 2,
 borderRadius: radius.md,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 },
 typeChipText: { fontSize: 13, fontWeight: '500', color: colors.textMuted },
 freqChip: {
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.sm,
 borderRadius: radius.full,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 },
 freqChipActive: {
 backgroundColor: colors.primary + '22',
 borderColor: colors.primaryLight,
 },
 deleteBtn: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'center',
 gap: spacing.xs,
 paddingVertical: spacing.sm,
 },
 deleteBtnText: { fontSize: 14, fontWeight: '500', color: colors.danger },
});

```

## src/screens/AddWeightScreen.tsx

```
import React, { useState, useEffect } from 'react';
import {
 View,
 Text,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 KeyboardAvoidingView,
 Platform,
 Alert,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp } from '@react-navigation/native';
import { Input } from '../components/Input';
import { ThemeToggle } from '../components/ThemeToggle';
import { Button } from '../components/Button';
import { useToast } from '../hooks/useToast';
import { spacing, useTheme, useThemedStyles, Palette } from '../theme';
import { getWeights, saveWeight, deleteWeight } from '../storage';
import { maskDate, isValidDate, isFuture, toISO, displayDate } from '../utils/date';
import { WeightEntry, RootStackParamList } from '../types';

type Route = RouteProp<RootStackParamList, 'AddWeight'>;

function maskWeight(text: string): string {
 return text
 .replace(/[^\d,\.]/g, '')
 .replace('.', ',')
 .replace(/,(?=[.]*)/g, '')
 .slice(0, 6);
}

function parseWeight(text: string): number | null {
 const value = Number(text.replace(',', '.'));
 if (!Number.isFinite(value) || value <= 0 || value > 200) return null;
 return value;
}

export default function AddWeightScreen() {
 const navigation = useNavigation();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { showToast } = useToast();
 const { petId, weightId } = useRoute<Route>().params;

 const [original, setOriginal] = useState<WeightEntry | null>(null);
 const [weight, setWeight] = useState('');
 const [date, setDate] = useState('');
 const [saving, setSaving] = useState(false);

 useEffect(() => {
 if (!weightId) return;
 getWeights(petId)
 .then(weights => {
 const entry = weights.find(w => w.id === weightId);
 if (!entry) return;
 setOriginal(entry);
 setWeight(entry.weightKg.toLocaleString('pt-BR'));
 setDate(displayDate(entry.date));
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar o registro de peso.'));
 }, [petId, weightId]);

 async function handleSave() {
 const weightKg = parseWeight(weight);
 if (!weightKg) {
 Alert.alert('Peso inválido', 'Informe o peso em kg (ex: 12,5).');
 return;
 }
 if (!date || !isValidDate(date)) {
 Alert.alert('Data inválida', 'Informe uma data válida (DD/MM/AAAA).');
 return;
 }
 if (isFuture(date)) {
 Alert.alert('Data inválida', 'A data da pesagem não pode ser no futuro.');
```

```

 }
 setSaving(true);
 try {
 await saveWeight({
 id: original?.id ?? `${Date.now()}-${Math.random().toString(36).slice(2)}`,
 petId,
 date: toISO(date),
 weightKg,
 createdAt: original?.createdAt ?? new Date().toISOString(),
 });
 navigation.goBack();
 showToast(weightId ? 'Pesagem atualizada' : 'Pesagem registrada');
 } catch {
 setSaving(false);
 Alert.alert('Erro', 'Não foi possível salvar o peso. Tente novamente.');
```

```

 }
 }

function confirmDelete() {
 if (!original) return;
 Alert.alert('Excluir pesagem', 'Deseja remover este registro de peso?', [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Excluir',
 style: 'destructive',
 onPress: async () => {
 try {
 await deleteWeight(original.id);
 navigation.goBack();
 showToast('Pesagem excluída');
 } catch {
 Alert.alert('Erro', 'Não foi possível excluir a pesagem.');
```

```

 }
 },
],
));
}

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Icons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>{weightId ? 'Editar Pesagem' : 'Nova Pesagem'}</Text>
 <ThemeToggle />
 </View>

 <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>
 <ScrollView contentContainerStyle={styles.scroll} keyboardShouldPersistTaps="handled">
 <Input
 label="Peso (kg)"
 placeholder="Ex: 12,5"
 value={weight}
 onChangeText={t => setWeight(maskWeight(t))}
 keyboardType="decimal-pad"
 maxLength={6}
 returnKeyType="next"
 />

 <Input
 label="Data da pesagem"
 placeholder="DD/MM/AAAA"
 value={date}
 onChangeText={t => setDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 returnKeyType="done"
 />

 <Button
 label={weightId ? 'Salvar Alterações' : 'Salvar Pesagem'}
 onPress={handleSave}
 loading={saving}
 />
 </ScrollView>
 </KeyboardAvoidingView>
 </SafeAreaView>
);

```

```

 {original && (
 <TouchableOpacity style={styles.deleteBtn} onPress={confirmDelete} activeOpacity={0.7}>
 <Ionicons name="trash-outline" size={16} color={colors.danger} />
 <Text style={styles.deleteBtnText}>Excluir pesagem</Text>
 </TouchableOpacity>
)}
 </ScrollView>
 </KeyboardAvoidingView>
 </SafeAreaView>
);
}

```

```

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 scroll: { padding: spacing.lg, gap: spacing.lg, paddingBottom: 48 },
 deleteBtn: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'center',
 gap: spacing.xs,
 paddingVertical: spacing.sm,
 },
 deleteBtnText: { fontSize: 14, fontWeight: '500', color: colors.danger },
});

```

## src/screens/WeightScreen.tsx

```
import React, { useState, useCallback } from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet, Alert } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getWeights } from '../storage';
import { displayDate } from '../utils/date';
import { WeightCard } from '../components/WeightCard';
import { WeightChart } from '../components/WeightChart';
import { EmptyState } from '../components/EmptyState';
import { ThemeToggle } from '../components/ThemeToggle';
import { WeightEntry, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Weight'>;
type Route = RouteProp<RootStackParamList, 'Weight'>;

export default function WeightScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { petId } = useRoute<Route>().params;
 const [weights, setWeights] = useState<WeightEntry[]>([]);

 useFocusEffect(
 useCallback(() => {
 getWeights(petId)
 .then(setWeights)
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar as pesagens.'));
 }, [petId]),
);

 function renderEntry({ item, index }: { item: WeightEntry; index: number }) {
 const previous = weights[index + 1];
 const diff = previous ? item.weightKg - previous.weightKg : null;
 return (
 <TouchableOpacity
 style={styles.row}
 onPress={() => navigation.navigate('AddWeight', { petId, weightId: item.id })}
 activeOpacity={0.7}
 >
 <View style={{ flex: 1 }}>
 <Text style={styles.rowWeight}>{item.weightKg.toLocaleString('pt-BR')} kg</Text>
 <Text style={styles.rowDate}>{displayDate(item.date)}</Text>
 </View>
 {diff !== null && (
 <Text
 style={[
 styles.rowDiff,
 { color: diff === 0 ? colors.textMuted : diff > 0 ? colors.warning : colors.success },
]}
 >
 {diff > 0 ? '+' : ''}
 {diff.toLocaleString('pt-BR', { maximumFractionDigits: 2 })} kg
 </Text>
)}
 <Ionicons name="chevron-forward" size={14} color={colors.textMuted} />
 </TouchableOpacity>
);
 }

 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>Controle de Peso</Text>
 <View style={styles.headerActions}>
 <ThemeToggle size={20} />
 <TouchableOpacity

```

```

 onPress={() => navigation.navigate('AddWeight', { petId })}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Nova pesagem"
 >
 <Icons name="add" size={26} color={colors.primaryLight} />
 </TouchableOpacity>
 </View>
 </View>

 <FlatList
 data={weights}
 keyExtractor={item => item.id}
 renderItem={renderEntry}
 contentContainerStyle={styles.list}
 showsVerticalScrollIndicator={false}
 ListHeaderComponent={
 weights.length > 0 ? (
 <View style={styles.summary}>
 <WeightCard weights={weights} />
 <WeightChart weights={weights} />
 <Text style={styles.sectionTitle}>Histórico completo</Text>
 </View>
) : null
 }
 ListEmptyComponent={
 <View style={styles.emptyWrapper}>
 <EmptyState
 image={require('../../assets/icons/heart.png')}
 title="Nenhuma pesagem registrada"
 text="Toque em + para registrar o primeiro peso e acompanhar a evolução."
 />
 </View>
 }
 />
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 headerActions: { flexDirection: 'row', alignItems: 'center', gap: spacing.md },
 list: { paddingHorizontal: spacing.lg, paddingBottom: 48 },
 summary: { gap: spacing.md, marginBottom: spacing.md },
 sectionTitle: { fontSize: 16, fontWeight: '600', color: colors.text, marginTop: spacing.sm },
 emptyWrapper: { paddingTop: spacing.xxl },
 row: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 marginBottom: spacing.sm,
 },
 rowWeight: { fontSize: 16, fontWeight: '600', color: colors.text },
 rowDate: { fontSize: 12, color: colors.textMuted },
 rowDiff: { fontSize: 13, fontWeight: '600' },
});

```

## src/screens/VaccinesScreen.tsx

```
import React, { useState, useCallback, useMemo } from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet, Alert } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getRecords } from '../storage';
import { displayDate, formatDaysUntil, daysUntilISO } from '../utils/date';
import { getVaccineStatus, getFulfilledRecordIds, VaccineStatus } from '../services/events';
import { EmptyState } from '../components/EmptyState';
import { ThemeToggle } from '../components/ThemeToggle';
import { MedicalRecord, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Vaccines'>;
type Route = RouteProp<RootStackParamList, 'Vaccines'>;

const STATUS_META: Record<VaccineStatus, { label: string }> = {
 ok: { label: 'Em dia' },
 due_soon: { label: 'Reforço próximo' },
 overdue: { label: 'Atrasada' },
 completed: { label: 'Reforço aplicado' },
};

export default function VaccinesScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { petId } = useRoute<Route>().params;
 const [vaccines, setVaccines] = useState<MedicalRecord[]>([]);
 const [allRecords, setAllRecords] = useState<MedicalRecord[]>([]);

 useFocusEffect(
 useCallback(() => {
 getRecords(petId)
 .then(records => {
 setAllRecords(records);
 setVaccines(records.filter(r => r.type === 'vaccine'));
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar as vacinas.'));
 }, [petId]),
);

 const statusColor: Record<VaccineStatus, string> = {
 ok: colors.success,
 due_soon: colors.warning,
 overdue: colors.danger,
 completed: colors.info,
 };

 const fulfilledIds = useMemo(() => getFulfilledRecordIds(allRecords), [allRecords]);
 const overdueCount = vaccines.filter(
 v => getVaccineStatus(v, allRecords, fulfilledIds) === 'overdue',
).length;
 const dueSoonCount = vaccines.filter(
 v => getVaccineStatus(v, allRecords, fulfilledIds) === 'due_soon',
).length;

 function renderVaccine({ item }: { item: MedicalRecord }) {
 const status = getVaccineStatus(item, allRecords, fulfilledIds) ?? 'ok';
 const details = [item.manufacturer, item.batch, item.clinic].filter(Boolean).join(' · ');
 const showCountdown =
 item.nextDate && (status === 'ok' || status === 'due_soon') && daysUntilISO(item.nextDate) >= 0;
 return (
 <TouchableOpacity
 style={styles.card}
 onPress={() => navigation.navigate('AddRecord', { petId, recordId: item.id })}
 activeOpacity={0.8}
 >
 <View style={[styles.statusStripe, { backgroundColor: statusColor[status] }]} />
 <View style={styles.cardBody}>
 <View style={styles.cardTop}>
 <Text style={styles.title}>{item.title}</Text>
 <View style={[styles.badge, { backgroundColor: statusColor[status] + '22' }]}>
 <Text style={styles.badgeText, { color: statusColor[status] }}>
 {STATUS_META[status].label}
 </Text>
 </View>
 </View>
 </View>
 </TouchableOpacity>
);
 }
}
```

```

 </View>
 </View>
 <Text style={styles.line}>Aplicada em {displayDate(item.date)}</Text>
 {item.nextDate ? (
 <Text style={styles.line}>
 Reforço: {displayDate(item.nextDate)}
 {showCountdown ? ` (${formatDaysUntil(daysUntilISO(item.nextDate)))}` : ''}
 </Text>
) : null}
 {details ? <Text style={styles.details}>{details}</Text> : null}
 {(status === 'due_soon' || status === 'overdue') && (
 <TouchableOpacity
 style={styles.doseBtn}
 onPress={() =>
 navigation.navigate('AddRecord', {
 petId,
 initialType: 'vaccine',
 prefill: {
 title: item.title,
 manufacturer: item.manufacturer,
 batch: item.batch,
 clinic: item.clinic,
 },
 })
 }
 activeOpacity={0.7}
 >
 <Ionicons name="add-circle-outline" size={14} color={colors.primaryLight} />
 <Text style={styles.doseBtnText}>Registrar dose aplicada</Text>
 </TouchableOpacity>
)}
 </View>
 </TouchableOpacity>
);
}

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>Vacinas</Text>
 <View style={styles.headerActions}>
 <ThemeToggle size={20} />
 <TouchableOpacity
 onPress={() => navigation.navigate('AddRecord', { petId, initialType: 'vaccine' })}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Nova vacina"
 >
 <Ionicons name="add" size={26} color={colors.primaryLight} />
 </TouchableOpacity>
 </View>
 </View>
 </View>

 <FlatList
 data={vaccines}
 keyExtractor={item => item.id}
 renderItem={renderVaccine}
 contentContainerStyle={styles.list}
 showsVerticalScrollIndicator={false}
 ListHeaderComponent={
 vaccines.length > 0 ? (
 <View style={styles.summaryRow}>
 <View style={styles.summaryCard}>
 <Text style={styles.summaryValue, { color: colors.success }}>
 {vaccines.length}
 </Text>
 <Text style={styles.summaryLabel}>aplicadas</Text>
 </View>
 <View style={styles.summaryCard}>
 <Text style={styles.summaryValue, { color: colors.warning }}>{dueSoonCount}</Text>
 <Text style={styles.summaryLabel}>reforço próximo</Text>
 </View>
 <View style={styles.summaryCard}>

```

```

 <Text style={[[styles.summaryValue, { color: colors.danger }]]}>{overdueCount}</Text>
 <Text style={styles.summaryLabel}>atrasadas</Text>
 </View>
 </View>
) : null
}
ListEmptyComponent={
 <View style={styles.emptyWrapper}>
 <EmptyState
 image={require('../assets/icons/shield.png')}
 title="Nenhuma vacina registrada"
 text="Toque em + para registrar a primeira vacina com data de reforço, fabricante e lote."
 />
 </View>
}
/>
</SafeAreaView>
);
}

```

```

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 headerActions: { flexDirection: 'row', alignItems: 'center', gap: spacing.md },
 list: { paddingHorizontal: spacing.lg, paddingBottom: 48 },
 summaryRow: { flexDirection: 'row', gap: spacing.sm, marginBottom: spacing.md },
 summaryCard: {
 flex: 1,
 alignItems: 'center',
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 paddingVertical: spacing.sm + 4,
 gap: 2,
 },
 summaryValue: { fontSize: 20, fontWeight: '700' },
 summaryLabel: { fontSize: 11, color: colors.textMuted },
 card: {
 flexDirection: 'row',
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 marginBottom: spacing.sm,
 overflow: 'hidden',
 },
 statusStripe: { width: 4 },
 cardBody: { flex: 1, padding: spacing.md, gap: 3 },
 cardTop: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 gap: spacing.sm,
 },
 title: { flex: 1, fontSize: 15, fontWeight: '600', color: colors.text },
 badge: {
 paddingHorizontal: spacing.sm,
 paddingVertical: 3,
 borderRadius: radius.full,
 },
 badgeText: { fontSize: 11, fontWeight: '600' },
 line: { fontSize: 13, color: colors.textSubtle },
 details: { fontSize: 12, color: colors.textMuted },
 doseBtn: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: 4,
 marginTop: spacing.xs,
 alignSelf: 'flex-start',
 },
 doseBtnText: { fontSize: 12, fontWeight: '600', color: colors.primaryLight },

```

```
emptyWrapper: { paddingTop: spacing.xxl },
});
```

## src/screens/MedicationsScreen.tsx

```
import React, { useState, useCallback, useMemo } from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet, Alert } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, typography, useTheme, useThemedStyles, Palette } from '../theme';
import { recordsRepository } from '../repositories/recordsRepository';
import { medicationsRepository } from '../repositories/medicationsRepository';
import { displayDate, daysUntilISO, formatDaysUntil } from '../utils/date';
import { isActiveMedication } from '../services/events';
import { FREQUENCY_LABELS } from '../labels';
import { EmptyState } from '../components/EmptyState';
import { ThemeToggle } from '../components/ThemeToggle';
import { Button } from '../components/Button';
import { useToast } from '../hooks/useToast';
import { MedicalRecord, MedicationDose, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Medications'>;
type Route = RouteProp<RootStackParamList, 'Medications'>;

/** "há 3 horas" / "há 2 dias" a partir de um ISO de data/hora. */
function formatTimeAgo(iso: string): string {
 const diffMs = Date.now() - new Date(iso).getTime();
 const hours = Math.floor(diffMs / 3600000);
 if (hours < 1) return 'há poucos minutos';
 if (hours < 24) return `há ${hours} ${hours === 1 ? 'hora' : 'horas'}`;
 const days = Math.floor(hours / 24);
 return `há ${days} ${days === 1 ? 'dia' : 'dias'}`;
}

/** Progresso do tratamento (0-100), ou null se não há data de início/fim válida. */
function treatmentProgress(record: MedicalRecord): number | null {
 if (!record.endDate) return null;
 const start = new Date(record.date).getTime();
 const end = new Date(record.endDate).getTime();
 if (end <= start) return null;
 const pct = ((Date.now() - start) / (end - start)) * 100;
 return Math.min(100, Math.max(0, pct));
}

export default function MedicationsScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { showToast } = useToast();
 const { petId } = useRoute<Route>().params;
 const [medications, setMedications] = useState<MedicalRecord[]>([]);
 const [doses, setDoses] = useState<MedicationDose[]>([]);

 useFocusEffect(
 useCallback(() => {
 Promise.all([recordsRepository.getByPet(petId), medicationsRepository.getDoses(petId)])
 .then(([records, loadedDoses]) => {
 setMedications(records.filter(r => r.type === 'medication'));
 setDoses(loadedDoses);
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar os remédios.'));
 }, [petId]),
);

 const dosesByRecord = useMemo(() => {
 const map = new Map<string, MedicationDose[]>();
 for (const dose of doses) {
 const list = map.get(dose.recordId) ?? [];
 list.push(dose);
 map.set(dose.recordId, list);
 }
 for (const list of map.values()) list.sort((a, b) => b.createdAt.localeCompare(a.createdAt));
 return map;
 }, [doses]);

 const handleMarkDose = useCallback(async (recordId: string) => {
 const now = new Date();
 const dose: MedicationDose = {
 id: `${Date.now()}-${Math.random().toString(36).slice(2)}`,
 recordId,
 };
 });
```

```

 petId,
 date: now.toISOString().slice(0, 10),
 createdAt: now.toISOString(),
 });
 try {
 await medicationsRepository.saveDose(dose);
 setDoses(prev => [dose, ...prev]);
 showToast('Dose registrada');
 } catch {
 Alert.alert('Erro', 'Não foi possível registrar a dose.');
```

```

 })
 </View>
</View>
);
}, [colors, styles, dosesByRecord, navigation, petId, handleMarkDose]);

const renderSection = useCallback(({ item }: { item: { title: string; data: MedicalRecord[] } }) => (
 <View style={styles.section}>
 <Text style={styles.sectionTitle}>{item.title}</Text>
 {item.data.map(med => renderMedication(med, item.title === 'Em uso'))}
 </View>
), [styles, renderMedication]);

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>Remédios</Text>
 <View style={styles.headerActions}>
 <ThemeToggle size={20} />
 <TouchableOpacity
 onPress={() => navigation.navigate('AddRecord', { petId, initialType: 'medication' })}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Novo remédio"
 >
 <Ionicons name="add" size={26} color={colors.primaryLight} />
 </TouchableOpacity>
 </View>
 </View>

 <FlatList
 data={sections}
 keyExtractor={item => item.title}
 showsVerticalScrollIndicator={false}
 contentContainerStyle={styles.list}
 renderItem={renderSection}
 ListEmptyComponent={
 <View style={styles.emptyWrapper}>
 <EmptyState
 icon="medkit"
 title="Nenhum remédio registrado"
 text="Toque em + para registrar um remédio, com frequência e data de término."
 />
 </View>
 }
 />
 </SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: typography.h4.fontSize, fontWeight: typography.h4.fontWeight, color: colors.text },
 headerActions: { flexDirection: 'row', alignItems: 'center', gap: spacing.md },
 list: { paddingHorizontal: spacing.lg, paddingBottom: 48 },
 section: { marginBottom: spacing.md },
 sectionTitle: {
 fontSize: 13,
 fontWeight: '600',
 color: colors.textMuted,
 marginBottom: spacing.sm,
 textTransform: 'uppercase',
 },
 card: {
 flexDirection: 'row',
 backgroundColor: colors.surface,
 }
});

```

```

borderRadius: radius.lg,
borderWidth: 1,
borderColor: colors.border,
marginBottom: spacing.sm,
overflow: 'hidden',
},
statusStripe: { width: 4 },
cardBody: { flex: 1, padding: spacing.md, gap: spacing.sm },
titleRow: { flexDirection: 'row', alignItems: 'center', justifyContent: 'space-between', gap: spacing.sm },
title: { flex: 1, fontSize: 15, fontWeight: '600', color: colors.text },
badge: { paddingHorizontal: spacing.sm, paddingVertical: 2, borderRadius: radius.full },
badgeText: { fontSize: 11, fontWeight: '600' },
line: { fontSize: 13, color: colors.textSubtle },
details: { fontSize: 12, color: colors.textMuted },
progressWrap: { gap: 4 },
progressTrack: {
 height: 6,
 borderRadius: radius.full,
 backgroundColor: colors.border,
 overflow: 'hidden',
},
progressFill: { height: '100%', borderRadius: radius.full },
progressLabel: { fontSize: 12, color: colors.textSubtle },
alertRow: { flexDirection: 'row', alignItems: 'center', gap: spacing.xs },
alertText: { fontSize: 12, fontWeight: '600', color: colors.warning },
lastDose: { fontSize: 12, color: colors.textMuted },
emptyWrapper: { paddingTop: spacing.xxl },
});

```

## src/screens/MedicalProfileScreen.tsx

```
import React, { useState, useEffect } from 'react';
import {
 View,
 Text,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 KeyboardAvoidingView,
 Platform,
 Alert,
 Switch,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp } from '@react-navigation/native';
import { Input } from '../components/Input';
import { ThemeToggle } from '../components/ThemeToggle';
import { Button } from '../components/Button';
import { useToast } from '../hooks/useToast';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getPets, savePet } from '../storage';
import { Pet, RootStackParamList } from '../types';

type Route = RouteProp<RootStackParamList, 'MedicalProfile'>;

function trimOrUndefined(value: string): string | undefined {
 const trimmed = value.trim();
 return trimmed ? trimmed : undefined;
}

export default function MedicalProfileScreen() {
 const navigation = useNavigation();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { showToast } = useToast();
 const { petId } = useRoute<Route>().params;

 const [pet, setPet] = useState<Pet | null>(null);
 const [neutered, setNeutered] = useState(false);
 const [allergies, setAllergies] = useState('');
 const [chronicConditions, setChronicConditions] = useState('');
 const [bloodType, setBloodType] = useState('');
 const [vetName, setVetName] = useState('');
 const [vetPhone, setVetPhone] = useState('');
 const [notes, setNotes] = useState('');
 const [saving, setSaving] = useState(false);

 useEffect(() => {
 getPets()
 .then(pets => {
 const found = pets.find(p => p.id === petId);
 if (!found) return;
 setPet(found);
 const profile = found.medicalProfile;
 if (!profile) return;
 setNeutered(profile.neutered ?? false);
 setAllergies(profile.allergies ?? '');
 setChronicConditions(profile.chronicConditions ?? '');
 setBloodType(profile.bloodType ?? '');
 setVetName(profile.vetName ?? '');
 setVetPhone(profile.vetPhone ?? '');
 setNotes(profile.notes ?? '');
 })
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar o perfil médico.'));
 }, [petId]);

 async function handleSave() {
 if (!pet) return;
 setSaving(true);
 try {
 await savePet({
 ...pet,
 medicalProfile: {
 neutered,
 allergies: trimOrUndefined(allergies),
 chronicConditions: trimOrUndefined(chronicConditions),
 bloodType: trimOrUndefined(bloodType),
 },
 });
 } catch {
 showToast('Erro ao salvar perfil médico');
 }
 }
}
```

```

 vetName: trimOrNull(vetName),
 vetPhone: trimOrNull(vetPhone),
 notes: trimOrNull(notes),
 },
});
navigation.goBack();
showToast('Perfil médico salvo');
} catch {
 setSaving(false);
 Alert.alert('Erro', 'Não foi possível salvar o perfil médico.');
}
}

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity>
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 </TouchableOpacity>
 <Text style={styles.headerTitle}>Perfil Médico</Text>
 <ThemeToggle />
 </View>

 <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>
 <ScrollView contentContainerStyle={styles.scroll} keyboardShouldPersistTaps="handled">
 <View style={styles.switchRow}>
 <View>
 <Text style={styles.switchLabel}>Castrado</Text>
 <Text style={styles.switchHint}>{neutered ? 'Sim' : 'Não'}</Text>
 </View>
 <Switch>
 value={neutered}
 onValueChange={setNeutered}
 trackColor={{ false: colors.surfaceElevated, true: colors.primary }}
 thumbColor={colors.text}
 </Switch>
 </View>

 <Input>
 label="Alergias"
 placeholder="Ex: Dipirona, frango"
 value={allergies}
 onChangeText={setAllergies}
 multiline
 </Input>
 <Input>
 label="Doenças crônicas"
 placeholder="Ex: Dermatite atópica"
 value={chronicConditions}
 onChangeText={setChronicConditions}
 multiline
 </Input>
 <Input>
 label="Tipo sanguíneo"
 placeholder="Ex: DEA 1.1 positivo"
 value={bloodType}
 onChangeText={setBloodType}
 </Input>
 <Input>
 label="Veterinário principal"
 placeholder="Ex: Dra. Ana Souza"
 value={vetName}
 onChangeText={setVetName}
 </Input>
 <Input>
 label="Telefone do veterinário"
 placeholder="Ex: (11) 99999-0000"
 value={vetPhone}
 onChangeText={setVetPhone}
 keyboardType="phone-pad"
 </Input>
 <Input>
 label="Observações médicas"
 placeholder="Informações importantes sobre a saúde do pet"
 value={notes}
 onChangeText={setNotes}
 </Input>
 </ScrollView>
 </KeyboardAvoidingView>
 </SafeAreaView>
);

```

```

 multiline
 />

 <Button label="Salvar Perfil" onPress={handleSave} loading={saving} disabled={!pet} />
 </ScrollView>
 </KeyboardAvoidingView>
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 scroll: { padding: spacing.lg, gap: spacing.lg, paddingBottom: 48 },
 switchRow: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.sm + 4,
 },
 switchLabel: { fontSize: 15, fontWeight: '500', color: colors.text },
 switchHint: { fontSize: 12, color: colors.textMuted },
});

```

## src/screens/EmergencyScreen.tsx

```
import React, { useState, useCallback } from 'react';
import {
 View,
 Text,
 ScrollView,
 TouchableOpacity,
 StyleSheet,
 Alert,
 Linking,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { Input } from '../components/Input';
import { ThemeToggle } from '../components/ThemeToggle';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { useToast } from '../hooks/useToast';
import { getPets, getRecords, getTutorInfo, saveTutorInfo } from '../storage';
import { isActiveMedication } from '../services/events';
import { FREQUENCY_LABELS } from '../labels';
import { Pet, MedicalRecord, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Emergency'>;
type Route = RouteProp<RootStackParamList, 'Emergency'>;

function InfoBlock({ label, value }: { label: string; value?: string }) {
 const styles = useThemedStyles(createStyles);
 return (
 <View style={styles.infoBlock}>
 <Text style={styles.infoLabel}>{label}</Text>
 <Text style={styles.infoValue, !value && styles.infoEmpty}>{value || 'Não informado'}</Text>
 </View>
);
}

export default function EmergencyScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { showToast } = useToast();
 const { petId } = useRoute<Route>().params;

 const [pet, setPet] = useState<Pet | null>(null);
 const [activeMeds, setActiveMeds] = useState<MedicalRecord[]>([]);
 const [tutorName, setTutorName] = useState('');
 const [tutorPhone, setTutorPhone] = useState('');
 const [editingTutor, setEditingTutor] = useState(false);

 useFocusEffect(
 useCallback(() => {
 Promise.all([
 getPets().then(pets => setPet(pets.find(p => p.id === petId) ?? null)),
 getRecords(petId).then(records => setActiveMeds(records.filter(isActiveMedication))),
 getTutorInfo().then(info => {
 setTutorName(info.name);
 setTutorPhone(info.phone);
 }),
]).catch(() => Alert.alert('Erro', 'Não foi possível carregar os dados de emergência.'));
 }, [petId]),
);

 async function handleSaveTutor() {
 if (tutorPhone.trim() && !/\d/.test(tutorPhone)) {
 Alert.alert('Telefone inválido', 'Informe um número de telefone válido.');
 return;
 }
 try {
 await saveTutorInfo({ name: tutorName.trim(), phone: tutorPhone.trim() });
 setEditingTutor(false);
 showToast('Dados do tutor salvos');
 } catch {
 Alert.alert('Erro', 'Não foi possível salvar os dados do tutor.');
 }
 }

 function handleCancelTutor() {

```

```

getTutorInfo().then(info => {
 setTutorName(info.name);
 setTutorPhone(info.phone);
});
setEditingTutor(false);
}

function call(phone?: string) {
 if (!phone) return;
 Linking.openURL(`tel:${phone.replace(/\D/g, '')}`).catch(() => {
 Alert.alert('Erro', 'Não foi possível iniciar a ligação.');
```

```

 >
 <Ionicons name="call" size={16} color="#fff" />
 </TouchableOpacity>
) : null}
 </View>
</>
)}
</View>

{/* Veterinário */}
<View style={styles.card}>
 <View style={styles.cardHeader}>
 <Text style={styles.cardTitle}>Veterinário</Text>
 <TouchableOpacity
 onPress={() => navigation.navigate('MedicalProfile', { petId })}
 hitSlop={{ top: 8, bottom: 8, left: 8, right: 8 }}
 >
 <Text style={styles.cardAction}>Editar</Text>
 </TouchableOpacity>
 </View>
 <InfoBlock label="Nome" value={profile?.vetName} />
 <View style={styles.phoneRow}>
 <InfoBlock label="Telefone" value={profile?.vetPhone} />
 {profile?.vetPhone ? (
 <TouchableOpacity
 style={styles.callBtn}
 onPress={() => call(profile.vetPhone)}
 accessibilityRole="button"
 accessibilityLabel="Ligar para o veterinário"
 >
 <Ionicons name="call" size={16} color="#fff" />
 </TouchableOpacity>
) : null}
 </View>
</View>

{/* Saúde */}
<View style={styles.card}>
 <Text style={styles.cardTitle}>Saúde</Text>
 <InfoBlock label="Alergias" value={profile?.allergies} />
 <InfoBlock label="Doenças crônicas" value={profile?.chronicConditions} />
 <InfoBlock label="Tipo sanguíneo" value={profile?.bloodType} />
 <InfoBlock label="Observações importantes" value={profile?.notes} />
</View>

{/* Medicamentos em uso */}
<View style={styles.card}>
 <Text style={styles.cardTitle}>Medicamentos em uso</Text>
 {activeMeds.length === 0 ? (
 <Text style={styles.infoEmpty}>Nenhum medicamento ativo.</Text>
) : (
 activeMeds.map((med => (
 <View key={med.id} style={styles.medRow}>
 <Ionicons name="medkit" size={14} color={colors.warning} />
 <Text style={styles.medText}>
 {med.title}
 {med.dosage ? ` · ${med.dosage}` : ''}
 {med.frequency ? ` · ${FREQUENCY_LABELS[med.frequency]}` : ''}
 </Text>
 </View>
))
)}
</View>
</ScrollView>
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 scroll: { padding: spacing.lg, gap: spacing.md, paddingBottom: 48 },
 banner: {
 flexDirection: 'row',

```

```

 alignItems: 'center',
 gap: spacing.sm,
 backgroundColor: colors.warning + '15',
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.warning + '44',
 padding: spacing.md,
 },
 bannerText: { flex: 1, fontSize: 13, color: colors.textSubtle, lineHeight: 19 },
 card: {
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 gap: spacing.sm,
 },
 cardHeader: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 },
 cardTitle: { fontSize: 15, fontWeight: '600', color: colors.text },
 cardHeaderActions: { flexDirection: 'row', alignItems: 'center', gap: spacing.lg },
 cardAction: { fontSize: 13, fontWeight: '600', color: colors.primaryLight },
 cardActionMuted: { fontSize: 13, fontWeight: '600', color: colors.textMuted },
 infoBlock: { gap: 2, flex: 1 },
 infoLabel: { fontSize: 11, color: colors.textMuted, letterSpacing: 0.4 },
 infoValue: { fontSize: 14, color: colors.text, lineHeight: 20 },
 infoEmpty: { fontSize: 14, color: colors.textMuted, fontStyle: 'italic' },
 phoneRow: { flexDirection: 'row', alignItems: 'center', gap: spacing.md },
 callBtn: {
 width: 36,
 height: 36,
 borderRadius: radius.full,
 backgroundColor: colors.success,
 alignItems: 'center',
 justifyContent: 'center',
 },
 medRow: { flexDirection: 'row', alignItems: 'center', gap: spacing.sm },
 medText: { fontSize: 14, color: colors.textSubtle, flex: 1 },
});

```

## src/screens/DocumentsScreen.tsx

```
import React, { useState, useCallback } from 'react';
import {
 View,
 Text,
 FlatList,
 TouchableOpacity,
 StyleSheet,
 Alert,
 Platform,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import * as DocumentPicker from 'expo-document-picker';
import * as ImagePicker from 'expo-image-picker';
import * as Sharing from 'expo-sharing';
import * as IntentLauncher from 'expo-intent-launcher';
import * as FileSystem from 'expo-file-system/legacy';
import { Input } from '../components/Input';
import { ThemeToggle } from '../components/ThemeToggle';
import { EmptyState } from '../components/EmptyState';
import { Button } from '../components/Button';
import { useToast } from '../hooks/useToast';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getDocuments, saveDocument, deleteDocument } from '../storage';
import { persistDocumentFile } from '../storage/files';
import { maskDate, isValidDate, toISO, displayDate } from '../utils/date';
import { DOCUMENT_KIND_LABELS } from '../labels';
import { PetDocument, DocumentKind, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Documents'>;
type Route = RouteProp<RootStackParamList, 'Documents'>;

const KINDS: DocumentKind[] = ['vaccination_card', 'exam', 'prescription', 'other'];

const KIND_ICONS: Record<DocumentKind, keyof typeof Ionicons.glyphMap> = {
 exam: 'flask',
 prescription: 'receipt',
 vaccination_card: 'card',
 other: 'document',
};

function kindColors(colors: Palette): Record<DocumentKind, string> {
 return {
 vaccination_card: colors.success,
 exam: colors.info,
 prescription: colors.warning,
 other: colors.textSubtle,
 };
}

const KIND_HINTS: Record<DocumentKind, string> = {
 vaccination_card: 'Carteira de vacinação do pet',
 exam: 'Hemogramas, raio-x, ultrassom...',
 prescription: 'Receitas e prescrições do veterinário',
 other: 'Qualquer outro documento',
};

interface PendingFile {
 uri: string;
 name: string;
 mimeType?: string;
}

export default function DocumentsScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const KIND_COLORS = kindColors(colors);
 const { showToast } = useToast();
 const { petId, kind } = useRoute<Route>().params;

 const [documents, setDocuments] = useState<PetDocument[]>([]);
 const [pending, setPending] = useState<PendingFile | null>(null);
 const [editingDoc, setEditingDoc] = useState<PetDocument | null>(null);
 const [title, setTitle] = useState('');
```

```

const [date, setDate] = useState('');
const [saving, setSaving] = useState(false);

const load = useCallback(() => {
 getDocuments(petId)
 .then(setDocuments)
 .catch(() => Alert.alert('Erro', 'Não foi possível carregar os documentos.'));
}, [petId]);

useFocusEffect(load);

async function pickFromCamera() {
 const perm = await ImagePicker.requestCameraPermissionsAsync();
 if (!perm.granted) {
 Alert.alert('Permissão necessária', 'Precisamos de acesso à câmera para tirar a foto.');
```

return;

```
 }
 const result = await ImagePicker.launchCameraAsync({ mediaTypes: ['images'], quality: 0.8 });
 if (result.canceled || !result.assets[0]) return;
 const asset = result.assets[0];
 const name = `foto-${Date.now()}.jpg`;
 setPending({ uri: asset.uri, name, mimeType: asset.mimeType ?? 'image/jpeg' });
 setTitle(prev => prev || (kind ? DOCUMENT_KIND_LABELS[kind] : ''));
}

async function pickFromGallery() {
 const perm = await ImagePicker.requestMediaLibraryPermissionsAsync();
 if (!perm.granted) {
 Alert.alert('Permissão necessária', 'Precisamos de acesso à galeria para selecionar a foto.');
```

return;

```
 }
 const result = await ImagePicker.launchImageLibraryAsync({ mediaTypes: ['images'], quality: 0.8 });
 if (result.canceled || !result.assets[0]) return;
 const asset = result.assets[0];
 const name = asset.fileName ?? `foto-${Date.now()}.jpg`;
 setPending({ uri: asset.uri, name, mimeType: asset.mimeType ?? 'image/jpeg' });
 setTitle(prev => prev || name.replace(/\\.\\w+$/, ''));
}

async function pickFromFiles() {
 const result = await DocumentPicker.getDocumentAsync({
 type: ['application/pdf', 'image/*'],
 copyToCacheDirectory: true,
 });
 if (result.canceled || !result.assets[0]) return;
 const asset = result.assets[0];
 setPending({ uri: asset.uri, name: asset.name, mimeType: asset.mimeType });
 setTitle(asset.name.replace(/\\.\\w+$/, ''));
}

function pickFile() {
 Alert.alert('Adicionar documento', 'De onde vem o arquivo?', [
 { text: 'Tirar foto', onPress: pickFromCamera },
 { text: 'Galeria', onPress: pickFromGallery },
 { text: 'Arquivo (PDF/imagem)', onPress: pickFromFiles },
 { text: 'Cancelar', style: 'cancel' },
]);
}

async function handleSave() {
 if (!pending && !editingDoc) || !kind) return;
 if (!title.trim()) {
 Alert.alert('Campo obrigatório', 'Informe o título do documento.');
```

return;

```
 }
 if (!date || !isValidDate(date)) {
 Alert.alert('Data inválida', 'Informe uma data válida (DD/MM/AAAA).');
```

return;

```
 }
 setSaving(true);
 try {
 if (editingDoc) {
 await saveDocument({
 ...editingDoc,
 title: title.trim(),
 date: toISO(date),
 });
 } else if (pending) {
 const id = `${Date.now()}-${Math.random().toString(36).slice(2)}`;
 const uri = await persistDocumentFile(pending.uri, id);
 await saveDocument({
```

```

 id,
 petId,
 title: title.trim(),
 kind,
 date: toISO(date),
 uri,
 mimeType: pending.mimeType,
 createdAt: new Date().toISOString(),
 });
}
const wasEditing = !!editingDoc;
setPending(null);
setEditingDoc(null);
setTitle('');
setDate('');
load();
showToast(wasEditing ? 'Documento atualizado' : 'Documento salvo');
} catch {
 Alert.alert('Erro', 'Não foi possível salvar o documento.');
} finally {
 setSaving(false);
}
}

function startEditing(doc: PetDocument) {
 setEditingDoc(doc);
 setTitle(doc.title);
 setDate(displayDate(doc.date));
}

function cancelForm() {
 setPending(null);
 setEditingDoc(null);
 setTitle('');
 setDate('');
}

function isImage(doc: PetDocument): boolean {
 if (doc.mimeType?.startsWith('image/')) return true;
 return /\.?(jpe?g|png|gif|webp|heic)$/i.test(doc.uri);
}

// Visualiza sem abrir a folha de compartilhamento: imagens dentro do app,
// PDFs no leitor padrão do sistema.
async function openDocument(doc: PetDocument) {
 if (isImage(doc)) {
 navigation.navigate('DocumentViewer', {
 uri: doc.uri,
 title: doc.title,
 mimeType: doc.mimeType,
 });
 return;
 }
 if (Platform.OS === 'web') {
 Alert.alert('Indisponível', 'Abra os documentos pelo aplicativo no celular.');
 return;
 }
 try {
 if (Platform.OS === 'android') {
 const contentUri = await FileSystem.getContentUriAsync(doc.uri);
 await IntentLauncher.startActivityAsync('android.intent.action.VIEW', {
 data: contentUri,
 type: doc.mimeType ?? 'application/pdf',
 flags: 1, // FLAG_GRANT_READ_URI_PERMISSION
 });
 } else {
 await Sharing.shareAsync(doc.uri, { mimeType: doc.mimeType });
 }
 } catch {
 Alert.alert(
 'Não foi possível abrir',
 'Nenhum aplicativo de PDF encontrado. Instale um leitor de PDF ou use o toque longo para compartilhar.',
);
 }
}

async function shareDocument(doc: PetDocument) {
 try {
 await Sharing.shareAsync(doc.uri, { mimeType: doc.mimeType });
 } catch {
 Alert.alert('Erro', 'Não foi possível compartilhar o documento.');
 }
}

```

```

 }
 }

function showOptions(doc: PetDocument) {
 Alert.alert(doc.title, 'O que deseja fazer?', [
 { text: 'Cancelar', style: 'cancel' },
 { text: 'Editar', onPress: () => startEditing(doc) },
 { text: 'Compartilhar', onPress: () => shareDocument(doc) },
 {
 text: 'Excluir',
 style: 'destructive',
 onPress: () => {
 Alert.alert('Excluir documento', `Remover "${doc.title}"?`, [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Excluir',
 style: 'destructive',
 onPress: async () => {
 try {
 await deleteDocument(doc.id);
 load();
 showToast('Documento excluído');
 } catch {
 Alert.alert('Erro', 'Não foi possível excluir o documento.');
```

```
// ----- Modo 2: documentos da categoria -----

const kindDocs = documents.filter(d => d.kind === kind);

return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Icons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>{DOCUMENT_KIND_LABELS[kind]}</Text>
 <TouchableOpacity
 onPress={pickFile}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Adicionar documento"
 >
 <Icons name="add" size={26} color={colors.primaryLight} />
 </TouchableOpacity>
 </View>

 {pending || editingDoc ? (
 <View style={styles.form}>
 {pending && (
 <View style={styles.fileRow}>
 <Icons name="attach" size={16} color={colors.primaryLight} />
 <Text style={styles.fileName} numberOfLines={1}>
 {pending.name}
 </Text>
 <TouchableOpacity onPress={cancelForm}>
 <Icons name="close-circle" size={18} color={colors.textMuted} />
 </TouchableOpacity>
 </View>
)}

 <Input
 label="Título"
 placeholder={kind === 'exam' ? 'Ex: Hemograma completo' : 'Ex: ' + DOCUMENT_KIND_LABELS[kind]}
 value={title}
 onChangeText={setTitle}
 />

 <Input
 label="Data"
 placeholder="DD/MM/AAAA"
 value={date}
 onChangeText={t => setDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />

 <View style={styles.formActions}>
 {editingDoc && (
 <TouchableOpacity style={styles.cancelBtn} onPress={cancelForm} activeOpacity={0.7}>
 <Text style={styles.cancelBtnText}>Cancelar</Text>
 </TouchableOpacity>
)}
 <View style={{ flex: 1 }}>
 <Button
 label={editingDoc ? 'Salvar Alterações' : 'Salvar Documento'}
 onPress={handleSave}
 loading={saving}
 />
 </View>
 </View>
 </View>
) : (
 <FlatList
 data={kindDocs}
 keyExtractor={item => item.id}
 contentContainerStyle={styles.list}
 showsVerticalScrollIndicator={false}
 renderItem={({ item }) => (
 <TouchableOpacity
 style={styles.docRow}

```

```

 onPress={() => openDocument(item)}
 onLongPress={() => showOptions(item)}
 activeOpacity={0.7}
 >
 <View style={styles.docIcon}>
 <Ionicons name={KIND_ICONS[item.kind]} size={18} color={KIND_COLORS[item.kind]} />
 </View>
 <View style={{ flex: 1 }}>
 <Text style={styles.docTitle}>{item.title}</Text>
 <Text style={styles.docMeta}>{displayDate(item.date)}</Text>
 </View>
 <TouchableOpacity
 onPress={() => showOptions(item)}
 hitSlop={{ top: 10, bottom: 10, left: 10, right: 10 }}
 accessibilityRole="button"
 accessibilityLabel="Opções do documento"
 >
 <Ionicons name="ellipsis-vertical" size={18} color={colors.textMuted} />
 </TouchableOpacity>
 </TouchableOpacity>
)}
 ListEmptyComponent={
 <View style={styles.emptyWrapper}>
 <EmptyState
 icon={KIND_ICONS[kind]}
 title={`Nenhum documento em ${DOCUMENT_KIND_LABELS[kind]}`}
 text="Toque em + para anexar um PDF ou imagem. Toque para visualizar ou use o menu (:) para editar, compartilhar ou
excluir."
 />
 </View>
 }
 />
)}
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 categories: { padding: spacing.lg, gap: spacing.sm },
 categoryCard: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 },
 categoryIcon: {
 width: 44,
 height: 44,
 borderRadius: radius.full,
 alignItems: 'center',
 justifyContent: 'center',
 },
 categoryTitle: { fontSize: 15, fontWeight: '600', color: colors.text },
 categoryHint: { fontSize: 12, color: colors.textMuted, marginTop: 2 },
 categoryCount: {
 minWidth: 26,
 height: 26,
 borderRadius: 13,
 backgroundColor: colors.surfaceElevated,
 alignItems: 'center',
 justifyContent: 'center',
 paddingHorizontal: 6,
 },
 categoryCountText: { fontSize: 12, fontWeight: '600', color: colors.textSubtle },
 list: { paddingHorizontal: spacing.lg, paddingBottom: 48 },
 emptyWrapper: { padding: spacing.xxl },
 form: { padding: spacing.lg, gap: spacing.lg },
 fileRow: {

```

```

flexDirection: 'row',
alignItems: 'center',
gap: spacing.sm,
backgroundColor: colors.surface,
borderRadius: radius.md,
borderWidth: 1,
borderColor: colors.border,
padding: spacing.md,
},
fileName: { flex: 1, fontSize: 13, color: colors.textSubtle },
formActions: { flexDirection: 'row', gap: spacing.sm },
cancelBtn: {
 paddingHorizontal: spacing.lg,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 alignItems: 'center',
 justifyContent: 'center',
},
cancelBtnText: { fontSize: 16, fontWeight: '600', color: colors.textMuted },
docRow: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 marginBottom: spacing.sm,
},
docIcon: {
 width: 40,
 height: 40,
 borderRadius: radius.md,
 backgroundColor: colors.surfaceElevated,
 alignItems: 'center',
 justifyContent: 'center',
},
docTitle: { fontSize: 14, fontWeight: '600', color: colors.text },
docMeta: { fontSize: 12, color: colors.textMuted, marginTop: 2 },
});

```

## src/screens/DocumentViewerScreen.tsx

```
import React from 'react';
import { View, Text, Image, TouchableOpacity, StyleSheet, Alert } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp } from '@react-navigation/native';
import * as Sharing from 'expo-sharing';
import { spacing, useTheme, useThemedStyles, Palette } from '../theme';
import { RootStackParamList } from '../types';

type Route = RouteProp<RootStackParamList, 'DocumentViewer'>;

/** Visualizador de imagens dentro do app (sem sair para compartilhar). */
export default function DocumentViewerScreen() {
 const navigation = useNavigation();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { uri, title, mimeType } = useRoute<Route>().params;

 async function handleShare() {
 try {
 await Sharing.shareAsync(uri, { mimeType });
 } catch {
 Alert.alert('Erro', 'Não foi possível compartilhar o documento.');
```

## src/screens/StatsScreen.tsx

```
import React, { useState, useCallback } from 'react';
import { View, Text, ScrollView, TouchableOpacity, StyleSheet, Alert } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useRoute, RouteProp, useFocusEffect } from '@react-navigation/native';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getRecords, getWeights } from '../storage';
import { RECORD_TYPE_LABELS, recordTypeColors, RECORD_TYPE_ICONS } from '../labels';
import { WeightChart } from '../components/WeightChart';
import { ThemeToggle } from '../components/ThemeToggle';
import { EmptyState } from '../components/EmptyState';
import { MedicalRecord, WeightEntry, RecordType, RootStackParamList } from '../types';

type Route = RouteProp<RootStackParamList, 'Stats'>;

const TYPES: RecordType[] = ['vaccine', 'consultation', 'medication', 'deworming', 'note'];

export default function StatsScreen() {
 const navigation = useNavigation();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const RECORD_TYPE_COLORS = recordTypeColors(colors);
 const { petId } = useRoute<Route>().params;
 const [records, setRecords] = useState<MedicalRecord[]>([]);
 const [weights, setWeights] = useState<WeightEntry[]>([]);

 useFocusEffect(
 useCallback(() => {
 Promise.all([getRecords(petId).then(setRecords), getWeights(petId).then(setWeights)]).catch(
 () => Alert.alert('Erro', 'Não foi possível carregar as estatísticas.'),
);
 }, [petId]),
);

 const counts = TYPES.map(type => ({
 type,
 count: records.filter(r => r.type === type).length,
 }));
 const maxCount = Math.max(1, ...counts.map(c => c.count));
 const total = records.length + weights.length;

 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Ionicons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>Estatísticas</Text>
 <ThemeToggle />
 </View>

 <ScrollView contentContainerStyle={styles.scroll} showsVerticalScrollIndicator={false}>
 <View style={styles.totalCard}>
 <Text style={styles.totalValue}>{total}</Text>
 <Text style={styles.totalLabel}>
 {total === 1 ? 'registro no prontuário' : 'registros no prontuário'}
 </Text>
 </View>

 {total === 0 && (
 <EmptyState
 icon="bar-chart-outline"
 title="Nenhum registro ainda"
 text="Adicione vacinas, consultas, remédios ou pesagens para acompanhar as estatísticas aqui."
 />
)}

 <View style={styles.card}>
 <Text style={styles.cardTitle}>Registros por tipo</Text>
 {counts.map(({ type, count }) => (
 <View key={type} style={styles.barRow}>
 <View style={styles.barLabelWrapper}>

```

```

 <Icons name={RECORD_TYPE_ICONS[type]} size={13} color={RECORD_TYPE_COLORS[type]} />
 <Text style={styles.barLabel}>{RECORD_TYPE_LABELS[type]}</Text>
 </View>
 <View style={styles.barTrack}>
 <View
 style={[
 styles.barFill,
 {
 backgroundColor: RECORD_TYPE_COLORS[type],
 width: `${Math.max(3, (count / maxCount) * 100)}%`,
 opacity: count === 0 ? 0.15 : 1,
 },
]}
 />
 </View>
 <Text style={styles.barCount}>{count}</Text>
 </View>
)))
 <View style={styles.barRow}>
 <View style={styles.barLabelWrapper}>
 <Icons name="scale" size={13} color={colors.primaryLight} />
 <Text style={styles.barLabel}>Pesagens</Text>
 </View>
 <View style={styles.barTrack}>
 <View
 style={[
 styles.barFill,
 {
 backgroundColor: colors.primaryLight,
 width: `${Math.max(3, (weights.length / Math.max(maxCount, weights.length)) * 100)}%`,
 opacity: weights.length === 0 ? 0.15 : 1,
 },
]}
 />
 </View>
 <Text style={styles.barCount}>{weights.length}</Text>
 </View>
</View>

{weights.length >= 2 ? (
 <WeightChart weights={weights} title="Evolução de peso" />
) : (
 <View style={styles.card}>
 <Text style={styles.cardTitle}>Evolução de peso</Text>
 <Text style={styles.hint}>
 Registre pelo menos duas pesagens para ver o gráfico de evolução.
 </Text>
 </View>
)}
</ScrollView>
</SafeAreaView>
);
}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: { fontSize: 16, fontWeight: '600', color: colors.text },
 scroll: { padding: spacing.lg, gap: spacing.md, paddingBottom: 48 },
 totalCard: {
 alignItems: 'center',
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.lg,
 },
 totalValue: { fontSize: 40, fontWeight: '700', color: colors.primaryLight },
 totalLabel: { fontSize: 13, color: colors.textMuted },
 card: {
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 padding: spacing.md,
 },
});

```

```
 gap: spacing.sm,
 },
 cardTitle: { fontSize: 13, fontWeight: '600', color: colors.textSubtle, marginBottom: spacing.xs },
 barRow: { flexDirection: 'row', alignItems: 'center', gap: spacing.sm },
 barLabelWrapper: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.xs,
 width: 104,
 },
 barLabel: { fontSize: 12, color: colors.textSubtle },
 barTrack: {
 flex: 1,
 height: 14,
 borderRadius: 7,
 backgroundColor: colors.background,
 overflow: 'hidden',
 },
 barFill: { height: '100%', borderRadius: 7 },
 barCount: { width: 24, fontSize: 13, fontWeight: '600', color: colors.text, textAlign: 'right' },
 hint: { fontSize: 13, color: colors.textMuted, lineHeight: 19 },
 });
```

## src/screens/SearchScreen.tsx

```
import React, { useState, useEffect } from 'react';
import {
 View,
 Text,
 FlatList,
 TextInput,
 TouchableOpacity,
 StyleSheet,
 Alert,
} from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation } from '@react-navigation/native';
import { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { spacing, radius, useTheme, useThemedStyles, Palette } from '../theme';
import { getPets, getAllRecords } from '../storage';
import { isActiveMedication } from '../services/events';
import { maskDate, isValidDate, toISO } from '../utils/date';
import { RECORD_TYPE_LABELS } from '../labels';
import { HealthRecordCard } from '../components/HealthRecordCard';
import { EmptyState } from '../components/EmptyState';
import { Pet, MedicalRecord, RecordType, RootStackParamList } from '../types';

type Nav = NativeStackNavigationProp<RootStackParamList, 'Search'>;

const TYPES: RecordType[] = ['vaccine', 'consultation', 'medication', 'deworming', 'note'];

function matchesQuery(record: MedicalRecord, query: string): boolean {
 const q = query.toLowerCase();
 return [record.title, record.description, record.diagnosis, record.vet, record.clinic]
 .filter(Boolean)
 .some(field => field!.toLowerCase().includes(q));
}

export default function SearchScreen() {
 const navigation = useNavigation<Nav>();
 const { colors } = useTheme();
 const styles = useThemedStyles(createStyles);
 const [pets, setPets] = useState<Pet[]>([]);
 const [records, setRecords] = useState<MedicalRecord[]>([]);
 const [query, setQuery] = useState('');
 const [typeFilter, setTypeFilter] = useState<RecordType | null>(null);
 const [fromDate, setFromDate] = useState('');
 const [toDate, setToDate] = useState('');
 const [onlyActiveMeds, setOnlyActiveMeds] = useState(false);

 useEffect(() => {
 Promise.all([getPets().then(setPets), getAllRecords().then(setRecords)]).catch(() =>
 Alert.alert('Erro', 'Não foi possível carregar os registros.'),
);
 }, []);

 const petNames = new Map(pets.map(p => [p.id, p.name]));

 const results = records
 .filter(r => !typeFilter || r.type === typeFilter)
 .filter(r => !query.trim() || matchesQuery(r, query.trim()))
 .filter(r => !(fromDate.length === 10 && isValidDate(fromDate)) || r.date >= toISO(fromDate))
 .filter(r => !(toDate.length === 10 && isValidDate(toDate)) || r.date <= toISO(toDate))
 .filter(r => !onlyActiveMeds || isActiveMedication(r))
 .sort((a, b) => b.date.localeCompare(a.date));

 const hasFilter =
 query.trim() || typeFilter || onlyActiveMeds || (fromDate.length === 10 && isValidDate(fromDate)) ||
 (toDate.length === 10 && isValidDate(toDate));

 const fromValid = fromDate.length === 10 && isValidDate(fromDate);
 const toValid = toDate.length === 10 && isValidDate(toDate);
 const dateRangeInvalid = fromValid && toValid && toISO(fromDate) > toISO(toDate);

 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}
 accessibilityRole="button"
 />
 </View>
 </SafeAreaView>
);
}
```

```

 accessibilityLabel="Voltar"
 >
 <Icons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <View style={styles.searchBox}>
 <Icons name="search" size={16} color={colors.textMuted} />
 <TextInput
 style={styles.searchInput}
 placeholder="Buscar vacinas, remédios, consultas..."
 placeholderTextColor={colors.textMuted}
 value={query}
 onChangeText={setQuery}
 autoFocus
 returnKeyType="search"
 />
 {query ? (
 <TouchableOpacity onPress={() => setQuery('')}>
 <Icons name="close-circle" size={16} color={colors.textMuted} />
 </TouchableOpacity>
) : null}
 </View>
 </View>

 <View style={styles.filters}>
 <View style={styles.chips}>
 {TYPES.map(t => {
 const active = typeFilter === t;
 return (
 <TouchableOpacity
 key={t}
 style={[styles.chip, active && styles.chipActive]}
 onPress={() => setTypeFilter(active ? null : t)}
 activeOpacity={0.7}
 >
 <Text style={[styles.chipText, active && styles.chipTextActive]}>
 {RECORD_TYPE_LABELS[t]}
 </Text>
 </TouchableOpacity>
);
 })}
 <TouchableOpacity
 style={[styles.chip, onlyActiveMeds && styles.chipActive]}
 onPress={() => setOnlyActiveMeds(v => !v)}
 activeOpacity={0.7}
 >
 <Text style={[styles.chipText, onlyActiveMeds && styles.chipTextActive]}>
 Remédios em uso
 </Text>
 </TouchableOpacity>
 </View>
 <View style={styles.dateRow}>
 <View style={styles.dateBox}>
 <Text style={styles.dateLabel}>De</Text>
 <TextInput
 style={styles.dateInput}
 placeholder="DD/MM/AAAA"
 placeholderTextColor={colors.textMuted}
 value={fromDate}
 onChangeText={t => setFromDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />
 </View>
 <View style={styles.dateBox}>
 <Text style={styles.dateLabel}>Até</Text>
 <TextInput
 style={styles.dateInput}
 placeholder="DD/MM/AAAA"
 placeholderTextColor={colors.textMuted}
 value={toDate}
 onChangeText={t => setToDate(maskDate(t))}
 keyboardType="numeric"
 maxLength={10}
 />
 </View>
 </View>
 {dateRangeInvalid && (
 <Text style={styles.dateWarning}>A data "De" é posterior à data "Até".</Text>
)}
 </View>

```

```

<FlatList
 data={results}
 keyExtractor={item => item.id}
 contentContainerStyle={styles.list}
 showsVerticalScrollIndicator={false}
 keyboardShouldPersistTaps="handled"
 renderItem={({ item }) => (
 <View style={{ marginBottom: spacing.sm }}>
 <HealthRecordCard
 record={item}
 petName={petNames.get(item.petId)}
 onPress={() =>
 navigation.navigate('AddRecord', { petId: item.petId, recordId: item.id })
 }
 />
 </View>
)}
 ListEmptyComponent={
 <View style={styles.emptyWrapper}>
 <EmptyState
 icon="search-outline"
 title={hasFilter ? 'Nada encontrado' : 'Busque no prontuário'}
 text={
 hasFilter
 ? 'Tente outros termos ou ajuste os filtros de tipo e data.'
 : 'Pesquise por nome de vacina, remédio, diagnóstico, veterinário ou clínica.'
 }
 />
 </View>
)
 />
</SafeAreaView>
);
}

```

```

const createStyles = (colors: Palette) => StyleSheet.create({
 container: { flex: 1, backgroundColor: colors.background },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.md,
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 searchBar: {
 flex: 1,
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.sm,
 backgroundColor: colors.surface,
 borderRadius: radius.full,
 borderWidth: 1,
 borderColor: colors.border,
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.sm,
 },
 searchInput: { flex: 1, fontSize: 14, color: colors.text, padding: 0 },
 filters: { paddingHorizontal: spacing.lg, gap: spacing.sm, marginBottom: spacing.md },
 chips: { flexDirection: 'row', gap: spacing.sm, flexWrap: 'wrap' },
 chip: {
 paddingHorizontal: spacing.md,
 paddingVertical: spacing.xs + 2,
 borderRadius: radius.full,
 backgroundColor: colors.surface,
 borderWidth: 1,
 borderColor: colors.border,
 },
 chipActive: { backgroundColor: colors.primary, borderColor: colors.primary },
 chipText: { fontSize: 12, color: colors.textMuted, fontWeight: '500' },
 chipTextActive: { color: '#fff' },
 dateRow: { flexDirection: 'row', gap: spacing.sm },
 dateBox: {
 flex: 1,
 flexDirection: 'row',
 alignItems: 'center',
 gap: spacing.sm,
 backgroundColor: colors.surface,
 borderRadius: radius.md,
 borderWidth: 1,
 borderColor: colors.border,
 paddingHorizontal: spacing.md,
 },
});

```

```
paddingVertical: spacing.sm,
},
dateWarning: { fontSize: 12, color: colors.warning },
dateLabel: { fontSize: 12, color: colors.textMuted, width: 24 },
dateInput: { flex: 1, fontSize: 13, color: colors.text, padding: 0 },
list: { paddingHorizontal: spacing.lg, paddingBottom: 48 },
emptyWrapper: { paddingTop: spacing.xl },
});
```

## src/screens/AboutScreen.tsx

```
import React, { useState, useCallback } from 'react';
import { View, Text, ScrollView, TouchableOpacity, StyleSheet, Image, ActivityIndicator } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation, useFocusEffect } from '@react-navigation/native';
import Constants from 'expo-constants';
import * as FileSystem from 'expo-file-system/legacy';
import * as Sharing from 'expo-sharing';
import * as DocumentPicker from 'expo-document-picker';
import { Alert } from 'react-native';
import { spacing, radius, typography, useTheme, useThemedStyles, Palette } from '../theme';
import { backupRepository } from '../repositories/backupRepository';
import { petsRepository } from '../repositories/petsRepository';
import { recordsRepository } from '../repositories/recordsRepository';
import { displayDate } from '../utils/date';
import { useToast } from '../hooks/useToast';
import { ThemeToggle } from '../components/ThemeToggle';
import { Button } from '../components/Button';

const FEATURES = [
 { image: require('../assets/icons/shield.png'), label: 'Proteção', text: 'Segurança' },
 { image: require('../assets/icons/paw.png'), label: 'Pet', text: 'Amor' },
 { image: require('../assets/icons/document.png'), label: 'Histórico', text: 'Organização' },
 { image: require('../assets/icons/heart.png'), label: 'Cuidado', text: 'Bem-estar' },
];

export default function AboutScreen() {
 const navigation = useNavigation();
 const { colors, scheme } = useTheme();
 const styles = useThemedStyles(createStyles);
 const { showToast } = useToast();
 const version = Constants.expoConfig?.version ?? '1.0.0';
 const [exporting, setExporting] = useState(false);
 const [importing, setImporting] = useState(false);
 const [loadingDemo, setLoadingDemo] = useState(false);
 const [lastBackup, setLastBackup] = useState<string | null>(null);
 const busy = exporting || importing || loadingDemo;

 useFocusEffect(
 useCallback(() => {
 backupRepository.getLastBackupDate().then(setLastBackup).catch(() => {});
 }, []),
);

 async function handleExport() {
 setExporting(true);
 try {
 const json = await backupRepository.export();
 const date = new Date().toISOString().slice(0, 10);
 const uri = `${FileSystem.cacheDirectory}petcare-backup-${date}.json`;
 await FileSystem.writeAsStringAsync(uri, json);
 if (await Sharing.isAvailableAsync()) {
 await Sharing.shareAsync(uri, {
 mimeType: 'application/json',
 dialogTitle: 'Salvar backup do PetCare',
 });
 }
 setLastBackup(await backupRepository.getLastBackupDate());
 showToast('Backup exportado com sucesso');
 } catch {
 Alert.alert('Erro', 'Não foi possível gerar o backup.');
```

```
 } finally {
 setExporting(false);
 }
 }

 async function handleImport() {
 const result = await DocumentPicker.getDocumentAsync({
 type: 'application/json',
 copyToCacheDirectory: true,
 });
 if (result.canceled) return;

 let json: string;
 try {
 json = await FileSystem.readAsStringAsync(result.assets[0].uri);
 } catch {
```

```

 Alert.alert('Erro', 'Não foi possível ler o arquivo selecionado.');
```

```

 return;
 }

 let summary: { pets: number; records: number; exportedAt?: string };
 try {
 summary = backupRepository.peekSummary(json);
 } catch {
 Alert.alert('Erro', 'Arquivo de backup inválido ou corrompido.');
```

```

 return;
 }

 const [currentPets, currentRecords] = await Promise.all([petsRepository.getAll(), recordsRepository.getAll()]);
 const exportedAtText = summary.exportedAt ? displayDate(summary.exportedAt.slice(0, 10)) : 'data desconhecida';

 Alert.alert(
 'Restaurar backup',
 `Este backup tem ${summary.pets} ${summary.pets === 1 ? 'pet' : 'pets'} e ${summary.records} ${summary.records === 1 ?
'registro' : 'registros'}, exportado em ${exportedAtText}.\\n\\n` +
 `Seus dados atuais (${currentPets.length} ${currentPets.length === 1 ? 'pet' : 'pets'}, ${currentRecords.length}
${currentRecords.length === 1 ? 'registro' : 'registros'}) serão substituídos. Fotos e documentos só continuam disponíveis se este
backup for do mesmo aparelho.`,
 [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Restaurar',
 style: 'destructive',
 onPress: async () => {
 setImporting(true);
 try {
 const { pets, records } = await backupRepository.import(json);
 showToast(
 `${pets} ${pets === 1 ? 'pet' : 'pets'} e ${records} ${records === 1 ? 'registro' : 'registros'} importados`,
);
 } catch {
 Alert.alert('Erro', 'Arquivo de backup inválido ou corrompido.');
```

```

 } finally {
 setImporting(false);
 }
 },
],
);
 }

 function handleLoadDemoData() {
 Alert.alert(
 'Carregar dados de demonstração',
 'Isso substitui TODOS os seus dados atuais por pets, registros e pesagens fictícios, pensados para demonstrar o app. Esta
ação não pode ser desfeita. Continuar?',
 [
 { text: 'Cancelar', style: 'cancel' },
 {
 text: 'Carregar',
 style: 'destructive',
 onPress: async () => {
 setLoadingDemo(true);
 try {
 await backupRepository.loadDemoData();
 showToast('Dados de demonstração carregados');
```

```

 } catch {
 Alert.alert('Erro', 'Não foi possível carregar os dados de demonstração.');
```

```

 } finally {
 setLoadingDemo(false);
 }
 },
],
),
);
 }

 const lastBackupText = lastBackup
 ? `Último backup: ${displayDate(lastBackup.slice(0, 10))} às ${lastBackup.slice(11, 16)}`
 : 'Nenhum backup ainda';

 return (
 <SafeAreaView style={styles.container} edges={['top']}>
 <View style={styles.header}>
 <TouchableOpacity
 onPress={() => navigation.goBack()}
 hitSlop={{ top: 12, bottom: 12, left: 12, right: 12 }}

```

```

 accessibilityRole="button"
 accessibilityLabel="Voltar"
 >
 <Icons name="arrow-back" size={24} color={colors.text} />
 </TouchableOpacity>
 <Text style={styles.headerTitle}>Sobre</Text>
 <ThemeToggle />
</View>

<ScrollView contentContainerStyle={styles.scroll} showsVerticalScrollIndicator={false}>
 <Image
 source={
 scheme === 'dark'
 ? require('../assets/logo-dark.png')
 : require('../assets/logo-light.png')
 }
 style={styles.logo}
 resizeMode="contain"
 />
 <Text style={styles.slogan}>Cuidar é o nosso compromisso ♡</Text>

 <View style={styles.features}>
 {FEATURES.map(f => (
 <View key={f.label} style={styles.feature}>
 <Image source={f.image} style={styles.featureIcon} resizeMode="contain" />
 <Text style={styles.featureLabel}>{f.label}</Text>
 <Text style={styles.featureText}>{f.text}</Text>
 </View>
))}
 </View>

 <Text style={styles.description}>
 O PetCare guarda o prontuário completo dos seus pets: vacinas, consultas, medicamentos,
 peso, exames e documentos – tudo organizado e sempre à mão, inclusive em emergências.
 </Text>

 <View style={styles.backupSection}>
 <Text style={styles.backupTitle}>Seus dados</Text>
 <Text style={styles.lastBackup}>{lastBackupText}</Text>
 <Button
 label={exporting ? 'Gerando backup...' : 'Exportar backup'}
 icon="download-outline"
 variant="secondary"
 onPress={handleExport}
 loading={exporting}
 disabled={busy && !exporting}
 />
 <Button
 label={importing ? 'Restaurando...' : 'Restaurar backup'}
 icon="refresh-outline"
 variant="secondary"
 onPress={handleImport}
 loading={importing}
 disabled={busy && !importing}
 />
 <Text style={styles.backupHint}>
 O backup é um arquivo JSON com pets, registros, pesagens e documentos. Os arquivos de
 fotos e documentos ficam salvos neste aparelho e não viajam dentro do backup.
 </Text>
 </View>

 <View style={styles.backupSection}>
 <Text style={styles.backupTitle}>Modo demonstração</Text>
 <Button
 label={loadingDemo ? 'Carregando...' : 'Carregar dados de demonstração'}
 icon="sparkles-outline"
 variant="secondary"
 onPress={handleLoadDemoData}
 loading={loadingDemo}
 disabled={busy && !loadingDemo}
 />
 <Text style={styles.backupHint}>
 Preenche o app com pets fictícios e histórico de exemplo – útil para explorar as
 funcionalidades. Substituí seus dados atuais.
 </Text>
 </View>

 <Text style={styles.version}>Versão {version}</Text>
</ScrollView>
</SafeAreaView>
);

```

```

}

const createStyles = (colors: Palette) => StyleSheet.create({
 container: {
 flex: 1,
 backgroundColor: colors.background,
 },
 header: {
 flexDirection: 'row',
 alignItems: 'center',
 justifyContent: 'space-between',
 paddingHorizontal: spacing.lg,
 paddingVertical: spacing.md,
 },
 headerTitle: {
 fontSize: typography.h4.fontSize,
 fontWeight: typography.h4.fontWeight,
 color: colors.text,
 },
 scroll: {
 alignItems: 'center',
 paddingHorizontal: spacing.xl,
 paddingTop: spacing.xxl,
 paddingBottom: 48,
 },
 logo: {
 width: 240,
 height: 80,
 },
 slogan: {
 marginTop: spacing.sm,
 fontSize: 14,
 color: colors.textSubtle,
 },
 features: {
 flexDirection: 'row',
 marginTop: spacing.xxl,
 marginBottom: spacing.xl,
 gap: spacing.md,
 },
 feature: {
 flex: 1,
 alignItems: 'center',
 backgroundColor: colors.surface,
 borderRadius: radius.lg,
 borderWidth: 1,
 borderColor: colors.border,
 paddingVertical: spacing.md,
 paddingHorizontal: spacing.xs,
 gap: 4,
 },
 featureIcon: {
 width: 28,
 height: 28,
 marginBottom: 2,
 },
 featureLabel: {
 fontSize: 12,
 fontWeight: '600',
 color: colors.text,
 },
 featureText: {
 fontSize: 11,
 color: colors.textMuted,
 },
 description: {
 fontSize: 14,
 lineHeight: 22,
 color: colors.textMuted,
 textAlign: 'center',
 },
 backupSection: {
 width: '100%',
 marginTop: spacing.xl,
 gap: spacing.sm,
 },
 backupTitle: {
 fontSize: 14,
 fontWeight: '600',
 color: colors.text,
 marginBottom: 2,
 },

```

```
},
lastBackup: {
 fontSize: 12,
 color: colors.textSubtle,
 marginBottom: spacing.xs,
},
backupHint: {
 fontSize: 11,
 lineHeight: 16,
 color: colors.textSubtle,
},
version: {
 marginTop: spacing.xl,
 fontSize: 12,
 color: colors.textSubtle,
},
});
```